

Hat

Table of contents

English version	10
Description	10
Quick start	10
System requirements	10
Installation and launch	11
Creating a simple project	12
Creating a project compatible with Visual Studio	14
Launching the autotest and result	18
Browser Cache	21
Clearing	21
Interface	22
The main menu	22
The Toolbar	24
The Project window - is the Explorer tab	24
The Project window - is the Test tab	28
The Project window - the Library tab	29
The Project window - the Mail tab	30
The system window	31
Reports	32
Autotest Report	32
Log file	35
Mail	36
Send a report on the failure of the autotest to the mail	36
Console launch mode (Jenkins)	38
Running the autotest from the command line	38
A text code editor	42
Code Editor (C#)	42
Using Visual Studio to edit code	46
The structure of the autotest	50
Simple project	50
The PageObjects and StepObjects pattern	53
Creating a new autotest and patterns	56
HatFramework	60
Class: Tester	60
Constructor	60
Tester	60
Constants	61
BY_CSS	61
BY_XPATH	61
IMAGE_STATUS_PROCESS	61
IMAGE_STATUS_PASSED	61
IMAGE_STATUS_FAILED	62
IMAGE_STATUS_MESSAGE	62
IMAGE_STATUS_WARNING	62
IMAGE_STATUS_DEBUG	62
COMPLETED	63
FAILED	63

PASSED	63
PROCESS	63
STOPPED	63
WARNING	64
DEBUG	64
DEFAULT	64
UTF8	64
UTF8BOM	65
WINDOWS1251	65
Variables	65
BrowserView	65
BrowserWindow	66
Locators	66
Locator	66
AddLocator	67
GetLocators	67
GetLocator	67
GetLocatorValue	67
GetCountLocators	68
ClearLocators	68
RemoveLocator	68
Methods for working with the browser	68
BrowserBasicAuthenticationAsync	68
BrowserClearNetworkAsync	69
BrowserCloseAsync	69
BrowserEnableSendMailAsync	69
BrowserFullScreenAsync	70
BrowserScreenshotAsync	70
BrowserSizeAsync	70
BrowserGetUserAgentAsync	71
BrowserSetUserAgentAsync	71
BrowserGetErrorsAsync	71
BrowserGetNetworkAsync	71
BrowserGoBackAsync	72
BrowserGoForwardAsync	72
BrowserPageReloadAsync	73
Methods for displaying messages	73
ConsoleMsg	73
ConsoleMsgError	74
ClearMessage	74
Description	74
DisableDebugInReport	75
SendMessage	75
SendMessageDebug	76
Methods for preparing and completing testing	76
TestBeginAsync	76
TestEndAsync	76
TestStopAsync	77
GetTestResult	77
DefineTestStop	77

Methods for performing actions	78
Attributes	78
GetAttributeFromElementAsync	79
GetAttributeFromElementByClassAsync	80
GetAttributeFromElementByIdAsync	81
GetAttributeFromElementByNameAsync	82
GetAttributeFromElementByTagAsync	83
GetAttributeFromElementsAsync	84
GetAttributeFromElementsByClassAsync	85
GetAttributeFromElementsByNameAsync	86
GetAttributeFromElementsByTagAsync	87
SetAttributeInElementAsync	88
SetAttributeInElementByClassAsync	89
SetAttributeInElementByIdAsync	90
SetAttributeInElementByNameAsync	91
SetAttributeInElementByTagAsync	92
SetAttributeInElementsAsync	93
SetAttributeInElementsByClassAsync	94
SetAttributeInElementsByNameAsync	95
SetAttributeInElementsByTagAsync	96
Values	96
GetValueFromElementAsync	97
GetValueFromElementByClassAsync	98
GetValueFromElementByIdAsync	99
GetValueFromElementByNameAsync	100
GetValueFromElementByTagAsync	101
SetValueInElementAsync	102
SetValueInElementByClassAsync	103
SetValueInElementByIdAsync	104
SetValueInElementByNameAsync	105
SetValueInElementByTagAsync	106
Clicking	106
ClickElementAsync	107
ClickElementByClassAsync	108
ClickElementByIdAsync	109
ClickElementByNameAsync	110
ClickElementByTagAsync	111
FocusElementAsync	112
IsClickableElementAsync	113
ScrollToElementAsync	114
Objects	114
GetElementAsync	115
GetFrameAsync	117
GetCountElementsAsync	119
GetCountElementsByClassAsync	120
GetCountElementsByNameAsync	121
GetCountElementsByTagAsync	122
GetHtmlFromElementAsync	123
GetHtmlFromElementByClassAsync	124
GetHtmlFromElementByIdAsync	125

GetHtmlFromElementByNameAsync	126
GetHtmlFromElementByTagAsync	127
IsVisibleElementAsync	128
MakeElementVisibleAsync	129
SetHtmlInElementAsync	130
SetHtmlInElementByClassAsync	131
SetHtmlInElementByIdAsync	132
SetHtmlInElementByNameAsync	133
SetHtmlInElementByTagAsync	134
Waiting	134
WaitAsync	135
WaitElementInDomAsync	136
WaitElementNotDomAsync	137
WaitNotVisibleElementAsync	138
WaitNotVisibleElementByClassAsync	139
WaitNotVisibleElementByIdAsync	140
WaitNotVisibleElementByNameAsync	141
WaitNotVisibleElementByTagAsync	142
WaitVisibleElementAsync	143
WaitVisibleElementByClassAsync	144
WaitVisibleElementByIdAsync	145
WaitVisibleElementByNameAsync	146
WaitVisibleElementByTagAsync	147
Search	147
FindElementAsync	148
FindElementByClassAsync	149
FindElementByIdAsync	150
FindElementByNameAsync	151
FindElementByTagAsync	152
FindVisibleElementAsync	153
FindVisibleElementByClassAsync	154
FindVisibleElementByIdAsync	155
FindVisibleElementByNameAsync	156
FindVisibleElementByTagAsync	157
Styles	157
GetStyleFromElementAsync	158
GetStyleFromElementByClassAsync	159
GetStyleFromElementByIdAsync	160
GetStyleFromElementByNameAsync	161
GetStyleFromElementByTagAsync	162
SetStyleInElementAsync	163
SetStyleInElementByClassAsync	164
SetStyleInElementByIdAsync	165
SetStyleInElementByNameAsync	166
SetStyleInElementByTagAsync	167
Page	167
GetCookiesAsync	168
GetListRedirectUrlAsync	169
GetTitleAsync	170
GetUrlAsync	171

GetUrlResponseAsync	172
GoToUrlAsync	173
GoToUrlBaseAuthAsync	174
LoadPageAsync	175
Text	175
GetTextFromElementAsync	176
GetTextFromElementByClassAsync	177
GetTextFromElementByIdAsync	178
GetTextFromElementByNameAsync	179
GetTextFromElementByTagAsync	180
SetTextInElementAsync	181
SetTextInElementByClassAsync	182
SetTextInElementByIdAsync	183
SetTextInElementByNameAsync	184
SetTextInElementByTagAsync	185
Methods for executing JavaScript	185
ExecuteJavaScriptAsync	185
Methods for executing Rest requests	185
RestGetAsync	185
RestGetBasicAuthAsync	186
RestGetStatusCodeAsync	187
RestPostAsync	187
Methods for measuring the time spent	188
TimerStart	188
TimerStop	188
Methods for sending email and message	189
SendMsgToMailAsync	189
SendMsgToTelegramAsync	189
SendMsgToBitrixChatAsync	190
Methods for checking the result	190
AssertEqualsAsync	190
AssertNotEqualsAsync	191
AssertTrueAsync	191
AssertFalseAsync	191
AssertNotNullAsync	192
AssertNullAsync	192
AssertNoErrorsAsync	192
AssertNetworkEventsAsync	193
Methods for working with files	193
FileDownloadAsync	193
FileGetHashMD5Async	193
FileReadAsync	194
FileWriteAsync	194
Methods for different tasks	194
CreateHashMD5FromTextAsync	194
GetProjectPath	194
Class: HTML_Element	195
Constructor	195
HTML_Element	195
Constants	195

BY_INDEX	195
BY_TEXT	195
BY_VALUE	196
Variables	196
Id	196
Name	196
Class	197
Type	197
Methods	197
ClickAsync	197
ClickMouseAsync	197
FocusAsync	198
GetAttributeAsync	198
GetHtmlAsync	198
GetLocatorAsync	198
GetOptionAsync	199
GetStyleAsync	199
GetTextAsync	199
GetValueAsync	200
IsClickableAsync	200
MakeVisibleAsync	200
ScrollToAsync	201
SelectOptionAsync	201
SetAttributeAsync	201
SetHtmlAsync	202
SetStyleAsync	202
SetTextAsync	202
SetValueAsync	202
WaitNotVisibleAsync	203
WaitVisibleAsync	203
Class: FRAMEElement	203
Constructor	203
FRAMEElement	203
Constants	204
BY_INDEX	204
BY_TEXT	204
BY_VALUE	204
Variables	204
Name	204
Index	205
Index	205
Methods	205
ClickElementAsync	205
FindElementAsync	205
FindVisibleElementAsync	206
GetAttributeFromElementAsync	206
GetAttributeFromElementsAsync	207
GetCountElementsAsync	207
GetHtmlFromElementAsync	207
GetOptionAsync	208

GetStyleFromElementAsync	208
GetTextFromElementAsync	209
GetTitleAsync	209
GetUrlAsync	209
GetValueFromElementAsync	210
IsClickableElementAsync	210
IsVisibleElementAsync	210
MakeVisibleAsync	211
ScrollToElementAsync	211
SelectOptionAsync	211
SetAttributeInElementAsync	212
SetAttributeInElementsAsync	212
SetHtmlInElementAsync	213
SetStyleInElementAsync	213
SetTextInElementAsync	213
SetValueInElementAsync	214
WaitNotVisibleElementAsync	214
WaitVisibleElementAsync	215
Plugins	215
HatPluginMySql	215
Installing the plugin	215
Connecting the plugin to the project	217
An example of an autotest	218
Class: TesterMySql	223
Constructor	223
TesterMySql	224
Methods	224
ConnectionOpenAsync	225
ConnectionCloseAsync	226
GetCountEntriesAsync	227
GetEntriesAsync	228
GetEntriesFromTableAsync	229
GetDataTableAsync	230
SetEntryAsync	231
EditEntryAsync	232
RemoveEntryAsync	233
FindEntryAsync	234
AssertHaveInTableAsync	235
AssertDontHaveInTableAsync	236
Practical examples	236
Autotest Group	236
Executing JavaScript code	243
Processing Json data using Newtonsoft	246
Processing XML data for checking the Sitemap	248
Alert, prompt and confirm dialog windows	252
Basic authorization	255
Google analytics and yandex metrika events	258
Version 1.3 (not current)	261
Description	261
SendMessage [changed since version 1.3.0]	261

SendMessageDebug [changed since version 1.3.0]	262
DefineTestStop [changed since version 1.3.0]	263

English version

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

Description

Hat

The peculiarity of the Hat browser is that autotests are performed directly in the browser without Selenium and WebDriver.

The built-in Net Framework contains a sufficient number of methods necessary to perform the basic tasks of test automation. The C# programming language and the built-in code editor are used to describe autotest scripts. You can also use Visual Studio as an editor. The user-friendly browser interface displays all the steps of the test with the corresponding description of sales. The verification result is generated in a report and sent to the specified email address. Autotests can be run from the command line of the Windows operating system. This happens when using autotests in popular continuous integration environments such as: Jenkins, TeamCity, GitLab CI/CD.

The official page: <https://somovstudio.github.io/>

Download the app from GitHub: <https://github.com/SomovStudio/Hat>

The version of this documentation corresponds to the browser version 1.5.2.4 (update 05/16/2026)

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

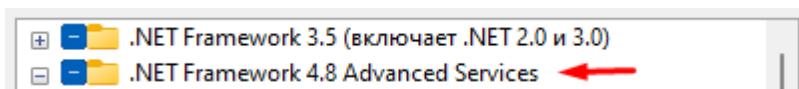
Quick start

Created with the Personal Edition of HelpNDoc: [Qt Help documentation made easy](#)

System requirements

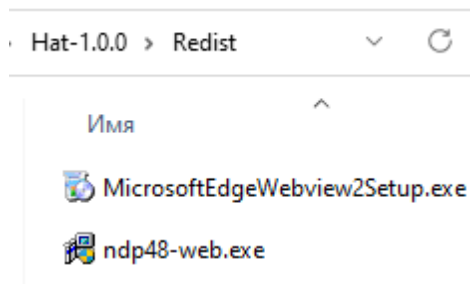
System requirements

1. The operating system Windows 8, 8.1, 10, 11
2. Package Microsoft .NET Framework 4.8 (download the installer [.NET Framework Runtime](#))



3. Package Microsoft Edge WebView2 (download the installer [WebView2 Runtime](#))

Note: the application archive in the Redist folder contains package installers Microsoft .NET Framework 4.8 и Microsoft Edge WebView2



Created with the Personal Edition of HelpNDoc: [Free iPhone documentation generator](#)

Installation and launch

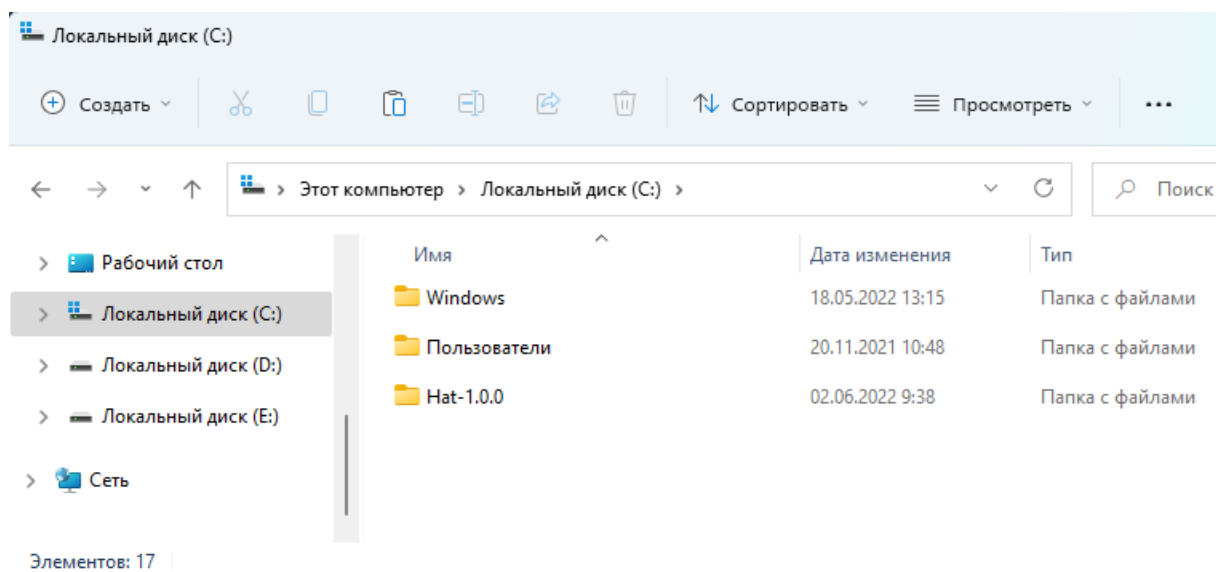
Installing and launching the browser Hat

1. Download the browser from the official GitHub page:
<https://github.com/SomovStudio/Hat/>

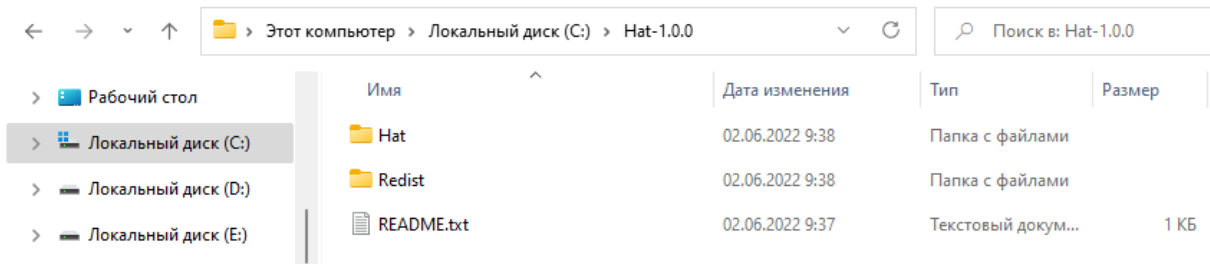
Check the archive with an antivirus program and if everything is in order, open the archive properties and turn on the "Unlock" flag so that the system does not ask you for permission every time you start the browser.

2. The application is completely portable and the downloaded archive can be unzipped to any location on the hard disk.

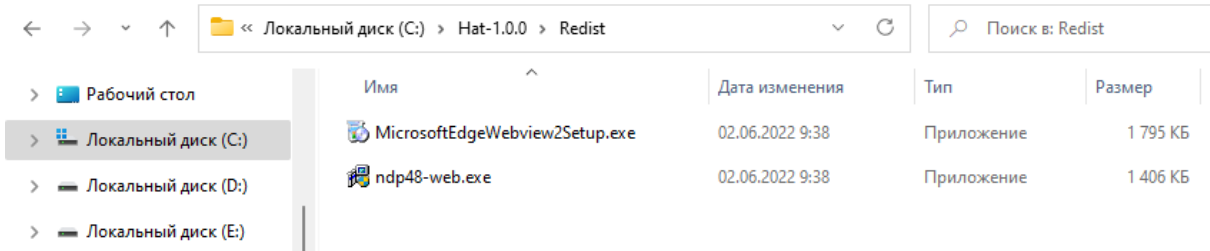
Tip: it is better not to install in the Program Files folder, otherwise you will have to give the application administrator rights, unzip it to the root of the disk.



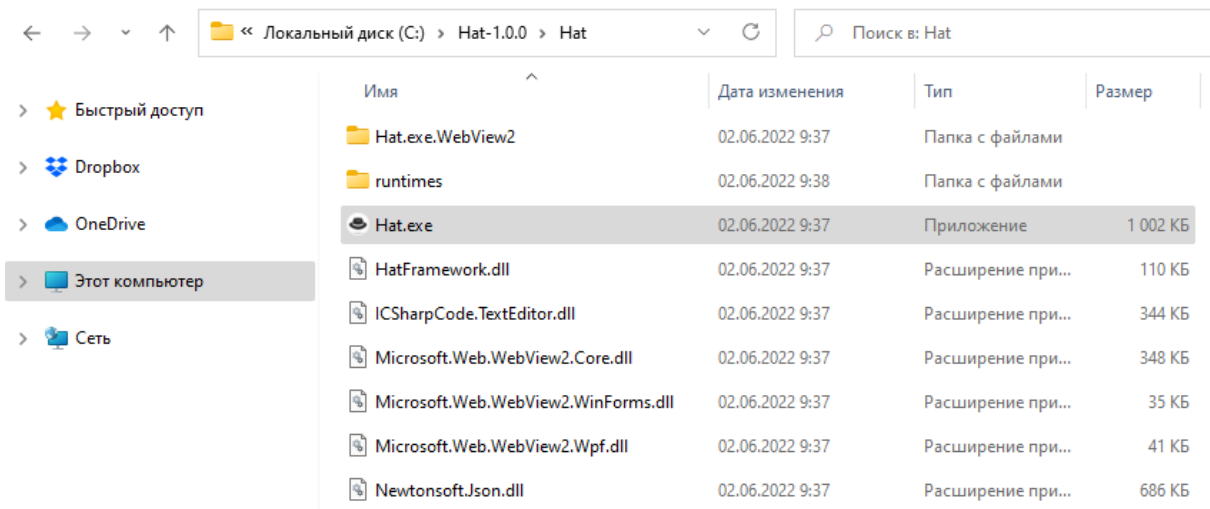
go to the folder Hat-1.0.0



3. The Redist folder contains the package installers Microsoft .NET Framework 4.8 и Microsoft Edge WebView2



4. The application itself is located in the Hat folder.



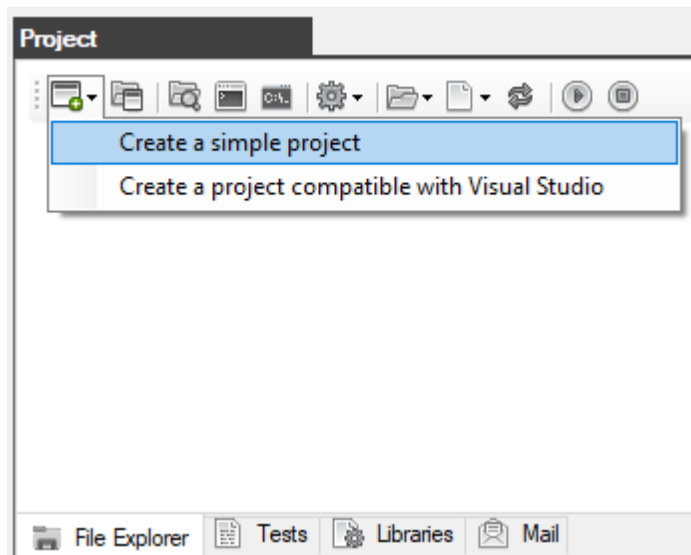
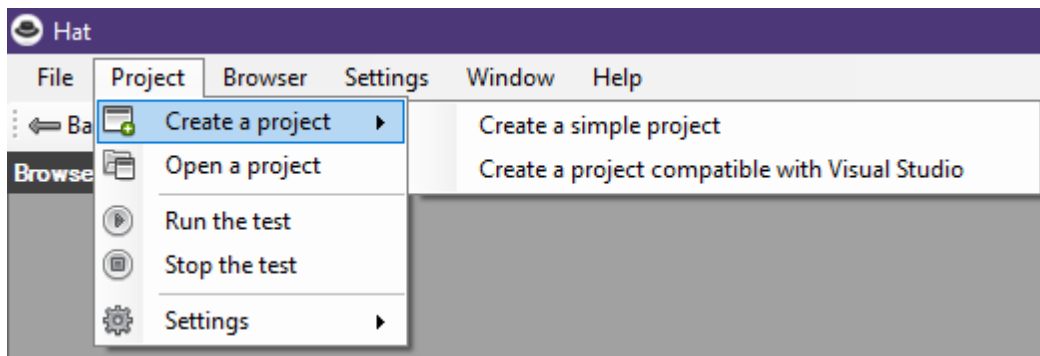
The browser is launched by a file Hat.exe

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

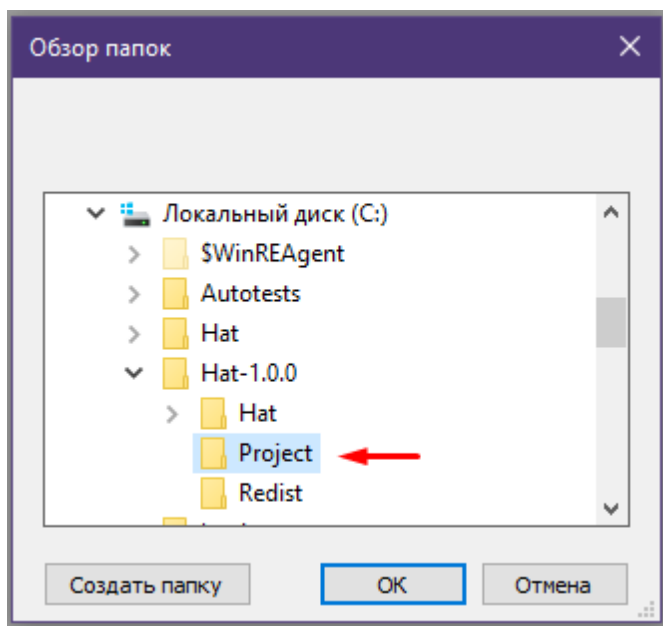
Creating a simple project

Creating a project and launching an autotest

1. In the Project window, on the Explorer tab or in the main Create Project menu, click the Create Simple Project button



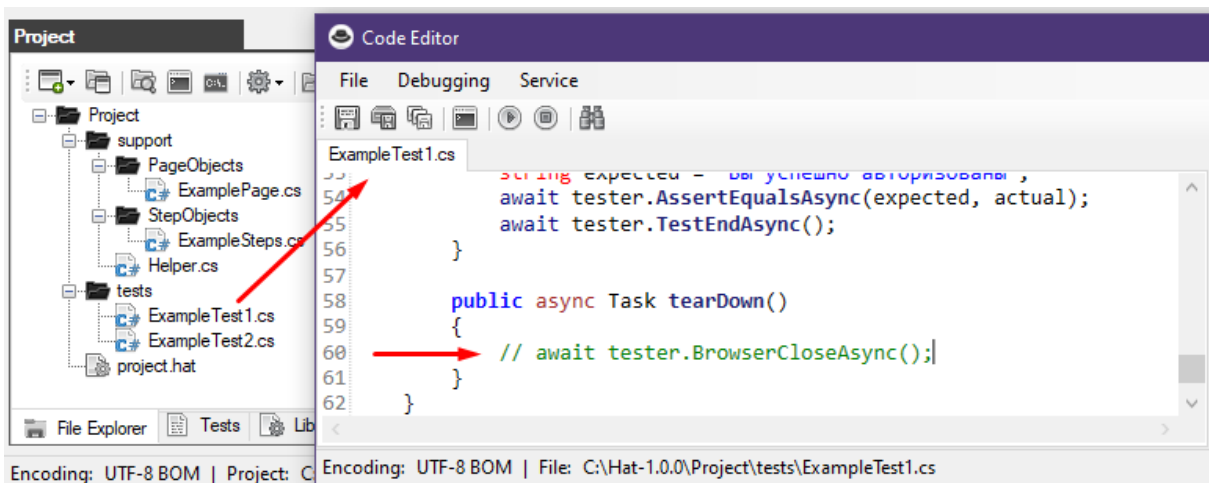
2. Create a folder that will contain the project with any name and anywhere on the disk. In this case, the Project folder has been created at C:\Hat-1.0.0\
Select the project folder and click OK



3. In the Project window, the contents of the project will be displayed on the Explorer tab. As you can see, the project is not empty, it contains several demo autotests.



4. Double-click on the ExampleTest1.cs file to open it in the code editor.
In line 60, a line is commented out with a call to the `BrowserCloseAsync()` method that closes the browser.
This is necessary so that the browser does not automatically close after the completion of the autotest.



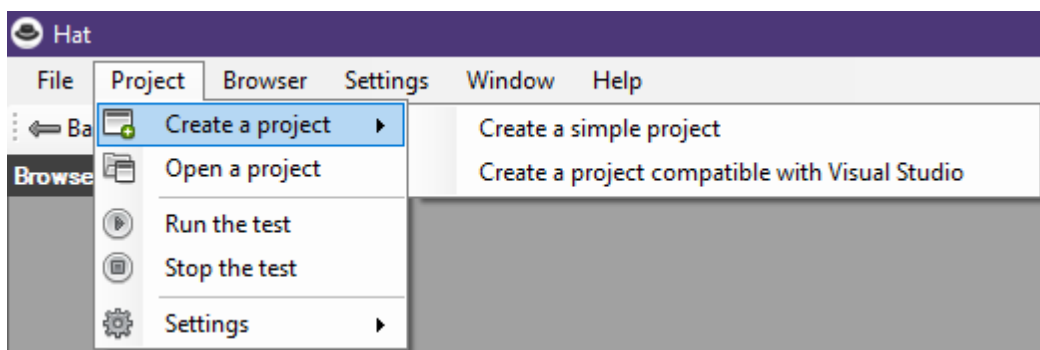
The project has been successfully created.

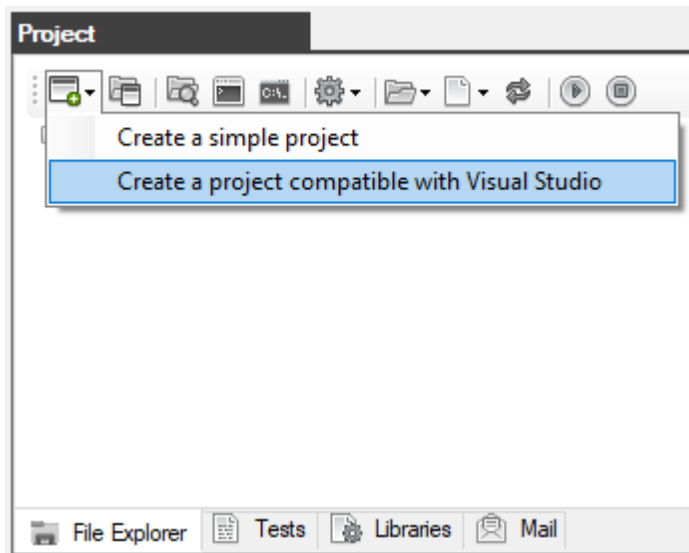
Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

Creating a project compatible with Visual Studio

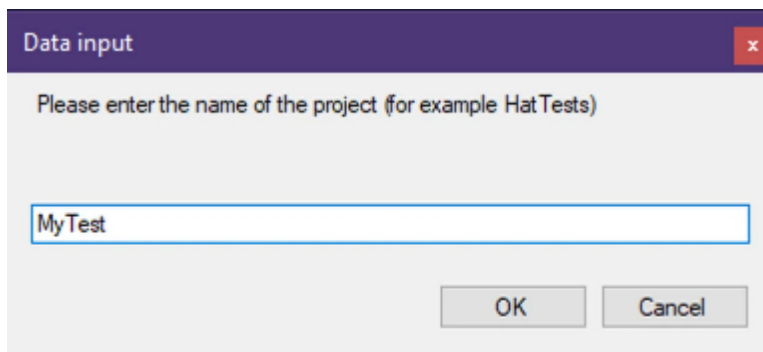
Creating a project compatible with Visual Studio

1. In the Project window, on the Explorer tab or in the main Create Project menu, click the "Creating a project compatible with Visual Studio" button

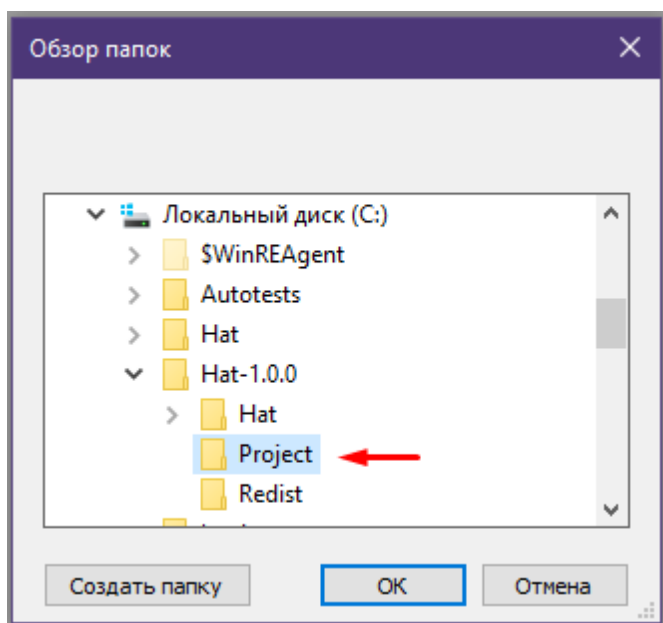




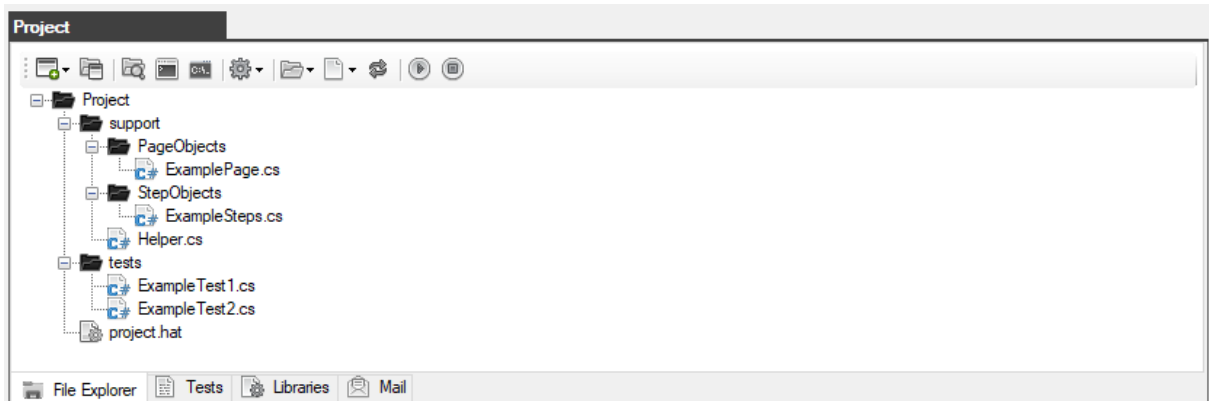
2. A window opens for entering the project name.
Prerequisite the project name must be in Latin letters and without spaces.



3. Create a folder that will contain the project with any name and anywhere on the disk.
In this case, the Project folder has been created at C:\Hat-1.0.0\
Select the project folder and click OK

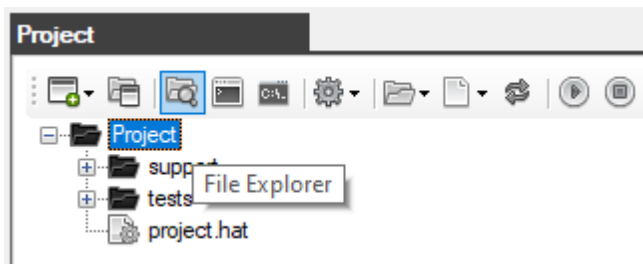


4. In the Project window, the contents of the project will be displayed on the Explorer tab. As you can see, the project is not empty, it contains several demo autotests.

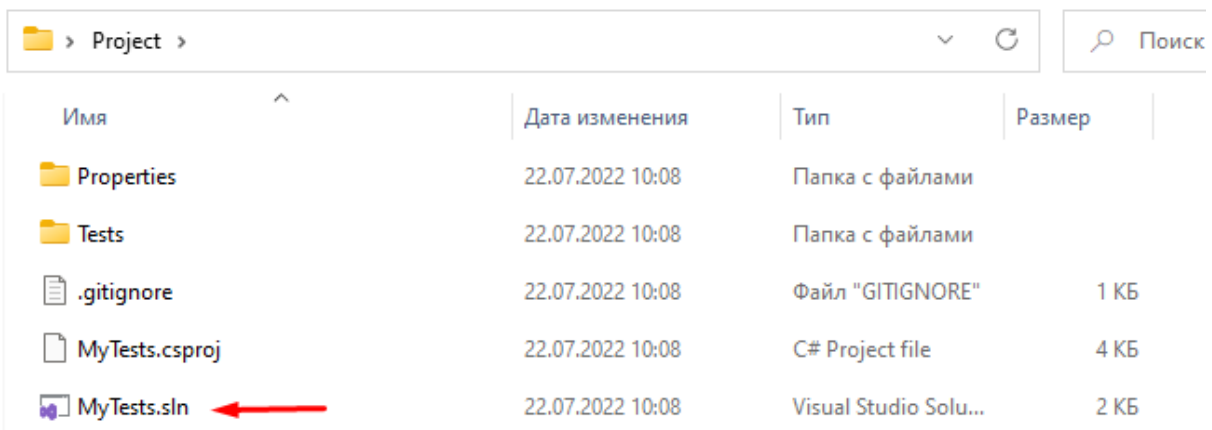


5. This project supports work in Visual Studio (it is assumed that you already have Visual Studio installed)

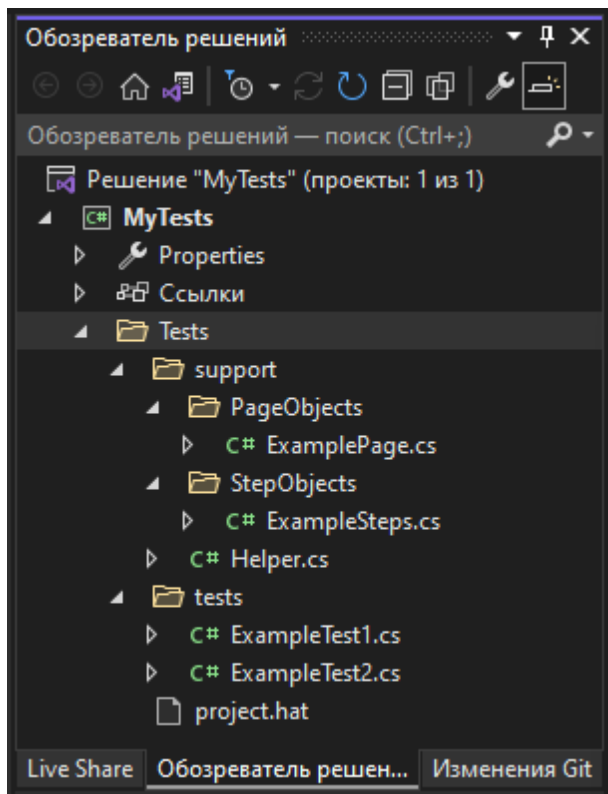
To open a project in Visual Studio, click on the Explorer button in the Project window



Then double-click on the file with the *.sln extension (in this case, MyTests.sln)



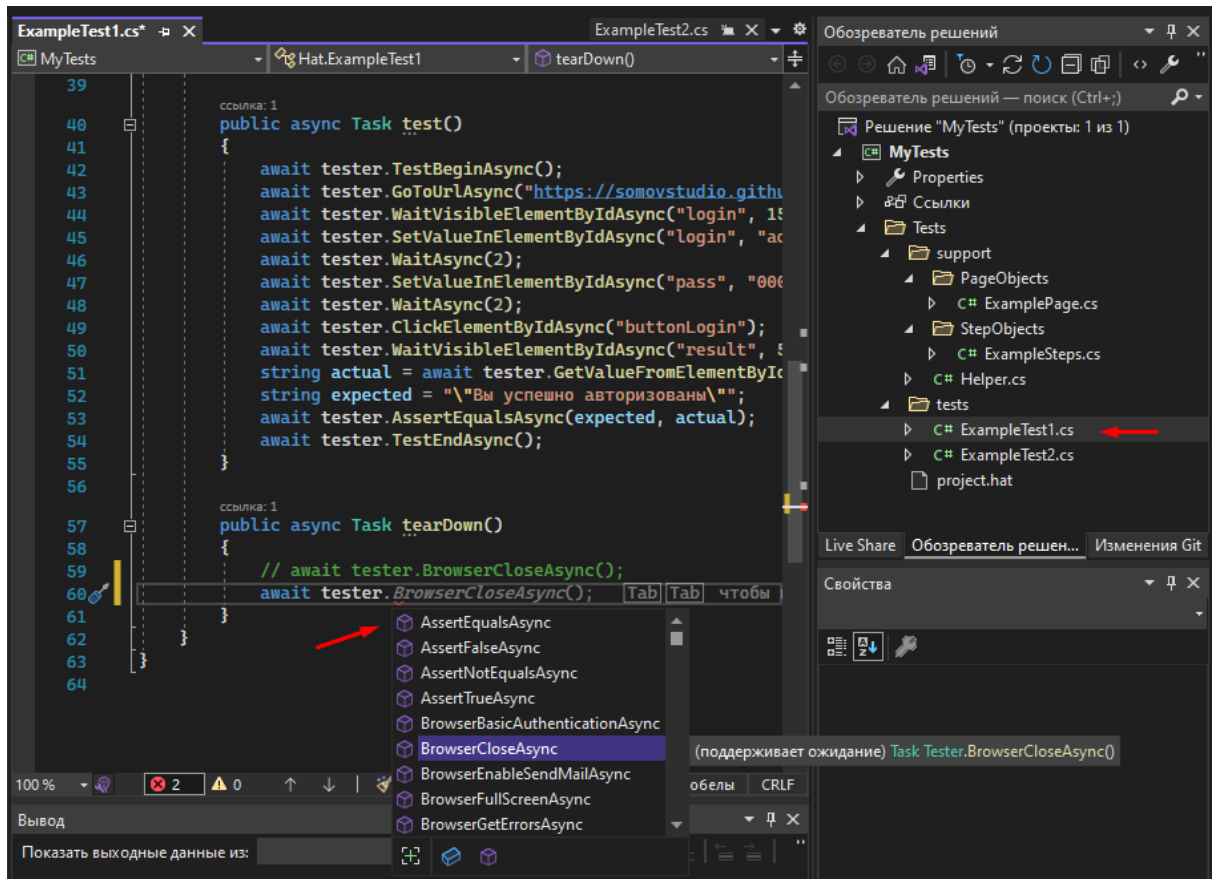
When Visual Studio is loaded, our project will immediately be opened and available for work in the Solution Explorer window



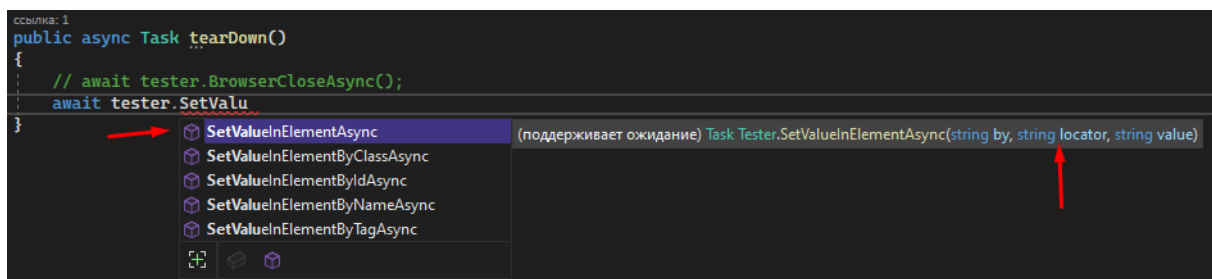
Double-click on the ExampleTest1.cs file to open it in the code editor.

In line 59, a line is commented out with a call to the `BrowserCloseAsync()` method that closes the browser.

This is necessary so that the browser does not automatically close after the completion of the autotest.



It is very convenient to use Visual Studio because the editor, when entering, shows a drop-down list with all available methods, as well as shows the syntax of the selected method and the variables required by it.

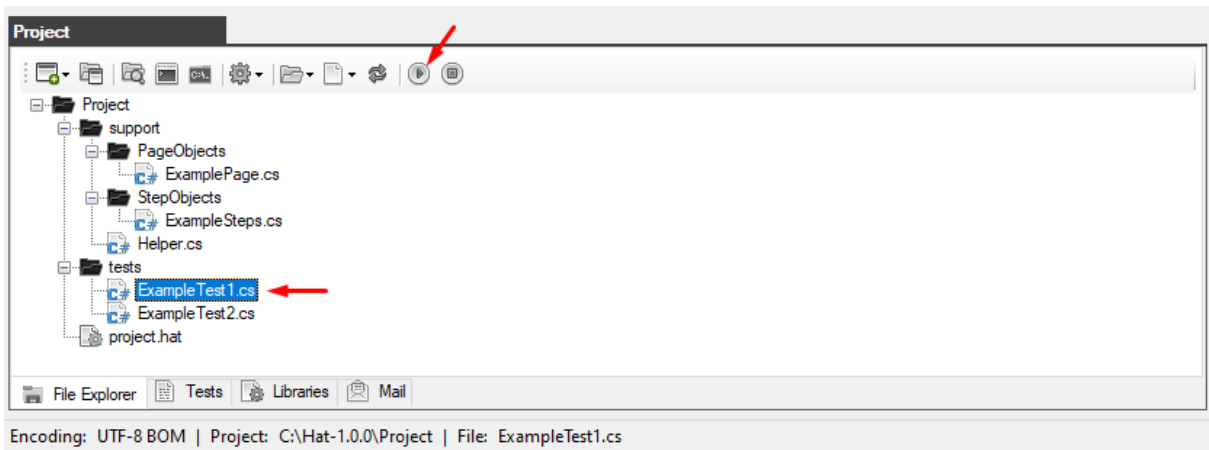


The project has been successfully created.

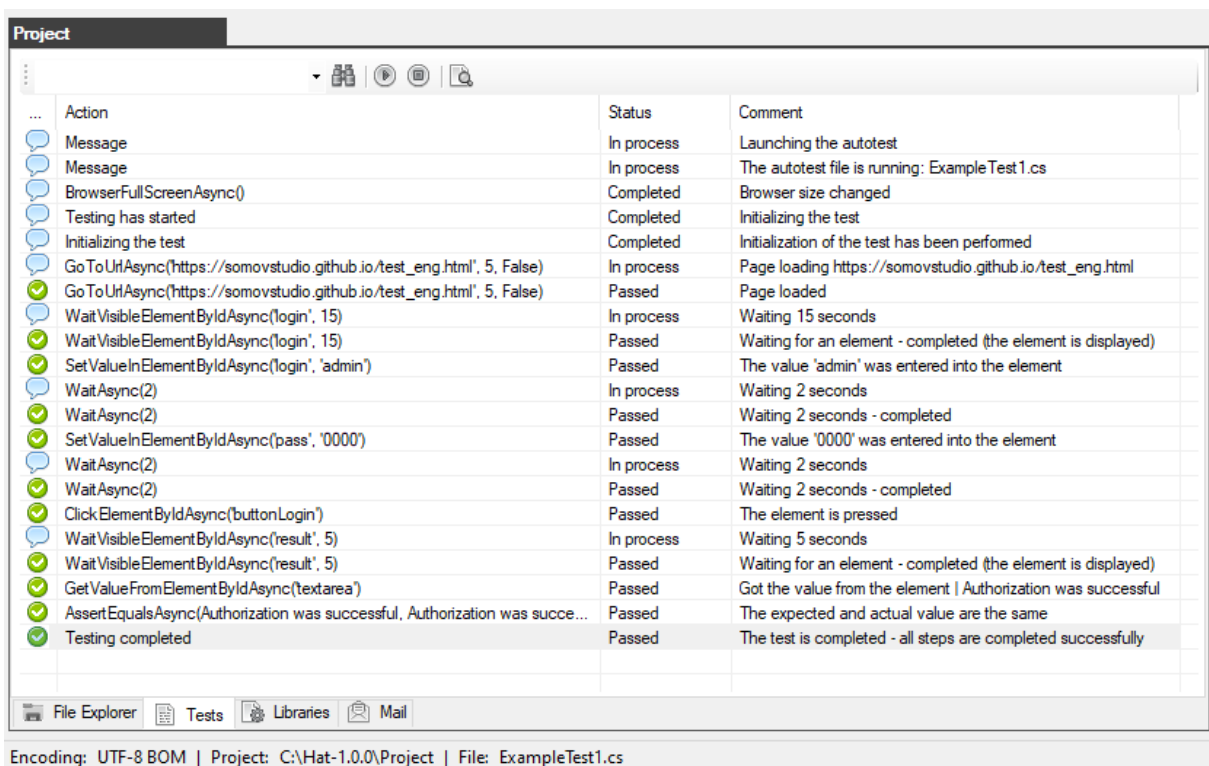
Launching the autotest and result

Launching the autotest

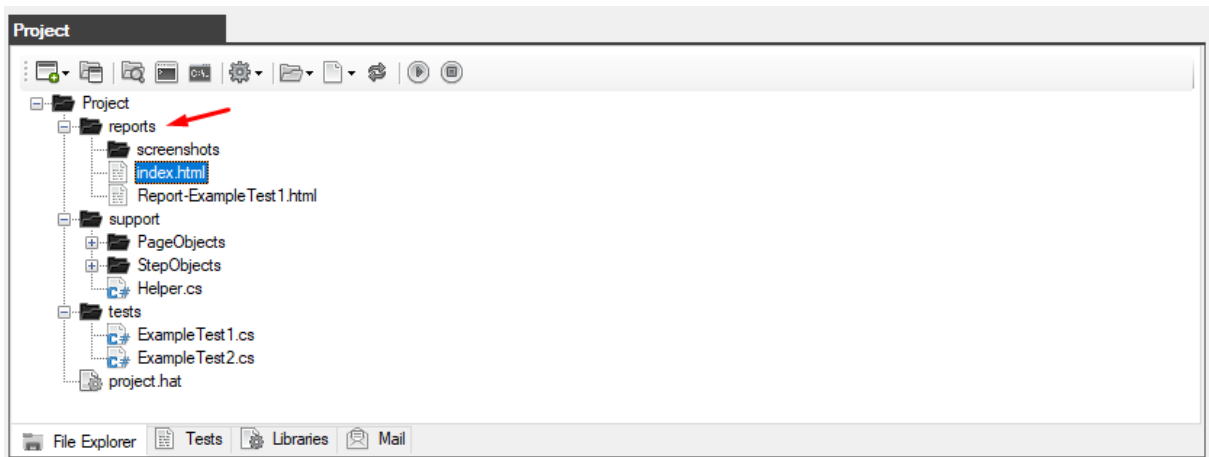
To run the autotest, select it in the tests folder and click on the "Run test" button
In our case, the ExampleTest1.cs file is launched



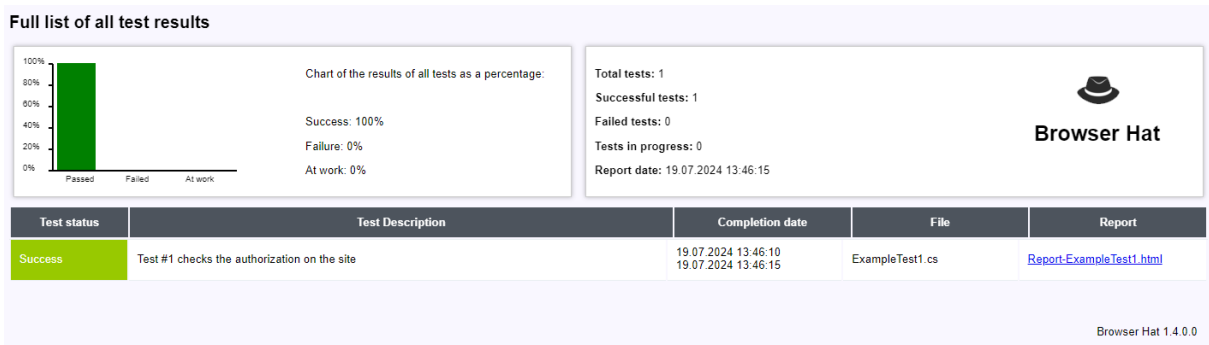
The verification process is displayed on the "Test" tab



Upon completion of the autotest, a folder "[report](#)" will be created in the root of the project, which contains all reports and screenshots with the results of the performed check.



Double-clicking on the report file (for example: `index.html` or `Report-ExampleTest1.html`) you will receive a report with a full description of the progress of the autotest. For more information about [reports](#) and sending reports by [mail](#), see the relevant sections.



Autotest Report

Description: Test #1 checks the authorization on the site
File: ExampleTest1.cs
Date: 19.07.2024 13:46:10

Result: **Success**

Status	Action	Comment
Completed	BrowserFullScreenAsync()	Browser size changed
Completed	Testing has started	Initializing the test
Completed	Initializing the test	Initialization of the test has been performed
Passed	GoToUrlAsync('https://somovstudio.github.io/test_eng.html', 5, False)	Page loaded
Passed	WaitVisibleElementByIdAsync('login', 15)	Waiting for an element - completed (the element is displayed)
Passed	SetValueInElementByIdAsync('login', 'admin')	The value 'admin' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	SetValueInElementByIdAsync('pass', '0000')	The value '0000' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	ClickElementByIdAsync('buttonLogin')	The element is pressed
Passed	WaitVisibleElementByIdAsync('result', 5)	Waiting for an element - completed (the element is displayed)
Passed	GetValueFromElementByIdAsync('textarea')	Got the value from the element Authorization was successful
Passed	AssertEqualsAsync(Authorization was successful, Authorization was successful)	The expected and actual value are the same
Passed	Testing completed	The test is completed - all steps are completed successfully

Browser Hat 1.4.0.0

As a result, the project was created, the autotest was launched and a report was received.

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

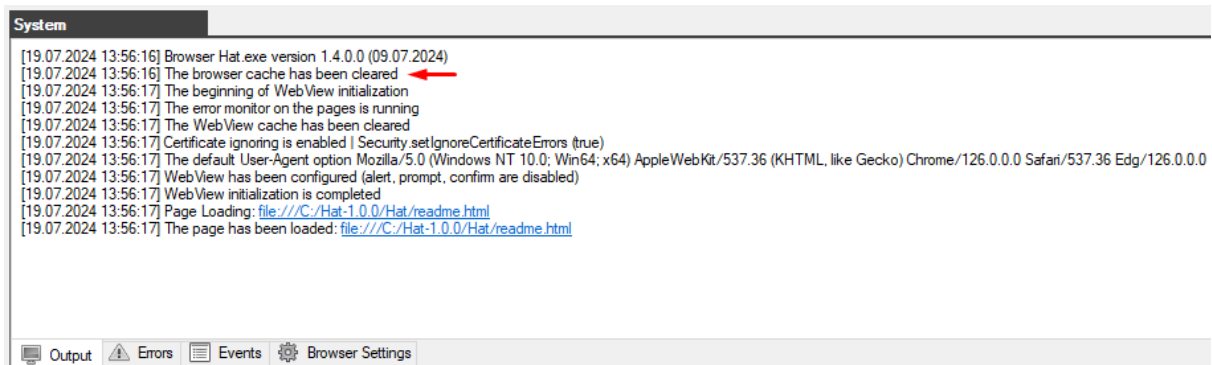
Browser Cache

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

Clearing

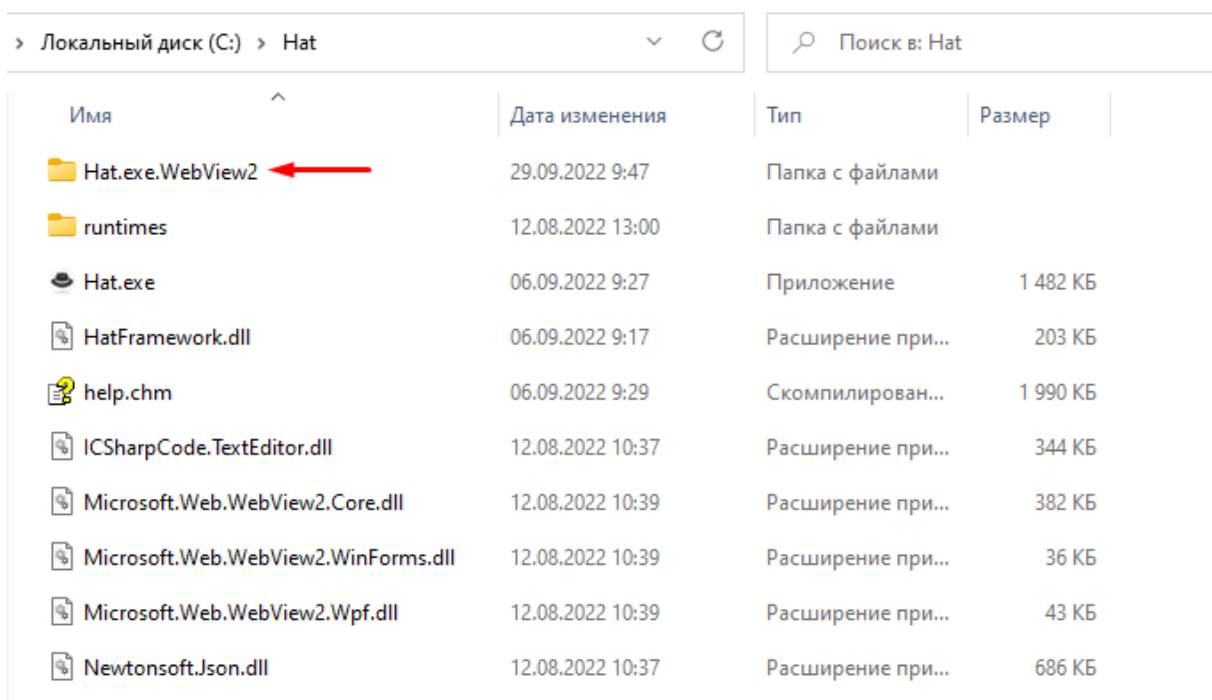
Кэш браузера - очистка

The browser automatically cleans the cache before each launch and reports this to the console



You can also clear the cache manually.

To do this, you need to close the browser and delete the "Hat.exe.WebView2" folder



The entire cache is stored in the folder "Hat.exe.WebView 2", after you delete it and launch the browser, this folder will be created again and a new cache of the current session will be stored in it.

Created with the Personal Edition of HelpNDoc: [Create help files for the Qt Help Framework](#)

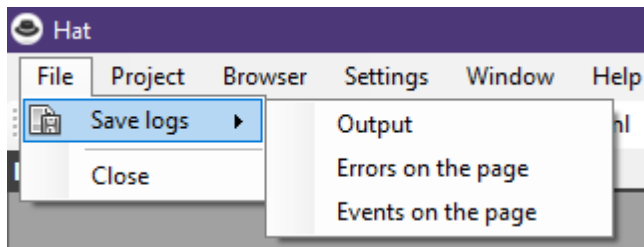
Interface

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

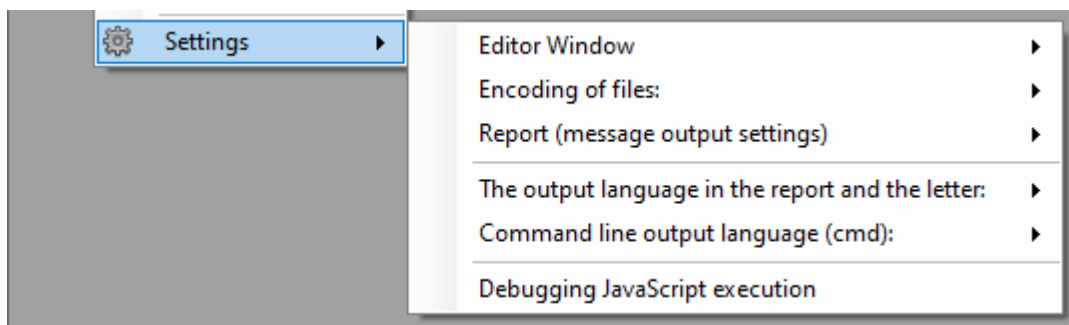
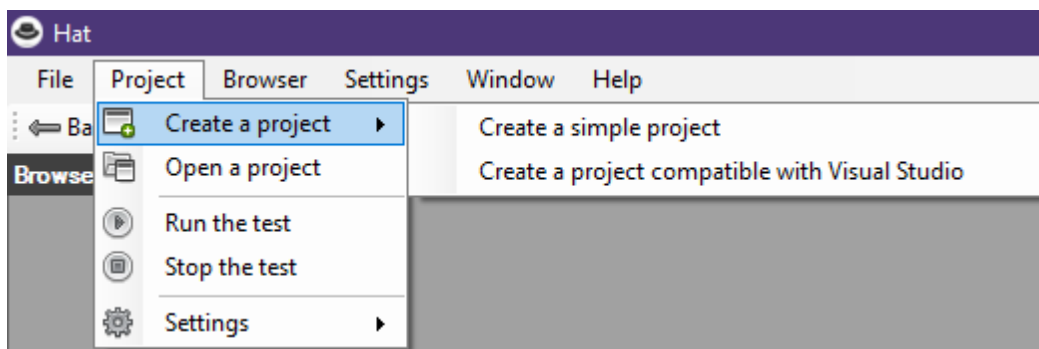
The main menu

The main menu

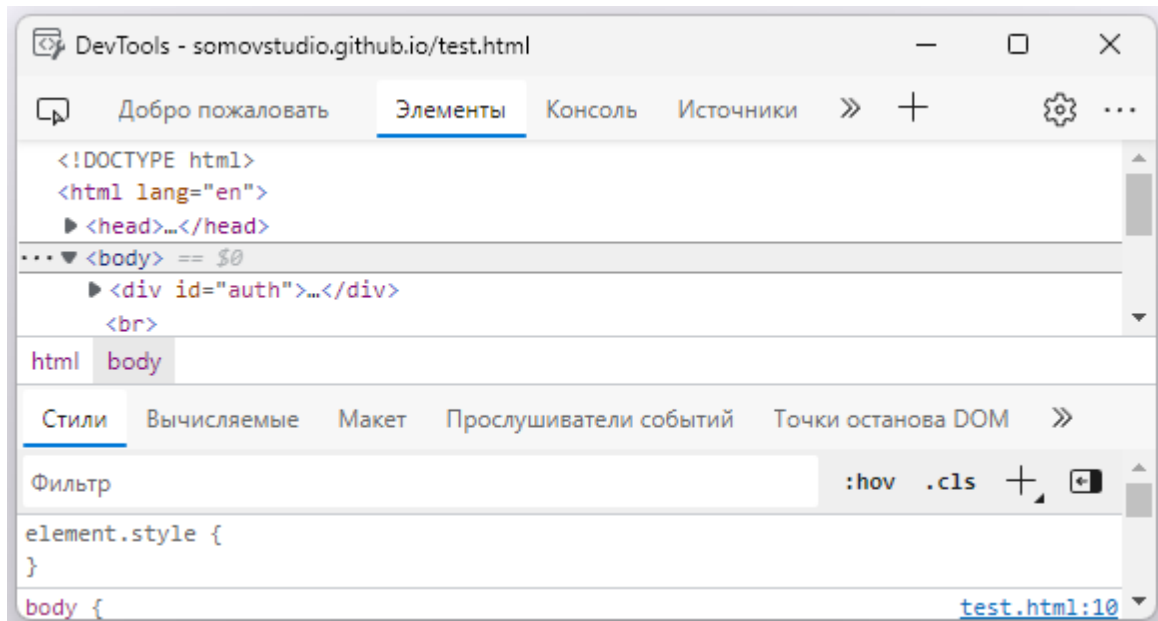
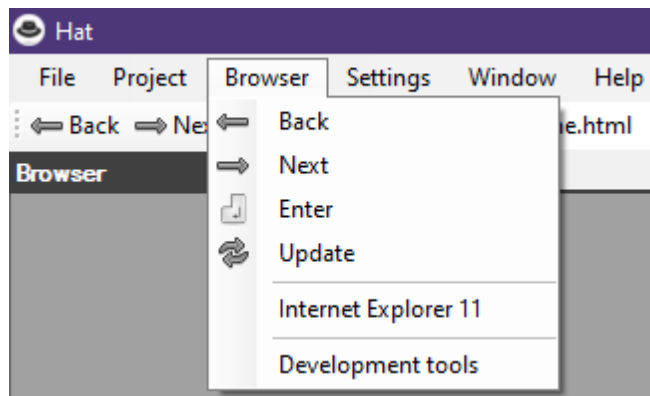
Menu "File" - allows you to save logs to txt files.



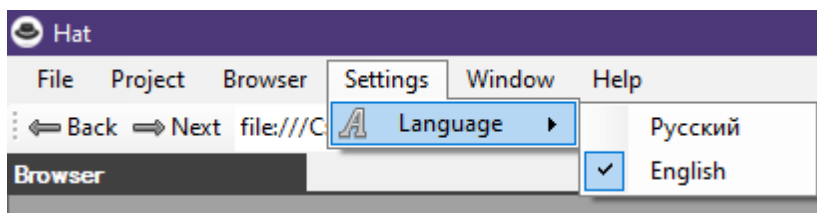
Menu "Project" - creating and opening autotest projects, performing and stopping autotests, as well as settings.



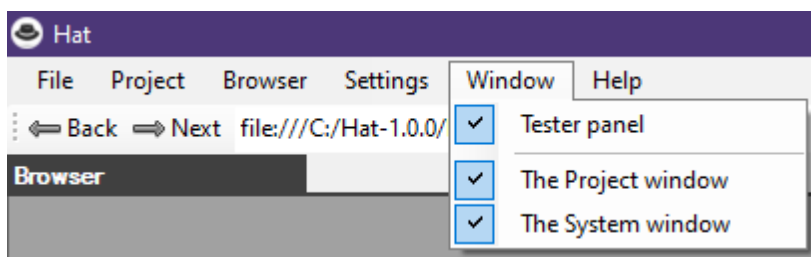
Menu "Browser" - it serves as a browser navigation, as well as allows you to open the developer dashboard "Development Tool" and open the Internet Explorer 11 window



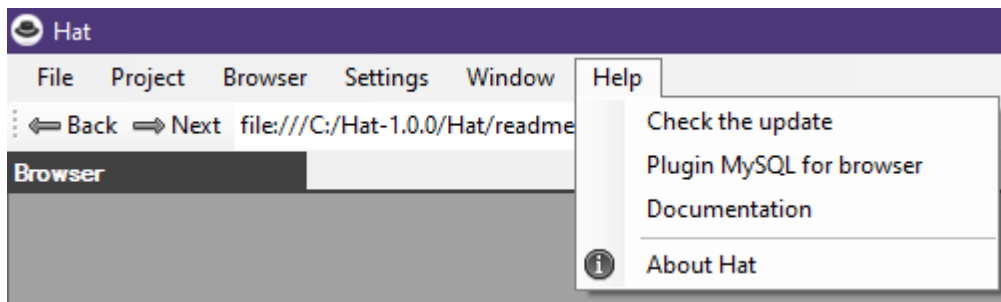
Menu "Settings" - setting the program language



Menu "Window" - allows you to hide or display the system browser windows.



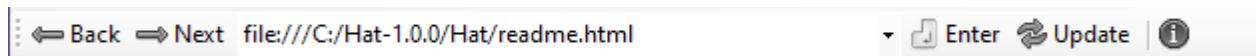
Menu "Help" - contains background information and links to the update and plugin



Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

The Toolbar

The Toolbar



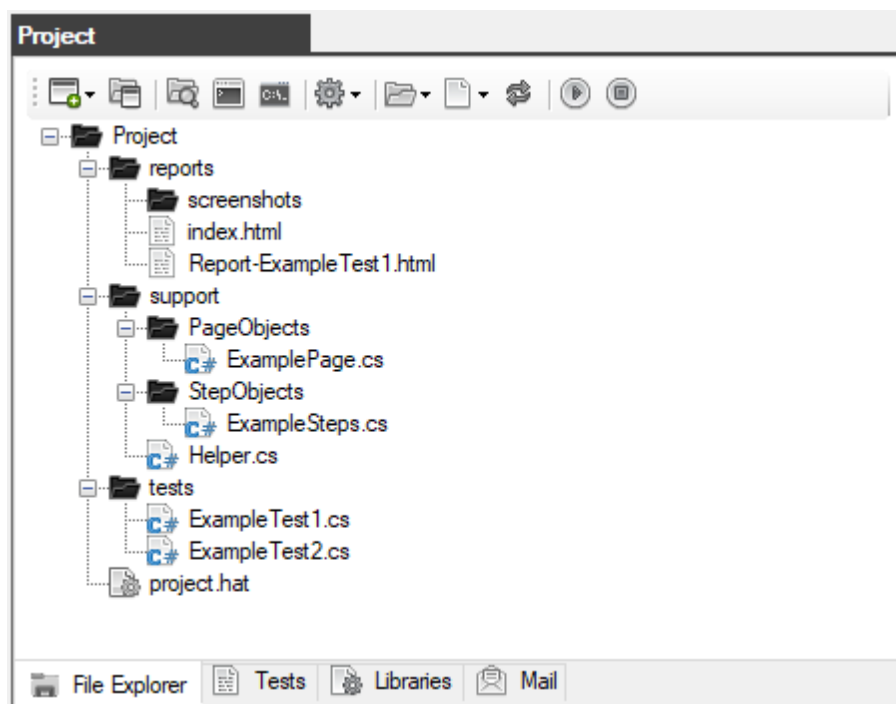
Contains a field for entering the site address and browser navigation buttons.

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

The Project window - is the Explorer tab

The Project window - is the Explorer tab

The main window for working with the project.



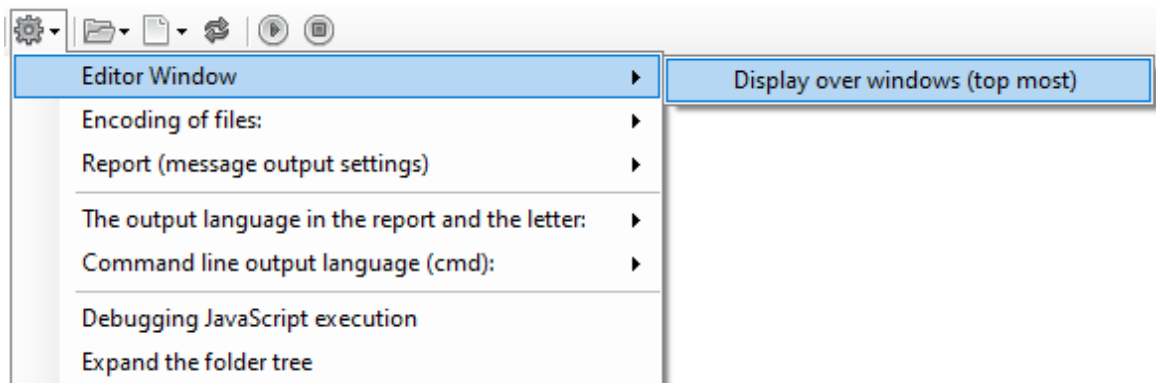
The toolbar contains the following buttons:



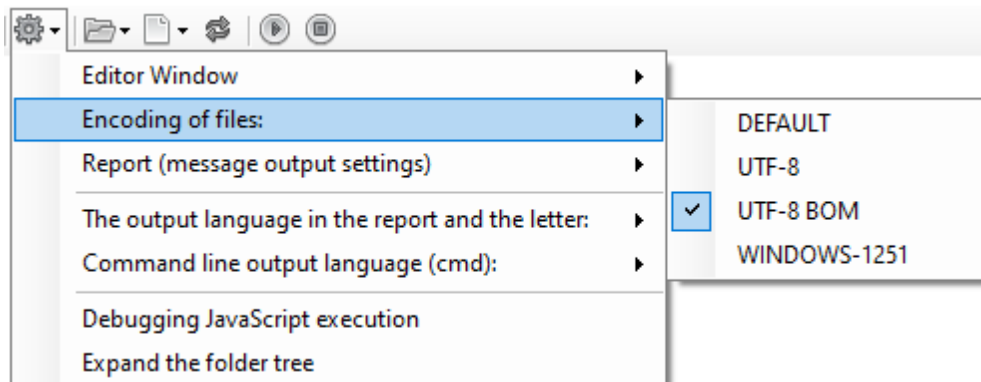
- o Create a project - creates a new autotest project (simple or compatible with Visual Studio)
- o Open a project - opens a previously created autotest project
- o Explorer - opens the project folder in Windows Explorer
- o Form a startup command - opens a window with an already formed autotest startup command for the Windows command prompt (cmd)
- o Command Prompt (cmd) - opens the Windows system console
- o Settings - the settings menu for working with the project
- o Folders - menu for working with folders
- o Files - menu for working with files
- o Update - updating information in the folder and file tree
- o Run the test - runs the selected autotest
- o Stop the test - stops the running autotest

Settings Menu

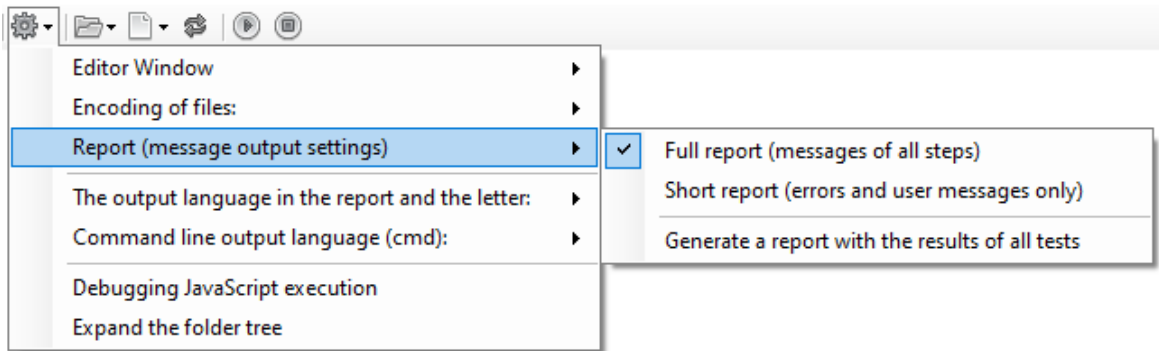
- o Choosing the way to display the editor window



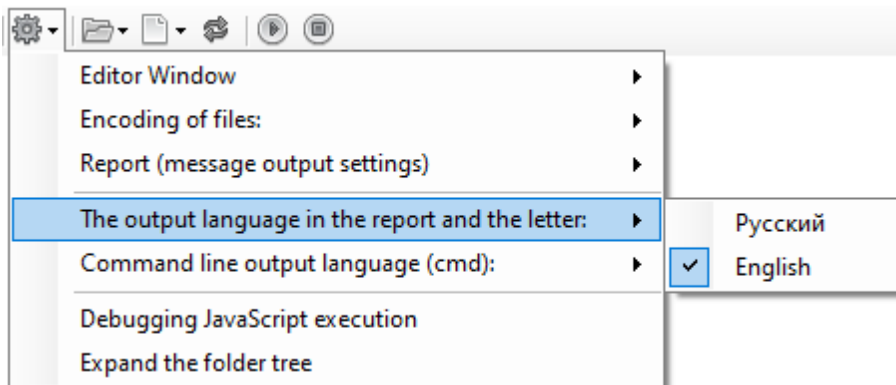
- o Choosing the encoding for the project files



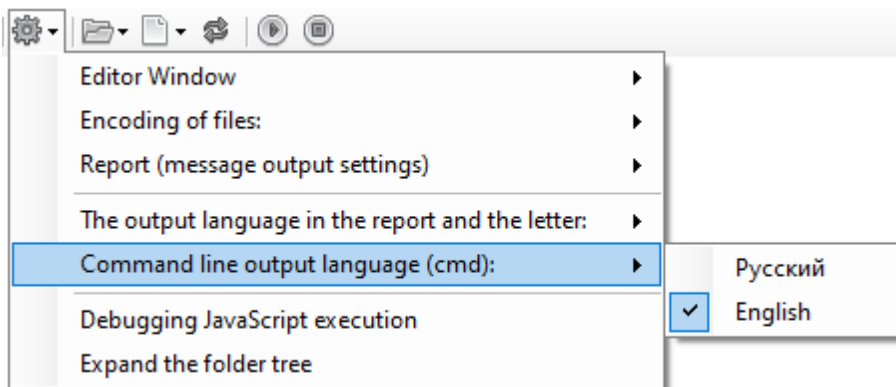
- o Configuring the output of messages in the report, as well as generating a report with all the results of the checks.



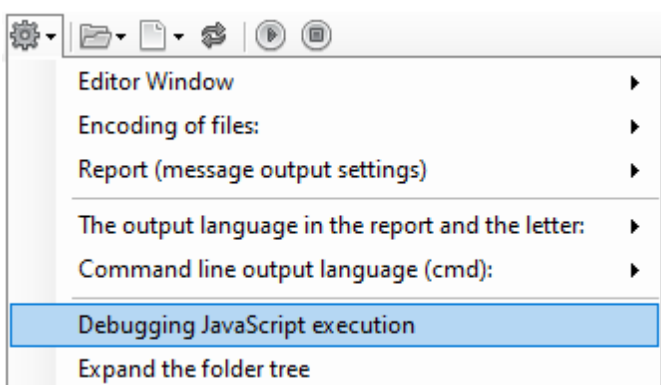
- o Selecting the output language in the report and letter



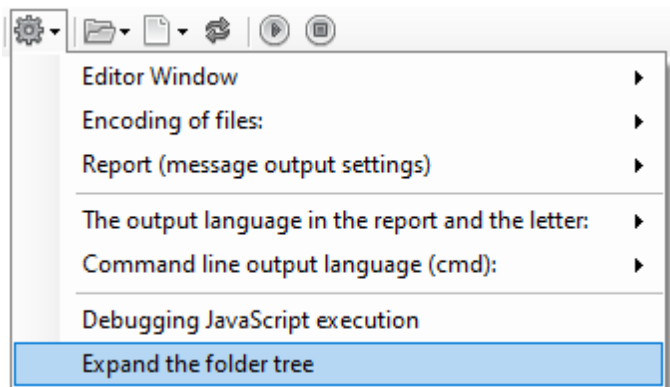
- o Selecting the output language on the command line (cmd)



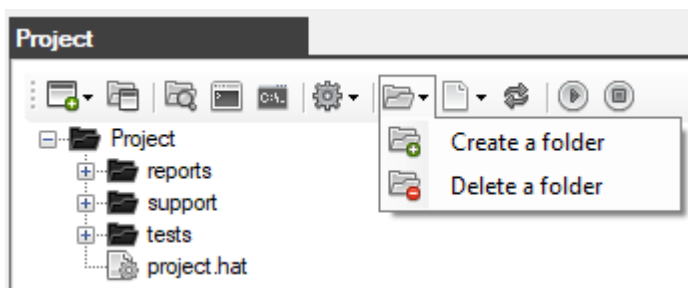
- o Enabling debugging when executing JavaScript code



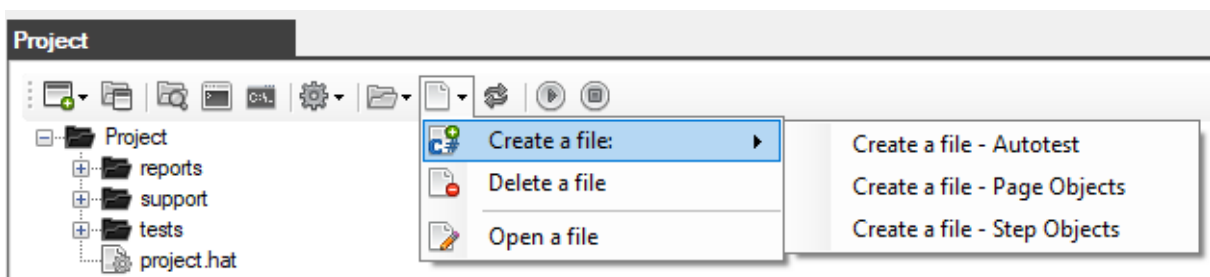
- o Expand the folder and file tree in Explorer



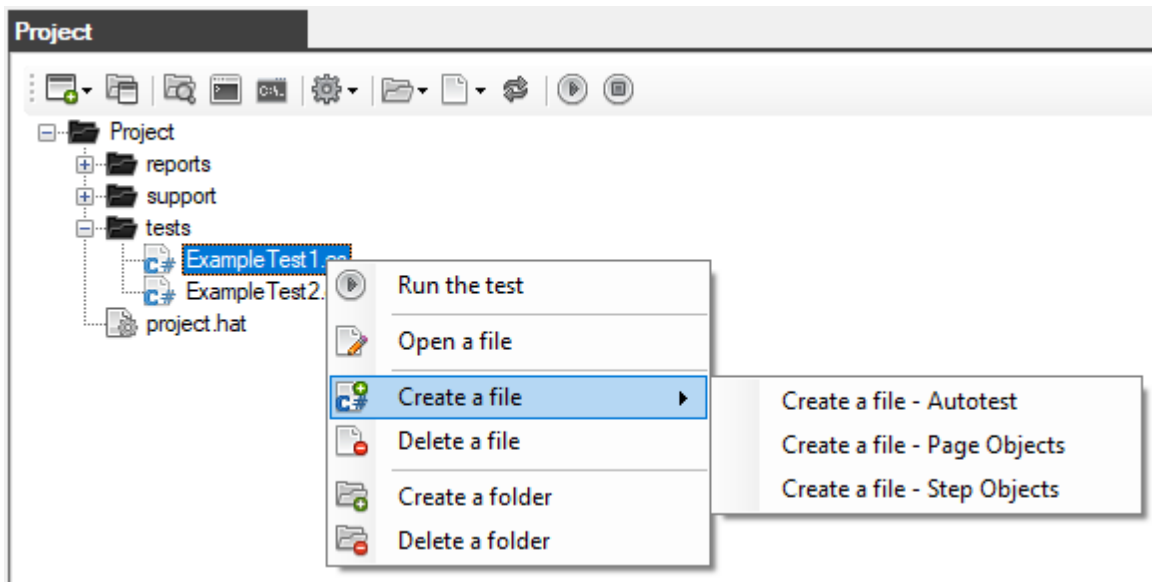
The folder menu allows you to create and delete folders



The file menu allows you to create new autotest files, open and delete existing files



The tree of project folders and files



Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

The Project window - is the Test tab

The Project window - is the Test tab

Table of autotest steps.

Project			
Action	Status	Comment	
Message	In process	Launching the autotest	
Message	In process	The autotest file is running: ExampleTest1.cs	
BrowserFullScreenAsync()	Completed	Browser size changed	
Testing has started	Completed	Initializing the test	
Initializing the test	Completed	Initialization of the test has been performed	
GoToUrlAsync('https://somovstudio.g...')	In process	Page loading https://somovstudio.github.io/test_eng.html	
GoToUrlAsync('https://somovstudio.g...')	Passed	Page loaded	
WaitVisibleElementByIdAsync('login', ...)	In process	Waiting 15 seconds	
WaitVisibleElementByIdAsync('login', ...)	Passed	Waiting for an element - completed (the element is displayed)	
SetValueInElementByIdAsync('login', '...')	Passed	The value 'admin' was entered into the element	
WaitAsync(2)	In process	Waiting 2 seconds	
WaitAsync(2)	Passed	Waiting 2 seconds - completed	
SetValueInElementByIdAsync('pass', '...')	Passed	The value '0000' was entered into the element	
WaitAsync(2)	In process	Waiting 2 seconds	
WaitAsync(2)	Passed	Waiting 2 seconds - completed	
ClickElementByIdAsync('buttonLogin')	Passed	The element is pressed	
WaitVisibleElementByIdAsync('result', 5)	In process	Waiting 5 seconds	
WaitVisibleElementByIdAsync('result', 5)	Passed	Waiting for an element - completed (the element is displayed)	
GetValueFromElementByIdAsync('text...')	Passed	Got the value from the element Authorization was successful	
AssertEqualsAsync('Authorization was ...')	Passed	The expected and actual value are the same	
Testing completed	Passed	The test is completed - all steps are completed successfully	

Project			
Action	Status	Comment	
Message	In process	Launching the autotest	
Message	In process	The autotest file is running: ExampleTest2.cs	
BrowserFullScreenAsync()	Completed	Browser size changed	
Testing has started	Completed	Initializing the test	
Initializing the test	Completed	Initialization of the test has been performed	
GoToUrlAsync('https://somovstudio.g...	In process	Page loading https://somovstudio.github.io/test_eng.html	
GoToUrlAsync('https://somovstudio.g...	Passed	Page loaded	
WaitVisibleElementByIdAsync('login', ...	In process	Waiting 15 seconds	
WaitVisibleElementByIdAsync('login', ...	Passed	Waiting for an element - completed (the element is displayed)	
SetValueInElementByIdAsync('login', '...	Passed	The value 'admin' was entered into the element	
WaitAsync(2)	In process	Waiting 2 seconds	
WaitAsync(2)	Passed	Waiting 2 seconds - completed	
SetValueInElementByIdAsync('pass', '...	Passed	The value '0001' was entered into the element	
WaitAsync(2)	In process	Waiting 2 seconds	
WaitAsync(2)	Passed	Waiting 2 seconds - completed	
ClickElementByIdAsync('buttonLogin')	Passed	The element is pressed	
WaitVisibleElementByIdAsync('result', 5)	In process	Waiting 5 seconds	
WaitVisibleElementByIdAsync('result', 5)	Passed	Waiting for an element - completed (the element is displayed)	
GetValueFromElementByIdAsync('text...	Passed	Got the value from the element Invalid login or password	
AssertEqualsAsync('Authorization was ...	Failed	The expected and actual value do not match	
Testing completed	Failed	Test completed - the test steps were executed unsuccessfully	

The toolbar contains the following buttons:

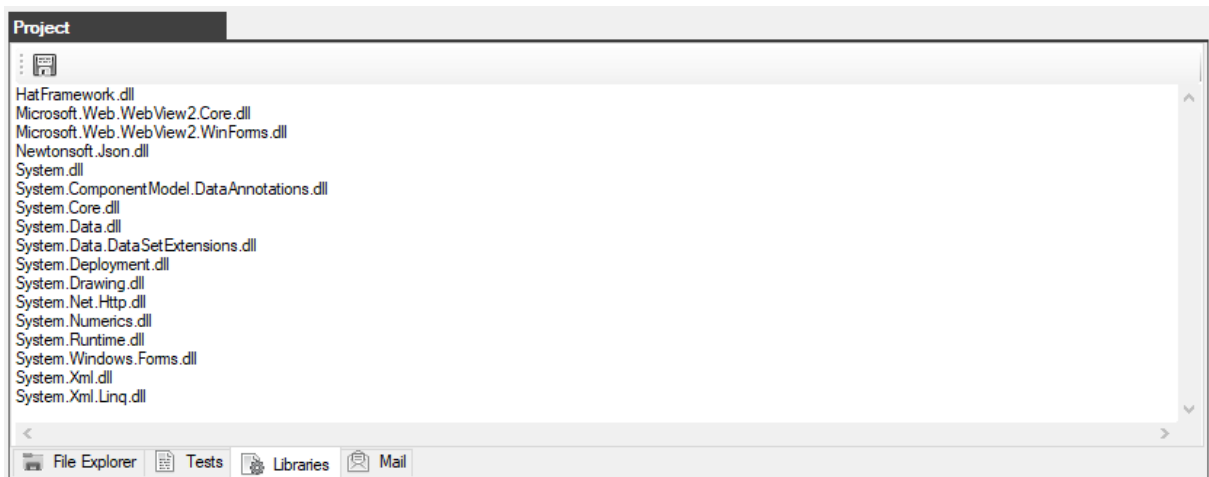


- o Field and Search button - creates a new autotest project
- o Open a project - opens a previously created autotest project
- o Run the test - runs the selected autotest
- o Stop the test - stops the running autotest
- o Detailed information about the step - opens a window with a description of the step

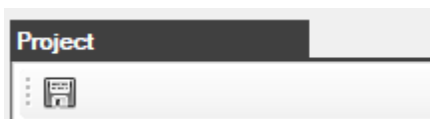
The Project window - the Library tab

The Project window - the Library tab

List of plug-in libraries required to perform autotests



The toolbar contains the following buttons:



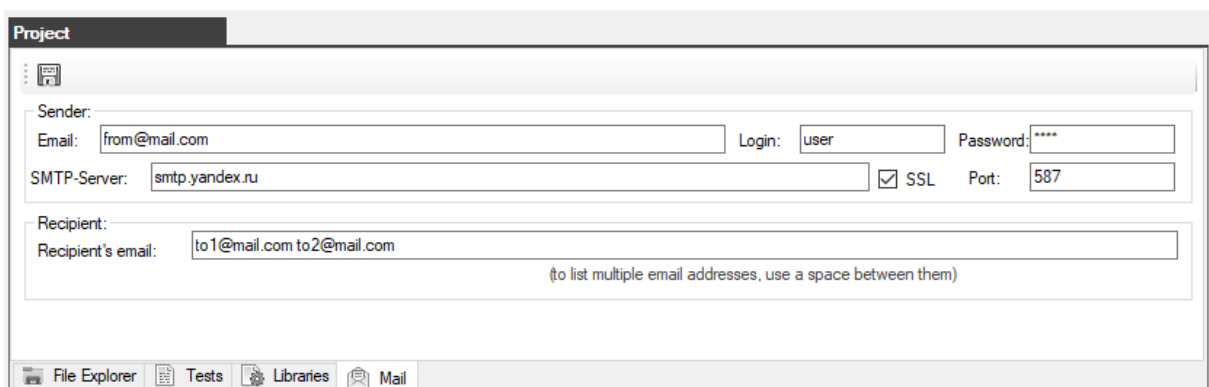
- o Save - saves the list of libraries to the [project.hat](#) project file

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

The Project window - the Mail tab

The Project window - the Mail tab

Mail server connection settings for sending emails with reports on the results of passing autotests



The toolbar contains the following buttons:



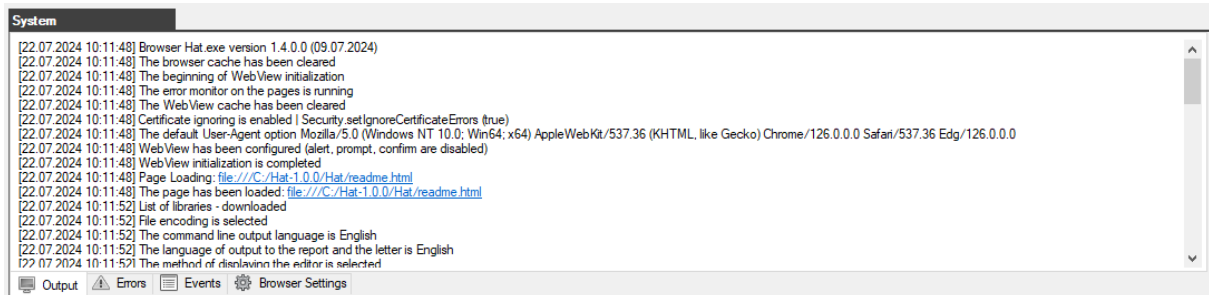
- o Save - saves the settings to the [project.hat](#) project file

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

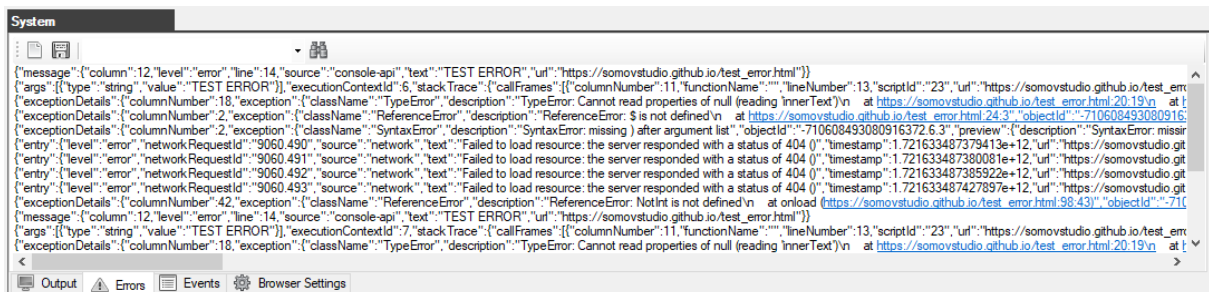
The system window

The system window

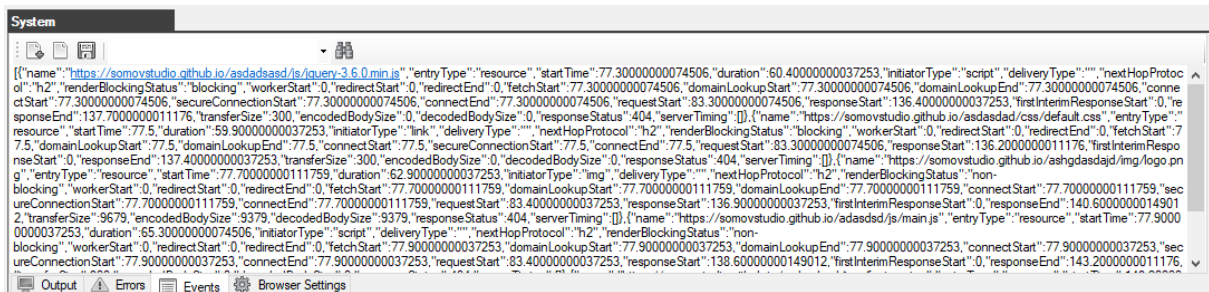
The "Output" tab displays all events that occur when the application is running.



The "Errors" tab displays all errors (console) occurring on loaded pages.



The "Events" tab - displays all events (network) occurring on loaded pages.



The "Browser Settings" tab is the main browser settings.



Reports

Created with the Personal Edition of HelpNDoc: [Easily create PDF Help documents](#)

Autotest Report

Autotest Report

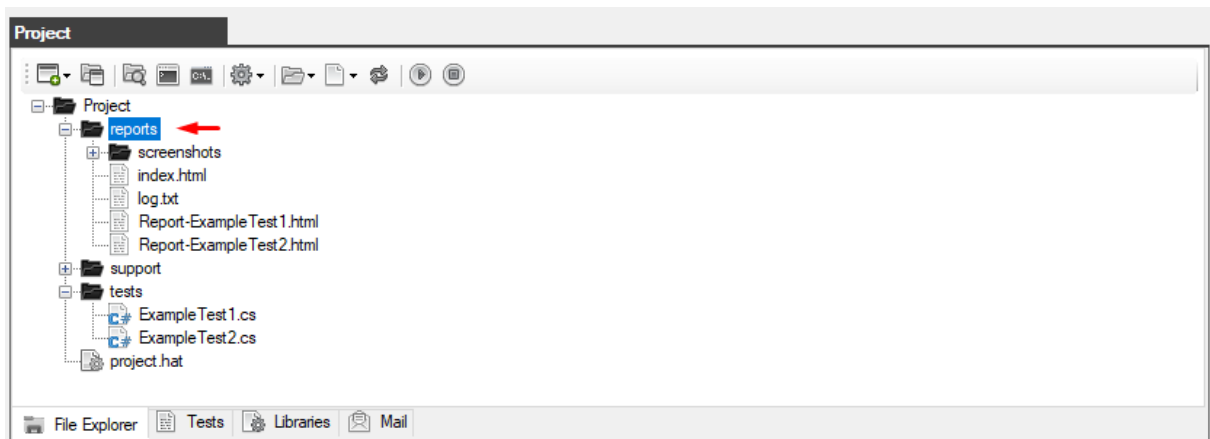
When you run the autotest, a new reports folder will be created in the project folder if it does not already exist.

After completion of the autotest, a file with a report in html format will be created.

The name of the report file corresponds to the name of the autotest file with the prefix Report

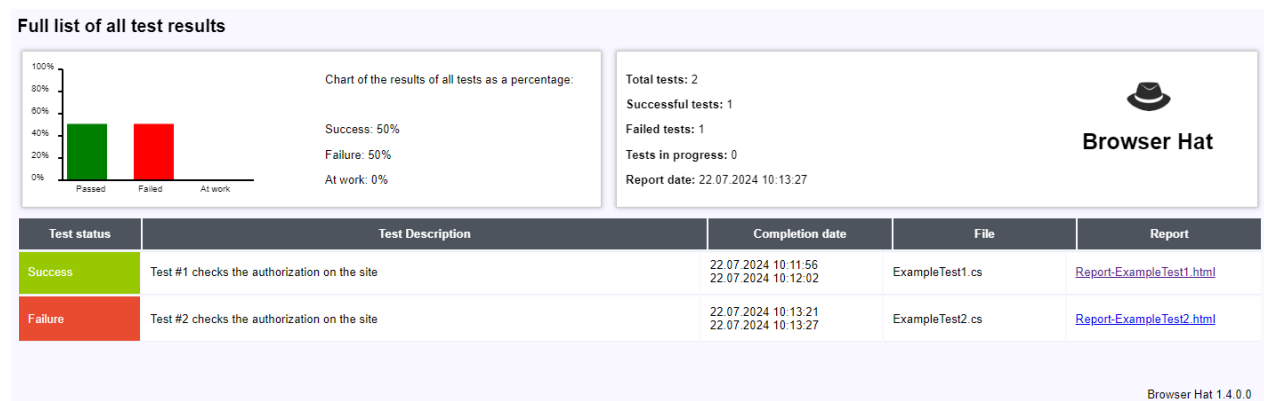
For example, when performing the ExampleTest1.cs autotest, a report will be created with the name Report-ExampleTest1.html

To view the results for all completed autotests, a file is created and updated index.html

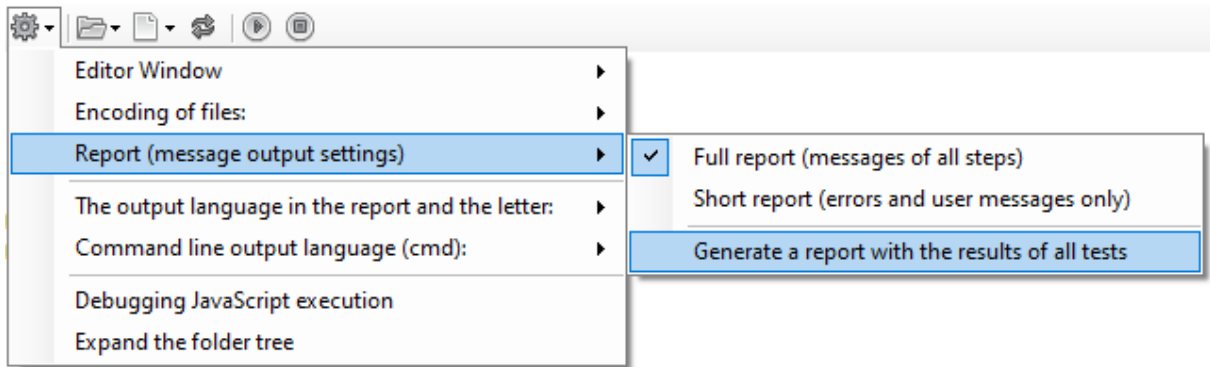


The resulting report opens in any browser

Full report on all completed inspections (index.html) it looks like this:



This report (index.html) is created or updated after the completion of any autotest.



This report can be generated or updated manually via the project settings menu.

The report with the successful completion of all steps and checks looks like this:

Autotest Report		
Description: Test #1 checks the authorization on the site File: ExampleTest1.cs Date: 22.07.2024 10:11:56		
Result: Success		
Status	Action	Comment
Completed	BrowserFullScreenAsync()	Browser size changed
Completed	Testing has started	Initializing the test
Completed	Initializing the test	Initialization of the test has been performed
Passed	GoToUrlAsync('https://somovstudio.github.io/test_eng.html', 5, False)	Page loaded
Passed	WaitVisibleElementByIdAsync('login', 15)	Waiting for an element - completed (the element is displayed)
Passed	SetValueInElementByIdAsync('login', 'admin')	The value 'admin' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	SetValueInElementByIdAsync('pass', '0000')	The value '0000' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	ClickElementByIdAsync('buttonLogin')	The element is pressed
Passed	WaitVisibleElementByIdAsync('result', 5)	Waiting for an element - completed (the element is displayed)
Passed	GetValueFromElementByIdAsync('textarea')	Got the value from the element Authorization was successful
Passed	AssertEqualsAsync(Authorization was successful, Authorization was successful)	The expected and actual value are the same
Passed	Testing completed	The test is completed - all steps are completed successfully

Browser Hat 1.4.0.0

The report on the failed test looks like this:

Autotest Report

Description: Test #2 checks the authorization on the site

File: ExampleTest2.cs

Date: 22.07.2024 10:13:21

Result: **Failure**

Status	Action	Comment
Completed	BrowserFullScreenAsync()	Browser size changed
Completed	Testing has started	Initializing the test
Completed	Initializing the test	Initialization of the test has been performed
Passed	GoToUriAsync("https://somovstudio.github.io/test_eng.html", 5, False)	Page loaded
Passed	WaitVisibleElementByIdAsync("login", 15)	Waiting for an element - completed (the element is displayed)
Passed	SetValueInElementByIdAsync("login", 'admin')	The value 'admin' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	SetValueInElementByIdAsync("pass", '0001')	The value '0001' was entered into the element
Passed	WaitAsync(2)	Waiting 2 seconds - completed
Passed	ClickElementByIdAsync("buttonLogin")	The element is pressed
Passed	WaitVisibleElementByIdAsync("result", 5)	Waiting for an element - completed (the element is displayed)
Passed	GetValueFromElementByIdAsync("textarea")	Got the value from the element Invalid login or password
Failed	AssertEqualsAsync(Authorization was successful, Invalid login or password)	The expected and actual value do not match
Failed	Testing completed	Test completed - the test steps were executed unsuccessfully

Screenshot

File: [image-22.07.2024-10-13-26.jpeg](#)

If any critical errors occur during execution, they will be reflected in the report as follows:

Отчет о работе автотеста

Файл: ExampleTest1.cs

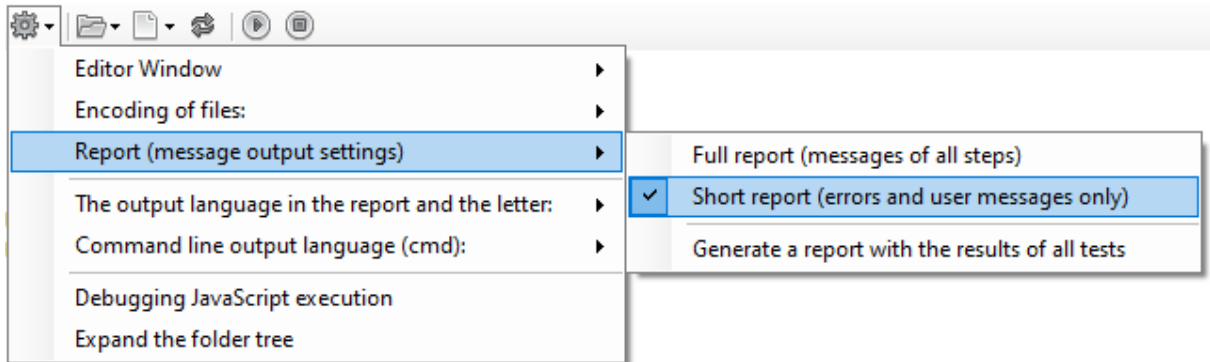
Результат: **Провальноно**

Статус	Действие	Комментарий
ОШИБКА		c:\Users\Catfish\Desktop\Hat\project\tests\ExampleTest1.cs(50,19) : error CS1502: Наиболее подходящий перегруженный метод для "HatFramework.Tester.ClickElementByIdAsync(string)" имеет несколько недопустимых аргументов
ОШИБКА		c:\Users\Catfish\Desktop\Hat\project\tests\ExampleTest1.cs(50,48) : error CS1503: Аргумент "1": преобразование типа из "int" в "string" невозможно
ОШИБКА		c:\Users\Catfish\Desktop\Hat\project\tests\ExampleTest1.cs(58,27) : warning CS1998: У этого асинхронного метода нет операторов "await", поэтому он будет выполняться синхронно. Рассмотрите возможность использования оператора "await" для ожидания неблокирующих вызовов API или конструкции "await Task.Run(...)" для выполнения фонового потока, ограниченного производительностью процессора.
ОШИБКА		c:\Users\Catfish\Desktop\Hat\project\tests\ExampleTest2.cs(53,27) : warning CS1998: У этого асинхронного метода нет операторов "await", поэтому он будет выполняться синхронно. Рассмотрите возможность использования оператора "await" для ожидания неблокирующих вызовов API или конструкции "await Task.Run(...)" для выполнения фонового потока, ограниченного производительностью процессора.
ОШИБКА		c:\Users\Catfish\Desktop\Hat\project\tests\ExampleTest3.cs(64,27) : warning CS1998: У этого асинхронного метода нет операторов "await", поэтому он будет выполняться синхронно. Рассмотрите возможность использования оператора "await" для ожидания неблокирующих вызовов API или конструкции "await Task.Run(...)" для выполнения фонового потока, ограниченного производительностью процессора.

Ошибок:

5

Using the project settings menu, you can switch the report type to "Summary Report", in which case only user messages and error messages will be displayed in the report.



So the reports will look like this:

Autotest Report

Description: Test #1 checks the authorization on the site
File: ExampleTest1.cs
Date: 22.07.2024 10:45:32

Result: **Success**

Status	Action	Comment
Completed	Testing has started	Initializing the test
Passed	Testing completed	The test is completed - all steps are completed successfully

Browser Hat 1.4.0.0

Autotest Report

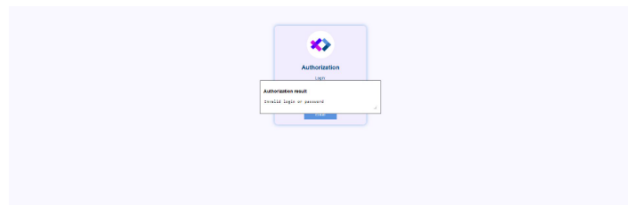
Description: Test #2 checks the authorization on the site
File: ExampleTest2.cs
Date: 22.07.2024 10:45:42

Result: **Failure**

Status	Action	Comment
Completed	Testing has started	Initializing the test
Failed	AssertEqualsAsync(Authorization was successful, Invalid login or password)	The expected and actual value do not match
Failed	Testing completed	Test completed - the test steps were executed unsuccessfully

Screenshot

File: [image-22-07-2024-10-45-48.jpeg](#)



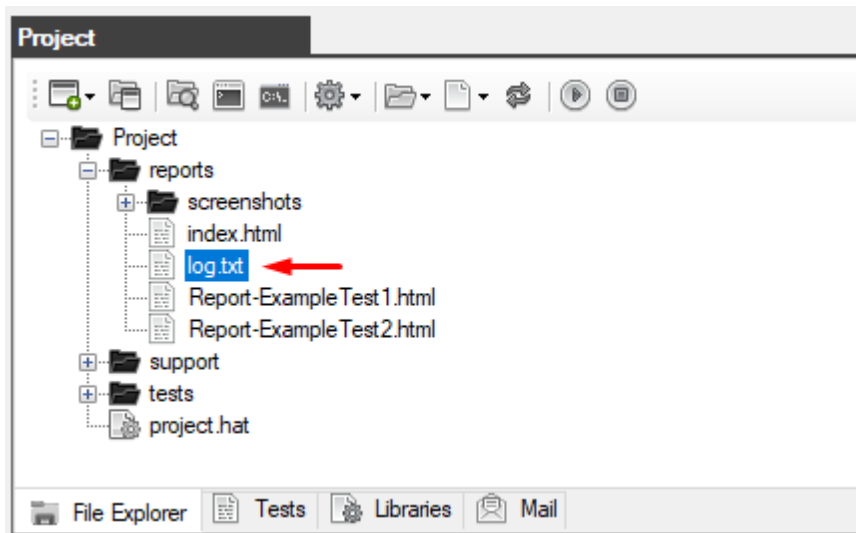
Browser Hat 1.4.0.0

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

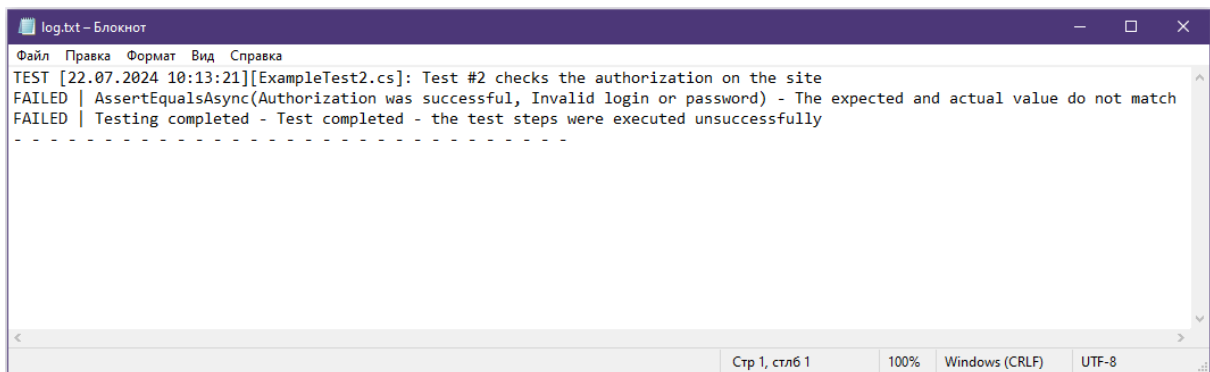
Log file

Log file (log.txt)

If the autotest fails, not only a report is created, but also an entry is made in the log file log.txt



The file records the date and time of the failure, the file name and description of the autotest, as well as the reasons for the failure of the autotest.



Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

Mail

Created with the Personal Edition of HelpNDoc: [Qt Help documentation made easy](#)

Send a report on the failure of the autotest to the mail

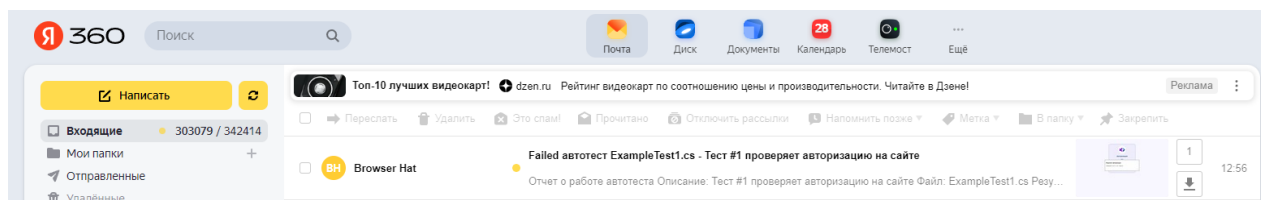
Send a report on the failure of the autotest to the mail

The connection to the mail server is configured in the "Project" window. This example uses a Yandex mailbox.

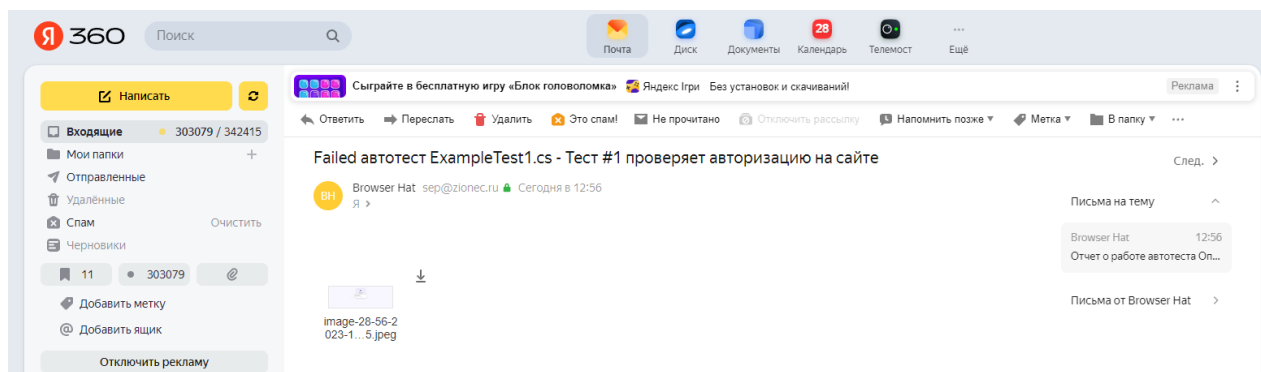
To use the function of sending a report to the mail, you need to enable this option in the autotest using the method `BrowserEnableSendMailAsync`

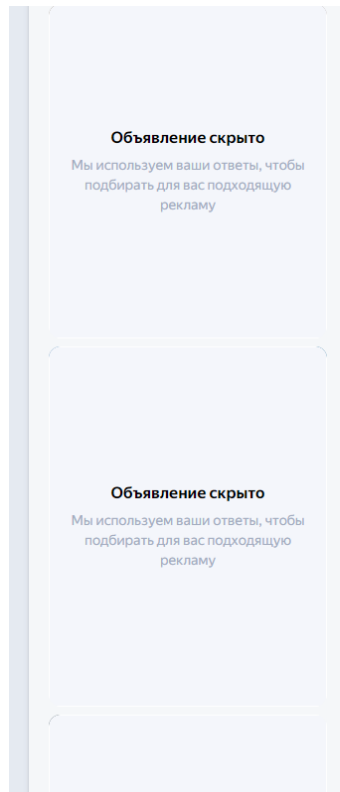
```
public async Task setUp()
{
    await tester.BrowserEnableSendMailAsync(true, false); // only in
case of failure
    await tester.BrowserEnableSendMailAsync(false, true); // only if
successful
    await tester.BrowserEnableSendMailAsync(); // in both cases
}
```

In case of failure, a report of the following type will be sent to the listed email addresses



The email will include a screenshot at the time of the error and a report with the steps of execution, as well as user messages.





Отчет о работе автотеста

Описание: Тест #1 проверяет авторизацию на сайте

Файл: ExampleTest1.cs

Результат: **Провально**

Статус	Действие	Комментарий
Выполнено	Сообщение	Запущен автотест из файла: ExampleTest1.cs
Выполнено	BrowserFullScreenAsync()	Размер браузера изменён
Выполнено	BrowserEnableSendMailAsync("True", "True")	Включена опция отправки отчета на почту
Выполнено	Тестирование началось	Выполнена инициализация теста
Успешно	GoToUrlAsync("https://somovstudio.github.io/test.html", 5)	Страница загружена
Успешно	WaitVisibleElementByIdAsync('login', 15)	Ожидание элемента - завершено (элемент отображается)
Успешно	SetValueInElementByIdAsync('login', 'admin')	Значение введено в элемент
Успешно	WaitAsync(2)	Ожидание 2 секунд - завершено
Успешно	SetValueInElementByIdAsync('pass', '0001')	Значение введено в элемент
Успешно	WaitAsync(2)	Ожидание 2 секунд - завершено
Успешно	ClickElementByIdAsync('buttonLogin')	Элемент нажат
Успешно	WaitVisibleElementByIdAsync('result', 5)	Ожидание элемента - завершено (элемент отображается)
Успешно	GetValueFromElementByIdAsync('textarea')	Получено значение из элемента
Провально	AssertEqualsAsync(Вы успешно авторизованы, Неверный логин или пароль)	Ожидаемое и фактическое значение не совпадают
Провально	Тестирование завершено	Тест завершен - шаги теста выполнены неуспешно
Скриншот	Файл: image-28-56-2023-12-56-35.jpeg	

You can send an email at any time, you need to use the function SendMsgToMailAsync

```
public async Task tearDown()
{
    if (tester.GetTestResult() == Tester.PASSED)
    {
        await tester.SendMsgToMailAsync("Text message", "The test was completed successfully");
    }
}
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

Console launch mode (Jenkins)

Created with the Personal Edition of HelpNDoc: [Easily create Web Help sites](#)

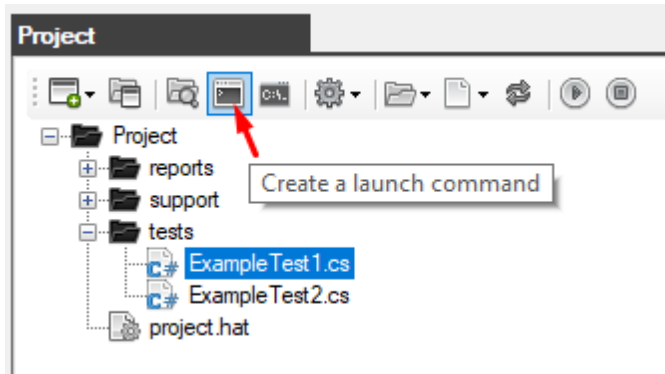
Running the autotest from the command line

Running the autotest from the command line

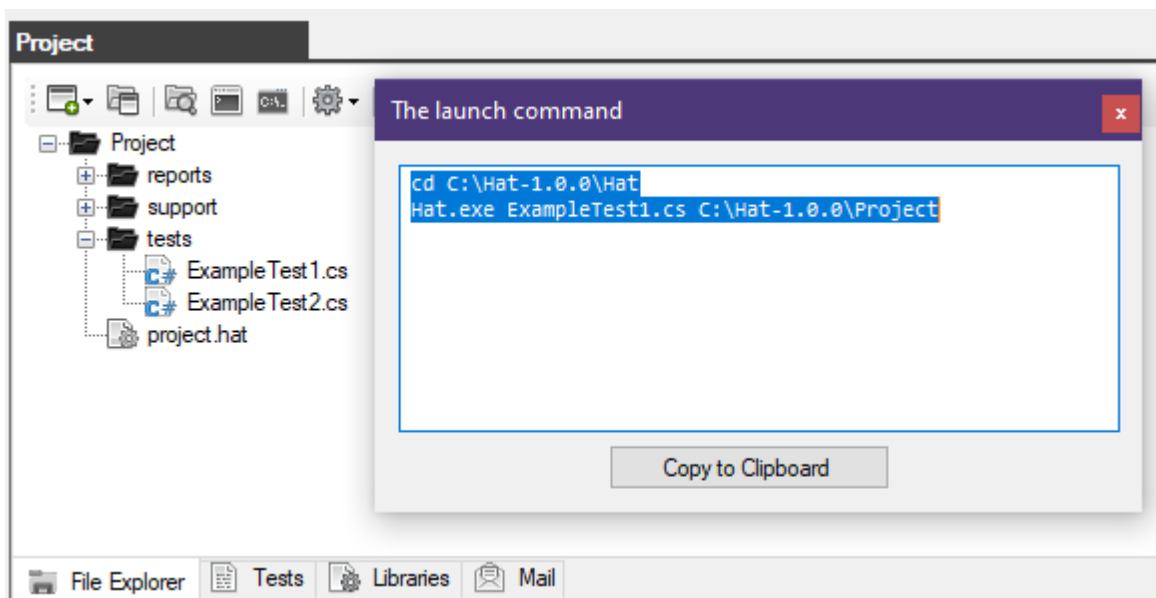
To run autotests in popular continuous integration systems, you need to use the Windows command line.



In the "Project" window (on the "Explorer" tab), select the autotest file that you want to run. Then click on the "Create a launch command" button on the toolbar



As a result, you will see a window with ready-made commands.

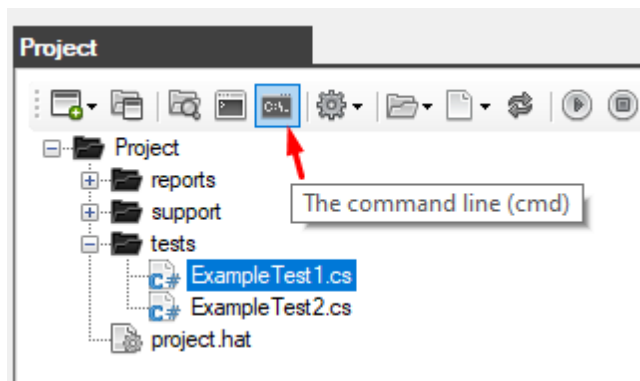


- o the first command: **cd C:\Hat-1.0.0\Hat** - this is a transition to the folder with the application Hat.exe
- o the second command: **Hat.exe ExampleTest1.cs C:\Hat-1.0.0\Project** - runs the autotest

The syntax of the startup command:

Hat.exe [Autotest_file_name] [Path_To_The_Project_Folder]

Open the command prompt



Use the received commands

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.4651]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Hat-1.0.0\Hat>cd C:\Hat-1.0.0\Hat

C:\Hat-1.0.0\Hat>Hat.exe ExampleTest1.cs C:\Hat-1.0.0\Project
```

The autotest will be launched, and all the execution steps will be reflected in the console


```

C:\Windows\System32\cmd.exe
C:\Hat-1.0.0\Hat>Hat.exe ExampleTest1.cs C:\Hat-1.0.0\Project

Browser Hat version 1.4.0.0 (09.07.2024)
The browser cache has been cleared
Launching the browser Hat.exe ...
The project is open (project version: 1.4.0.0)
Updated report file Report-ExampleTest1.html

A report has been created with the results of all tests

The autotest file is running: ExampleTest1.cs
Step[process]: Launching the autotest
Step[process]: The autotest file is running: ExampleTest1.cs
Description: Test #1 checks the authorization on the site
Step[completed]: Browser size changed
Step[completed]: Initializing the test
Step[completed]: Initialization of the test has been performed

-- The test begins -----
Step[process]: Page loading https://somovstudio.github.io/test_eng.html
Step[passed]: Page loaded
Step[process]: Waiting 15 seconds
Step[passed]: Waiting for an element - completed (the element is displayed)
Step[passed]: The value 'admin' was entered into the element
Step[process]: Waiting 2 seconds
Step[passed]: Waiting 2 seconds - completed
Step[passed]: The value '0000' was entered into the element
Step[process]: Waiting 2 seconds
Step[passed]: Waiting 2 seconds - completed
Step[passed]: The element is pressed
Step[process]: Waiting 5 seconds
Step[passed]: Waiting for an element - completed (the element is displayed)
Step[passed]: Got the value from the element | Authorization was successful
Step[passed]: The expected and actual value are the same
Step[passed]: The test is completed - all steps are completed successfully

The test is completed - passed
-----
Updated report file Report-ExampleTest1.html

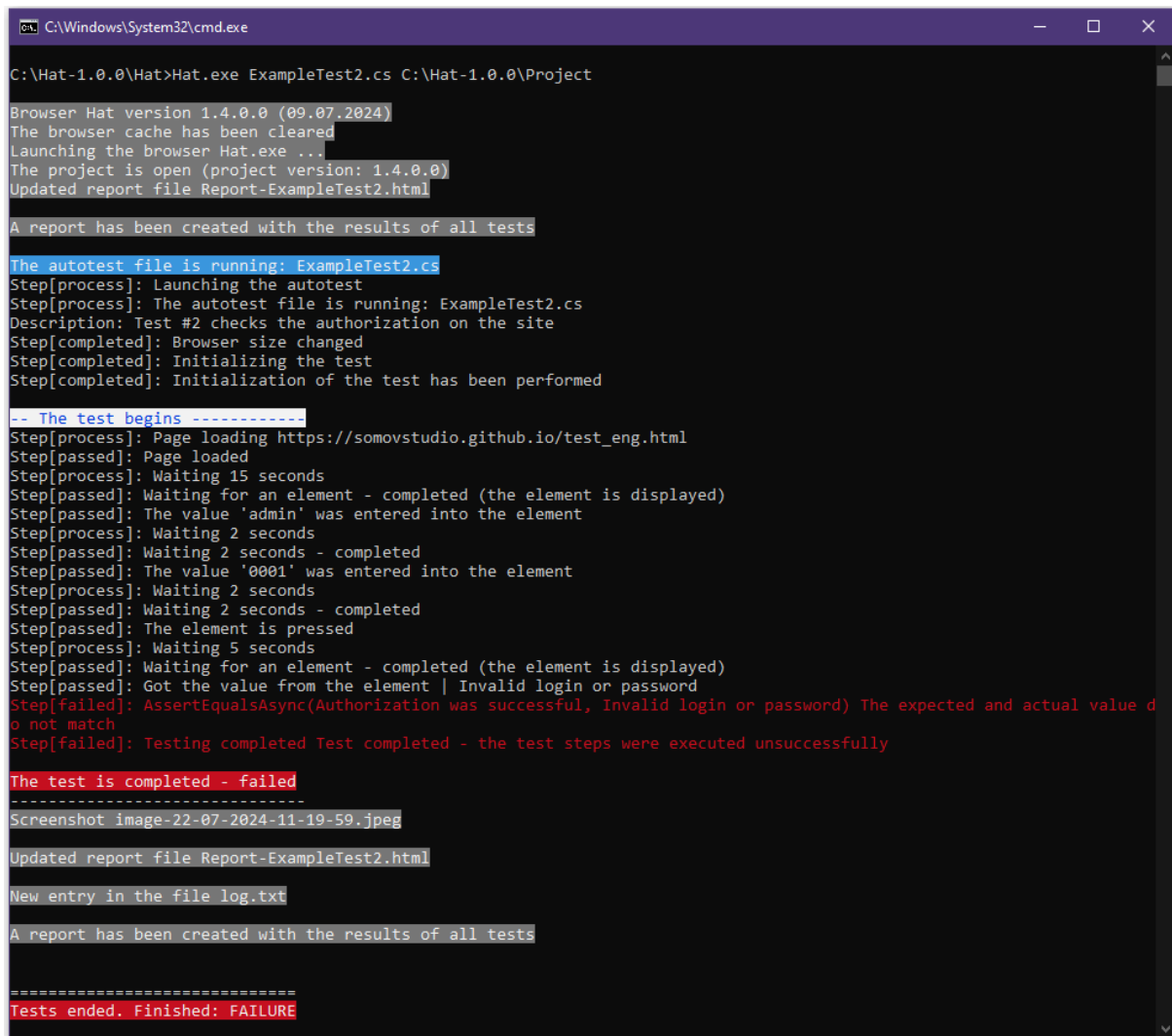
A report has been created with the results of all tests

=====
Tests ended. Finished: SUCCESS
Step[completed]: Closing the browser - completed

C:\Hat-1.0.0\Hat>

```

In case of an error, the step will show the method and its properties where the error occurred.



```

C:\Windows\System32\cmd.exe
C:\Hat-1.0.0\Hat>Hat.exe ExampleTest2.cs C:\Hat-1.0.0\Project
Browser Hat version 1.4.0.0 (09.07.2024)
The browser cache has been cleared
Launching the browser Hat.exe ...
The project is open (project version: 1.4.0.0)
Updated report file Report-ExampleTest2.html

A report has been created with the results of all tests

The autotest file is running: ExampleTest2.cs
Step[process]: Launching the autotest
Step[process]: The autotest file is running: ExampleTest2.cs
Description: Test #2 checks the authorization on the site
Step[completed]: Browser size changed
Step[completed]: Initializing the test
Step[completed]: Initialization of the test has been performed

-- The test begins -----
Step[process]: Page loading https://somovstudio.github.io/test_eng.html
Step[passed]: Page loaded
Step[process]: Waiting 15 seconds
Step[passed]: Waiting for an element - completed (the element is displayed)
Step[passed]: The value 'admin' was entered into the element
Step[process]: Waiting 2 seconds
Step[passed]: Waiting 2 seconds - completed
Step[passed]: The value '0001' was entered into the element
Step[process]: Waiting 2 seconds
Step[passed]: Waiting 2 seconds - completed
Step[passed]: The element is pressed
Step[process]: Waiting 5 seconds
Step[passed]: Waiting for an element - completed (the element is displayed)
Step[passed]: Got the value from the element | Invalid login or password
Step[failed]: AssertEqualsAsync(Authorization was successful, Invalid login or password) The expected and actual value d
0 not match
Step[failed]: Testing completed Test completed - the test steps were executed unsuccessfully

The test is completed - failed
-----
Screenshot Image-22-07-2024-11-19-59.jpeg
Updated report file Report-ExampleTest2.html
New entry in the file log.txt
A report has been created with the results of all tests

=====
Tests ended. Finished: FAILURE

```

At the end, the result of the entire autotest is shown.

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

A text code editor

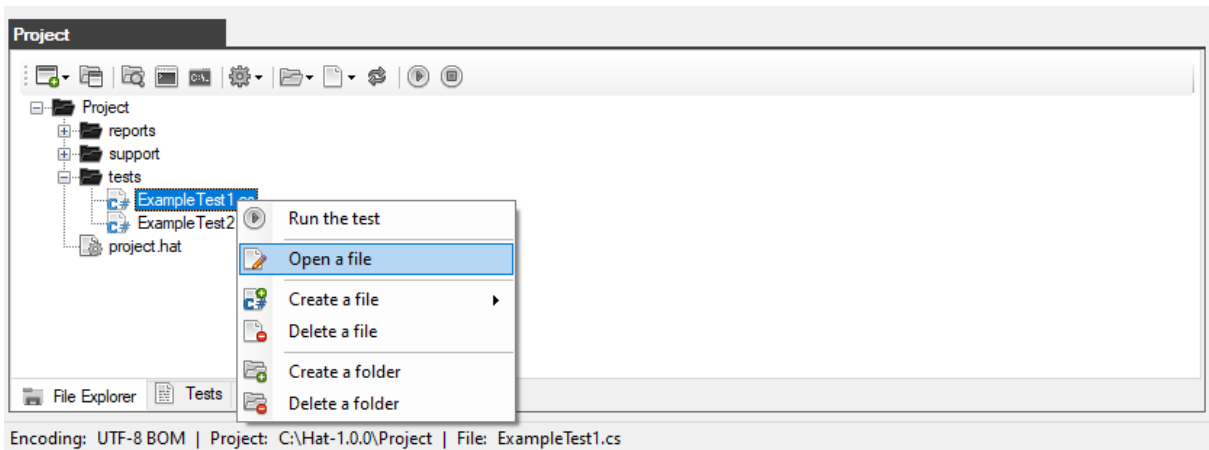
Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

Code Editor (C#)

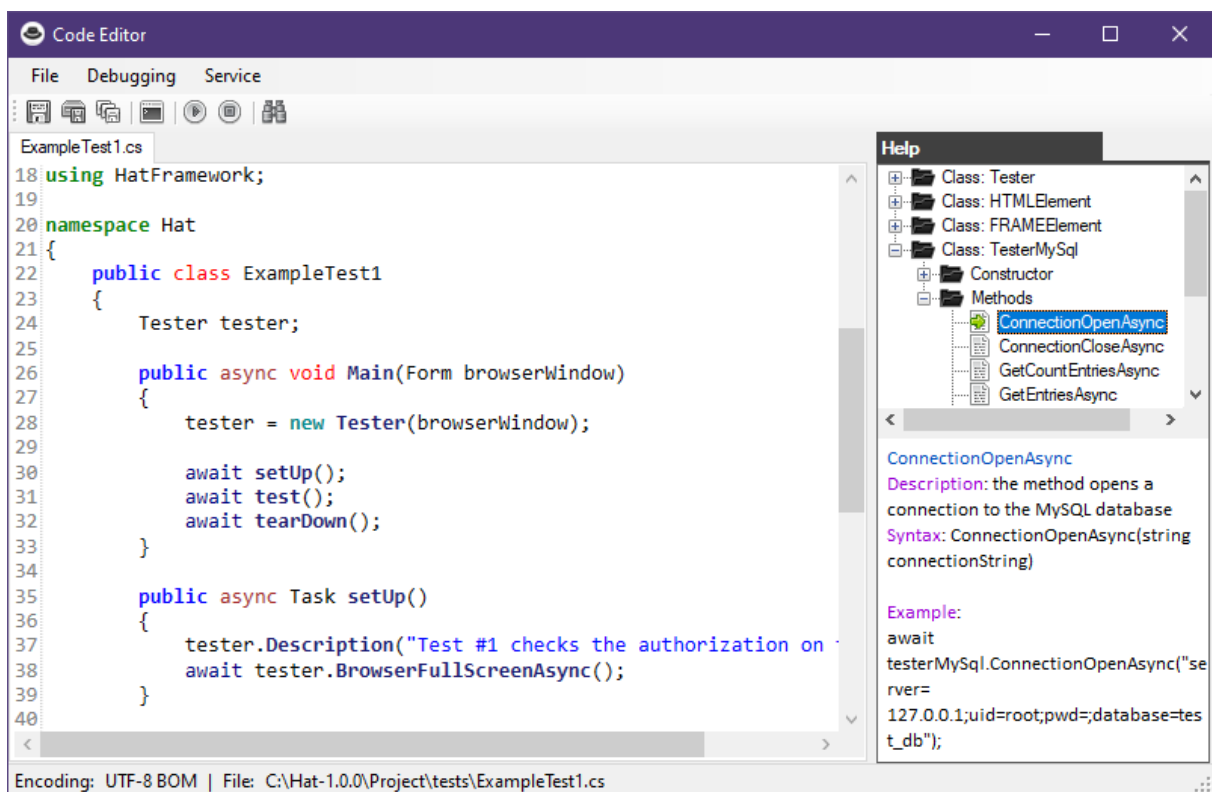
Code Editor (C#)

Autotests are edited in a special editor.

To open a file with the *.cs extension in the editor, you need to double-click on the desired file in the Project window (on the Explorer tab) or open the context menu and select "Open file".

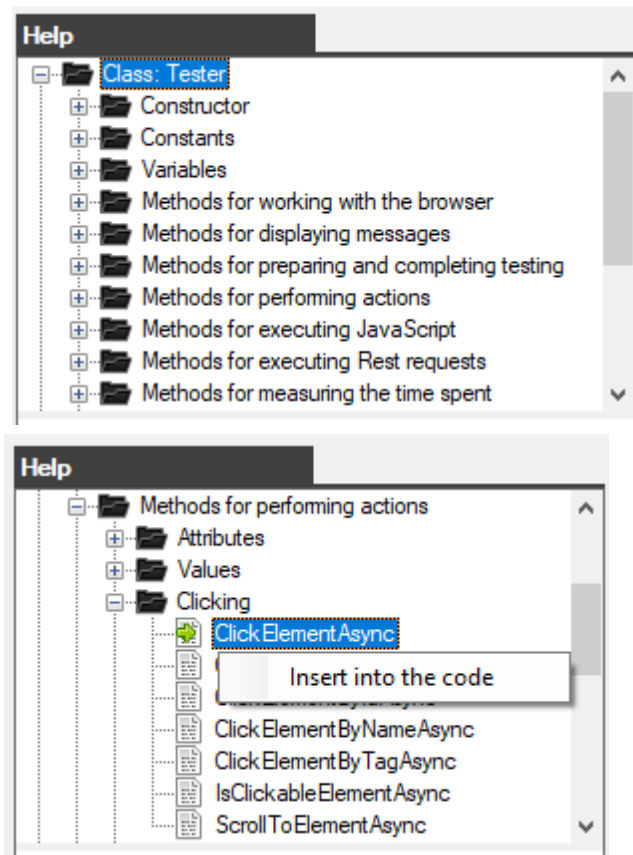


The following editor window will open in front of you.



An editor with keyword highlighting, as well as reference information about the built-in methods of the Net Framework framework.

You do not need to manually type the names of the methods, it will be enough to click on it twice (or through the context menu item) so that the method is automatically inserted into the code.

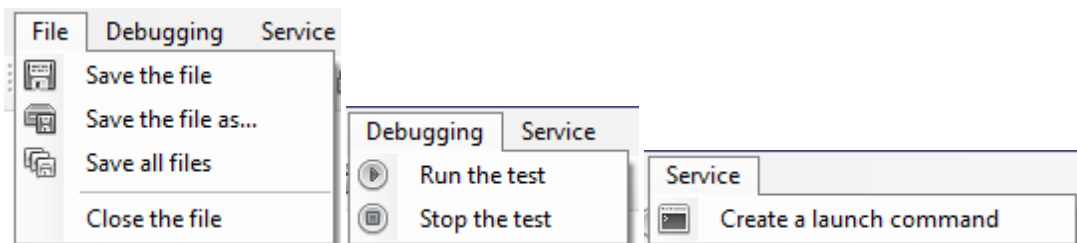


There are buttons on the toolbar:



- o Save File - saves changes to the current file
- o Save file as - allows you to save the script to a new file
- o Save all files - saves changes to all open files
- o Form a startup command - opens a window with an already formed autotest startup command for the Windows command line (cmd)
- o Run the test - runs the selected autotest
- o Stop the test - stops the running autotest
- o Search - executes the search box

The main menu



File:

- o Save File - saves changes to the current file
- o Save file as - allows you to save the script to a new file

- o Save all files - saves changes to all open files
- o Close file - closes the file in the current tab

Debugging:

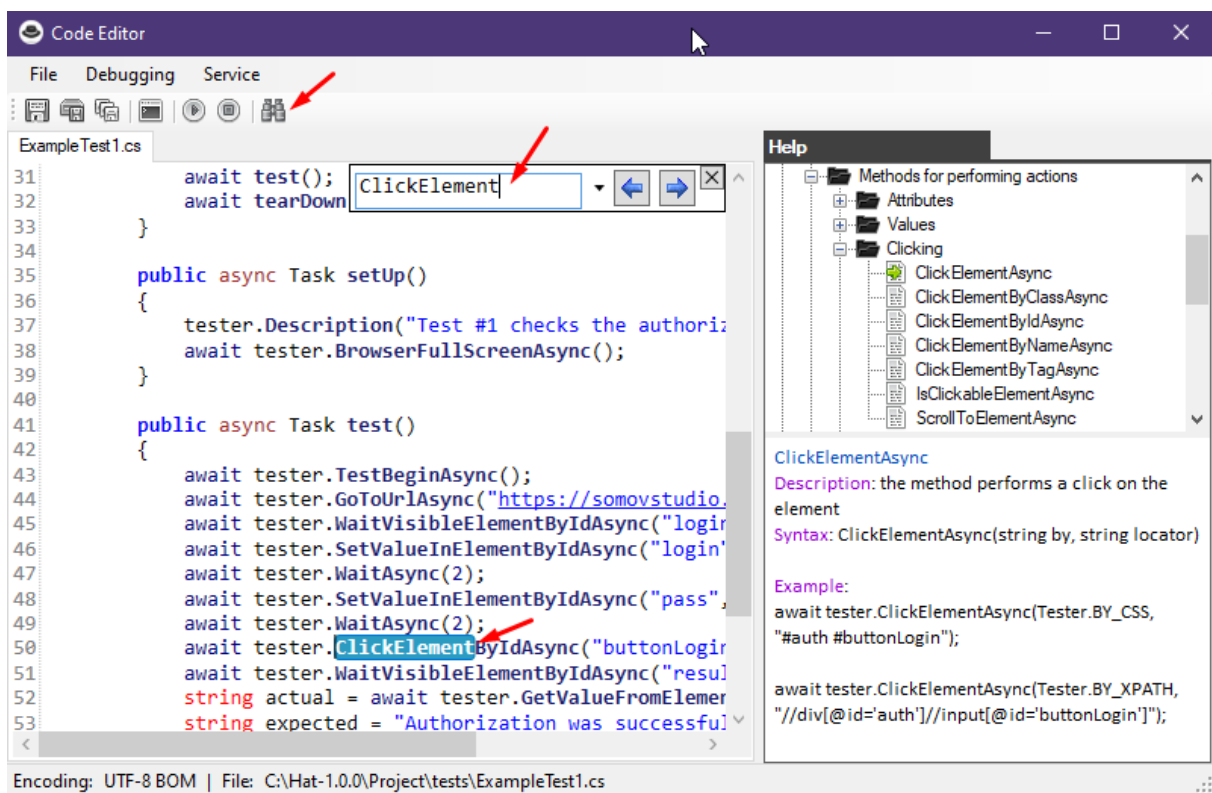
- o Run the test - runs the selected autotest
- o Stop the test - stops the running autotest

Service:

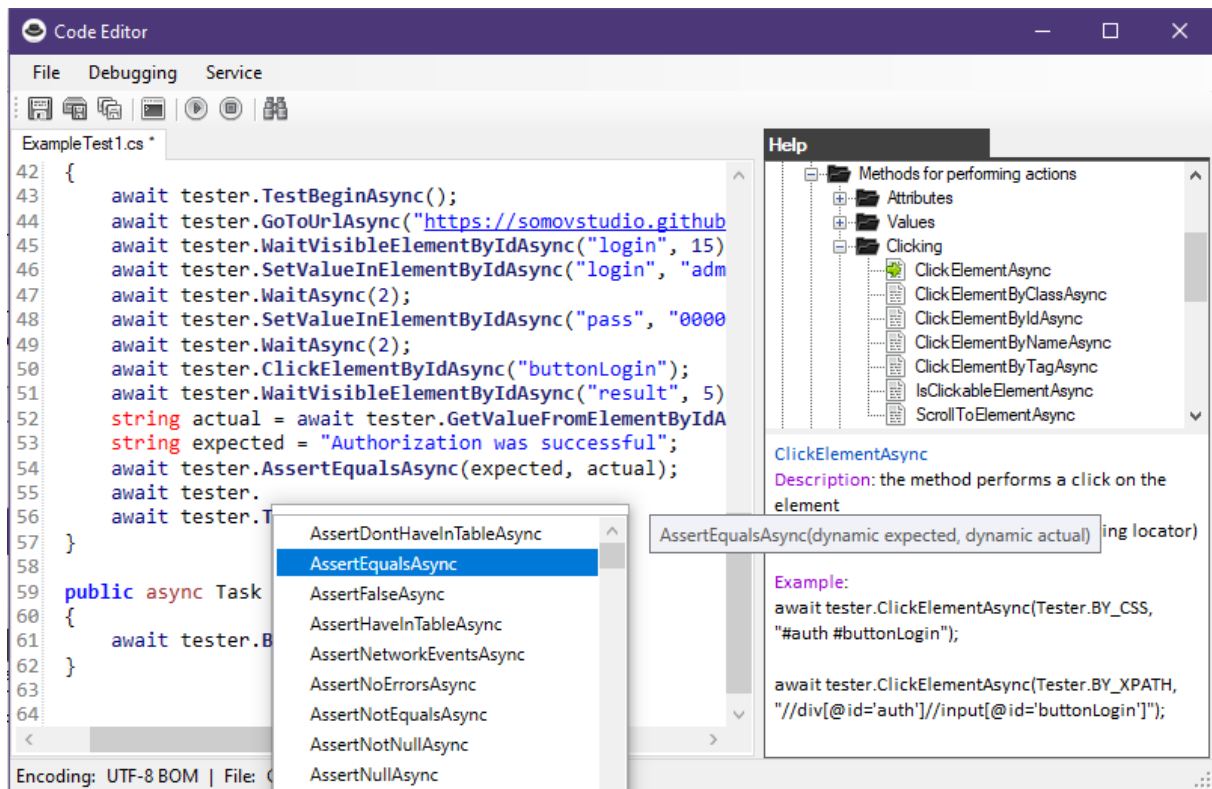
- o Form a startup command - opens a window with an already formed autotest startup command for the Windows command line (cmd)

Performing a search

To open the search window, click on the search button  or press CTRL+F.



The editor allows you to export a list of built-in methods to simplify writing code.



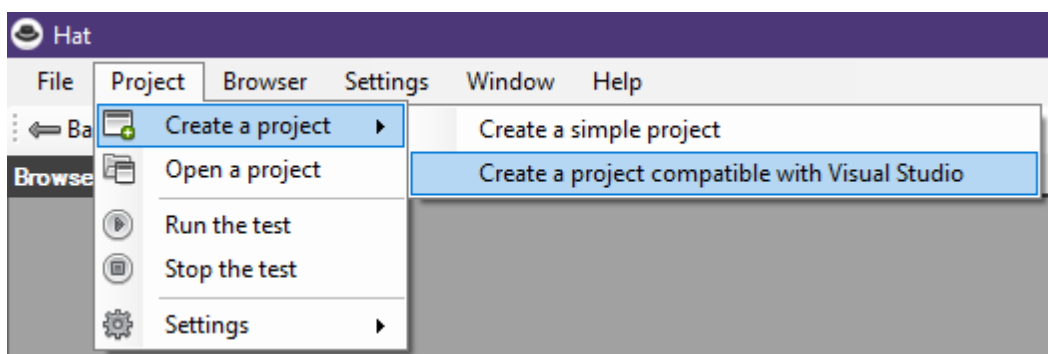
Created with the Personal Edition of HelpNDoc: [Qt Help documentation made easy](#)

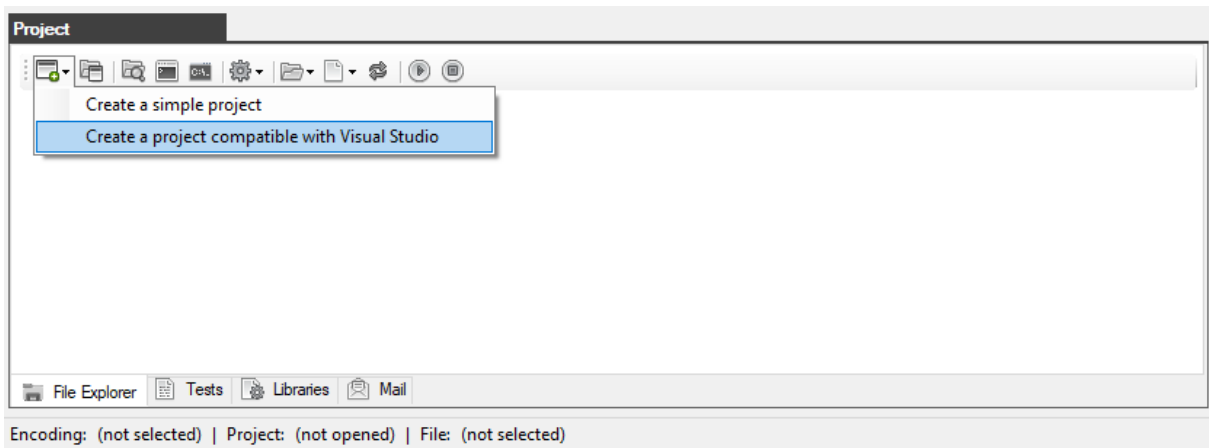
Using Visual Studio to edit code

Using Visual Studio to edit code

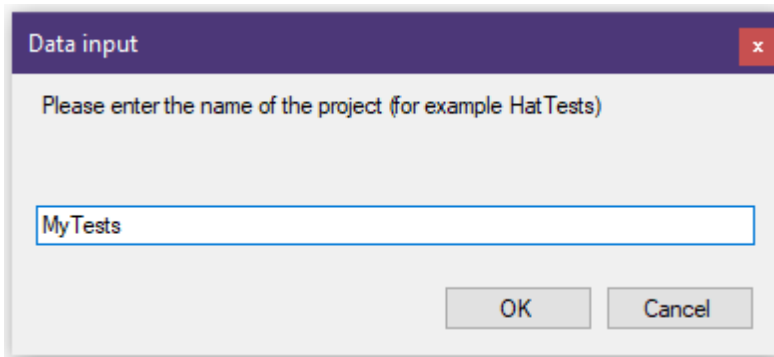
In order to use the autotest project as a Visual Studio editor, it must be created as follows:

1. In the Project window, on the Explorer tab or in the main Create Project menu, click the "Create a project compatible with Visual Studio" button

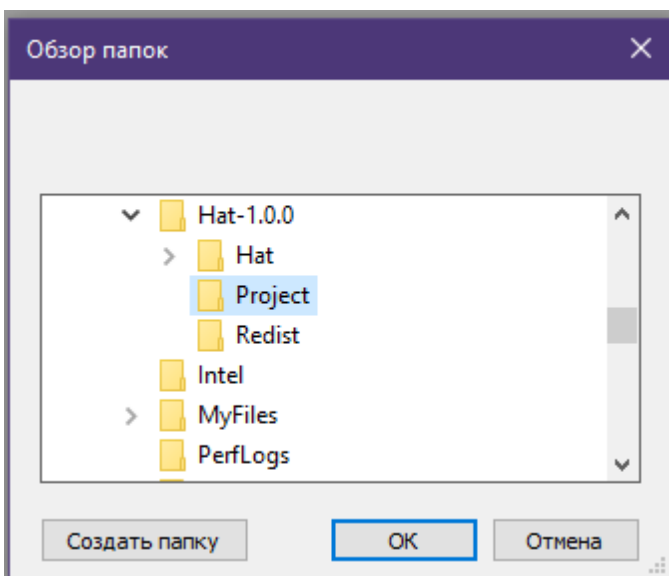




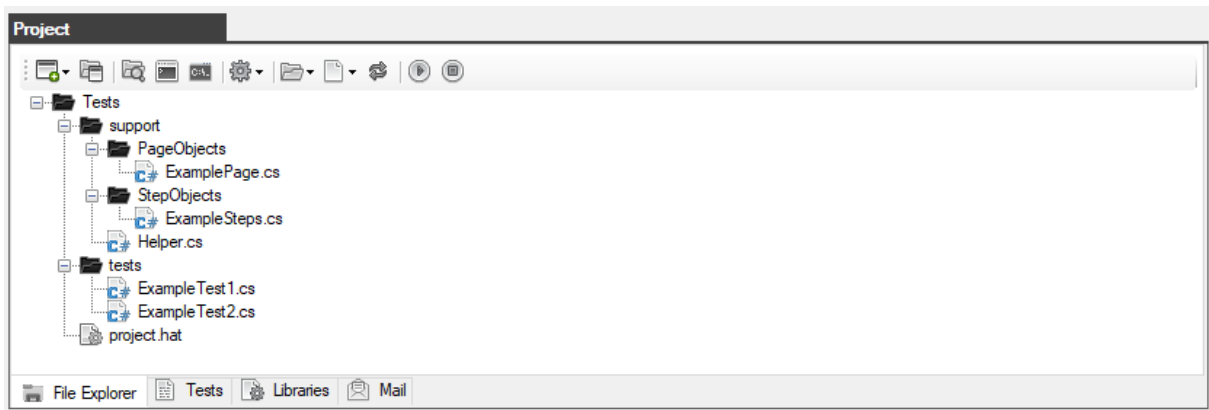
2. A pop-up will open to enter the project name.
Prerequisite the project name must be in Latin letters and without spaces.



3. Create a folder that will contain the project with any name and anywhere on the disk.
In this case, the Project folder has been created at C:\Hat-1.0.0\
Select the project folder and click OK

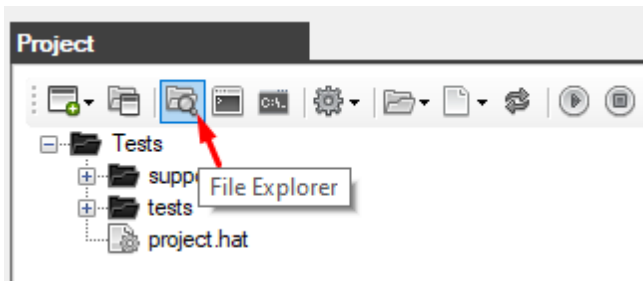


4. In the Project window, the contents of the project will be displayed on the Explorer tab.

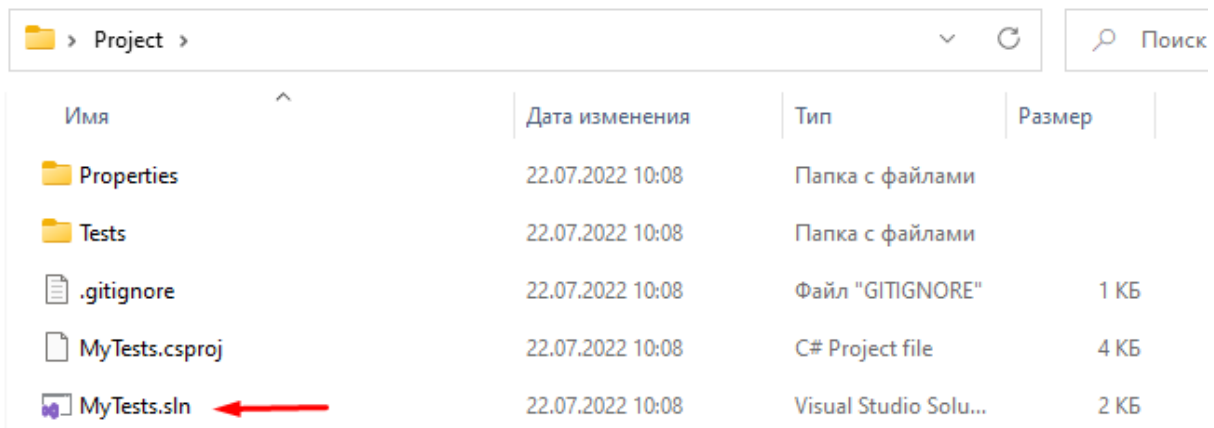


5. This project supports work in Visual Studio (it is assumed that you already have Visual Studio installed)

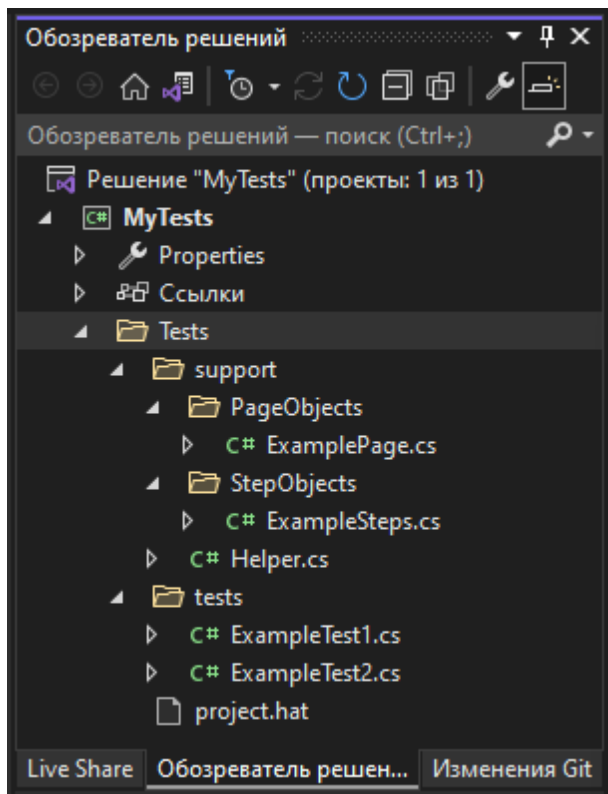
To open a project in Visual Studio, click on the Explorer button in the Project window



Then double-click on the file with the *.sln extension (in this case, MyTests.sln)



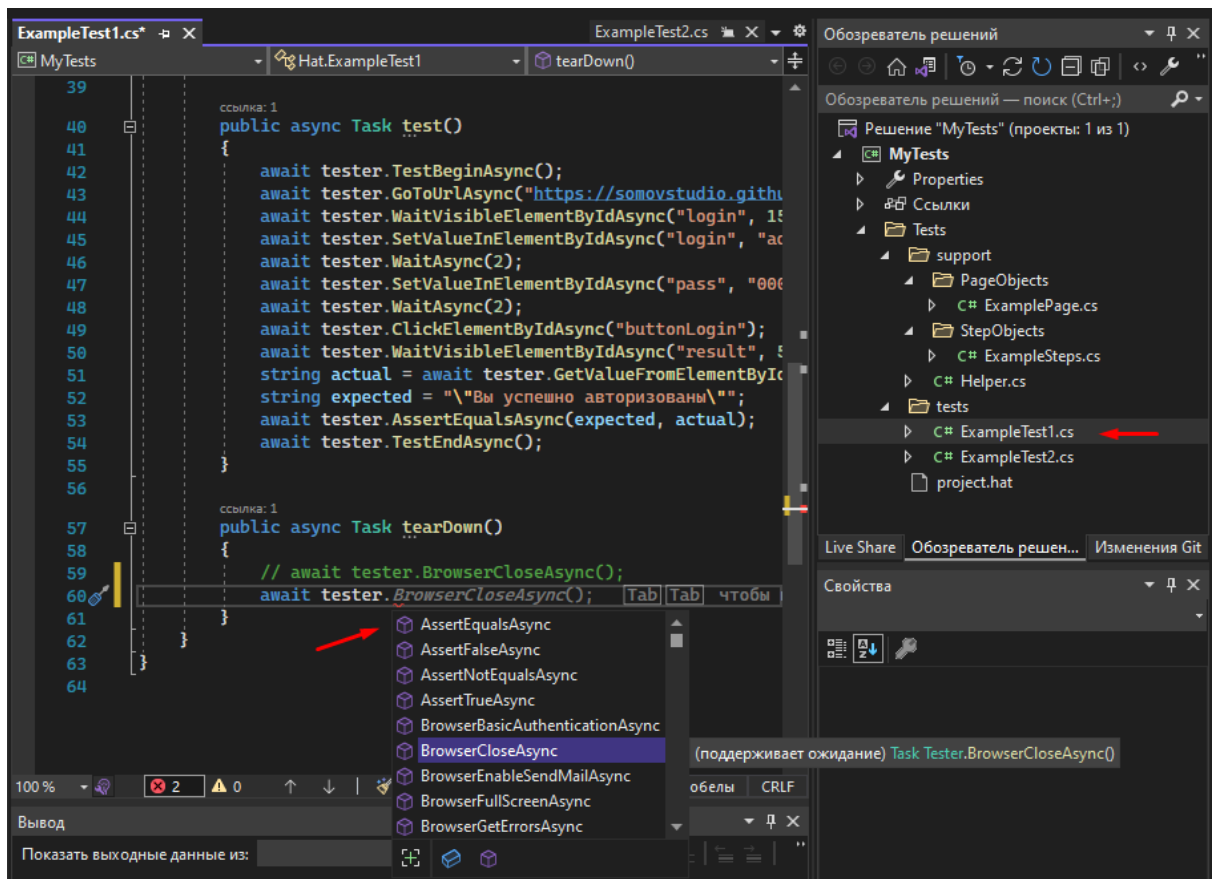
When Visual Studio is loaded, our project will immediately be opened and available for work in the Solution Explorer window



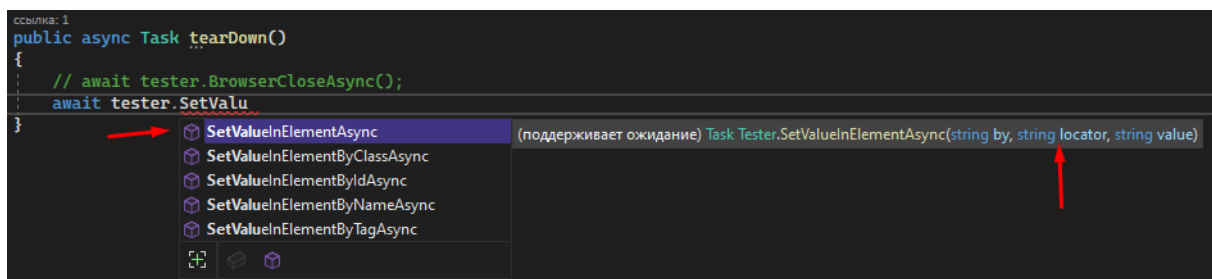
Double-click on the ExampleTest1.cs file to open it in the code editor.

In line 59, a line is commented out with a call to the `BrowserCloseAsync()` method that closes the browser.

This is necessary so that the browser does not automatically close after the completion of the autotest.



It is very convenient to use Visual Studio because the editor, when entering, shows a drop-down list with all available methods, as well as shows the syntax of the selected method and the variables required by it.



The project has been successfully created and is ready to work with Visual Studio.

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

The structure of the autotest

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

Simple project

Simple project

Autotests are described in a programming language C#.

The structure of the autotest:

<pre> using System; using System .Collections.Generic; using System.ComponentModel; using System.Windows.Forms; using System.Threading; using System.Threading.Tasks; using System.IO; using System.Data; using System.Drawing; using System.Linq; using System.Text; using System .Text .RegularExpressions; using System.Net; using System.Net.Http; using System.Net.Http.Headers; using System.Reflection; using Newtonsoft.Json; using HatFramework; </pre>	At the beginning there is a list of plug-in libraries. HatFramework - this is an embedded framework in the browser.
<pre> namespace Hat { public class ExampleTest1 { } } </pre>	Namespace and class. The class name must be the same as the file name.
<pre> Tester tester; public async void Main(Form browserWindow) { tester = new Tester(browserWindow); await setUp(); await test(); await tearDown(); } </pre>	A global variable is declared whose type is Tester The main function with which the autotest starts is described. This function initializes a global variable whose type is Tester. Next comes the call of asynchronous functions.
<pre> public async Task setUp() { } public async Task test() { await </pre>	Asynchronous functions are described that perform: • setUp - function before the test • task - test function • tearDown - function after the test (the names of the functions can be any, the meaning itself and their sequence are important)

```

tester.TestBeginAsync();

        await
tester.TestEndAsync();
}

public async Task
tearDown()
{
}

```

Note: the test should always start with the TestBeginAsync() method and end with the TestEndAsync() method

The demo autotest is complete:

Файл: ExampleTest1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest1
    {
        Tester tester;

        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow);

            await setUp();
            await test();
            await tearDown();
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync();
        }

        public async Task test()
        {
            await tester.TestBeginAsync();

```

```

        await
tester.GoToUrlAsync("https://somovstudio.github.io/test_eng.html", 5);
        await tester.WaitVisibleElementByIdAsync("login", 15);
        await tester.SetValueInElementByIdAsync("login", "admin");
        await tester.WaitAsync(2);
        await tester.SetValueInElementByIdAsync("pass", "0000");
        await tester.WaitAsync(2);
        await tester.ClickElementByIdAsync("buttonLogin");
        await tester.WaitVisibleElementByIdAsync("result", 5);
        string actual = await
tester.GetValueFromElementByIdAsync("textarea");
        string expected = "Authorization was successful";
        await tester.AssertEqualsAsync(expected, actual);
        await tester.TestEndAsync();
    }

    public async Task tearDown()
    {
        await tester.BrowserCloseAsync();
    }
}

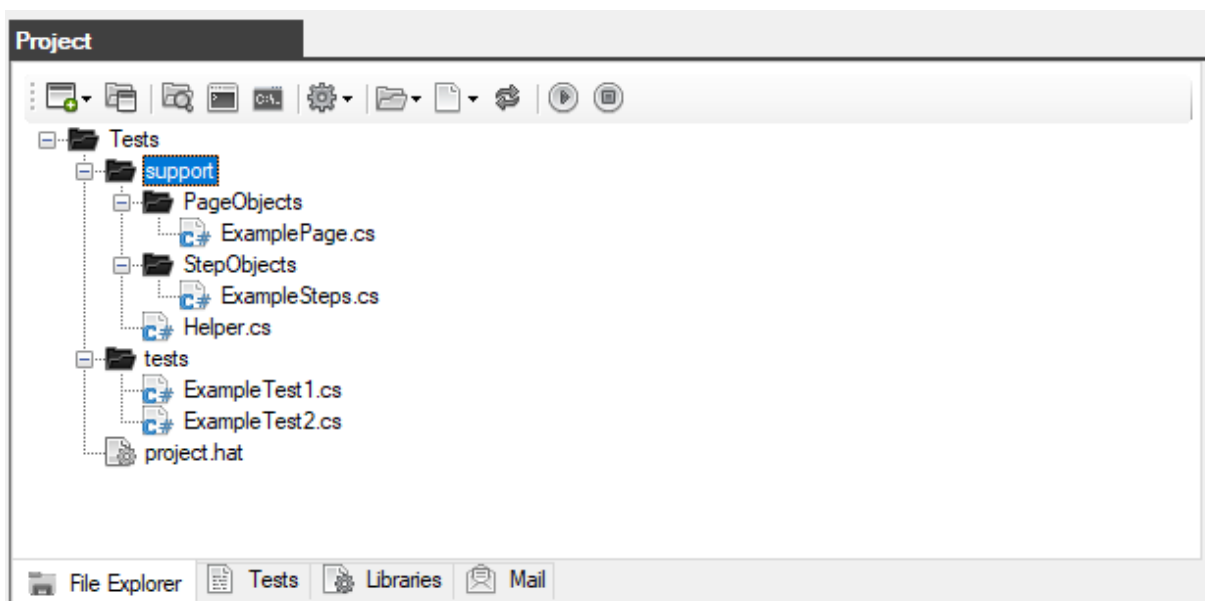
```

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

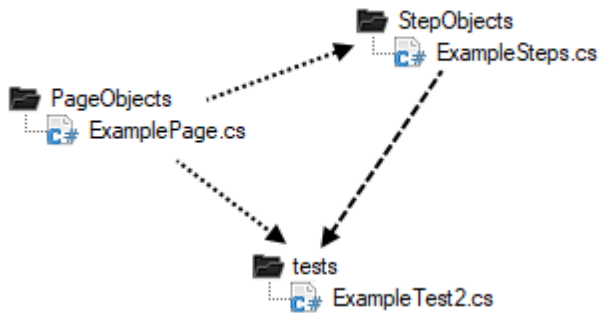
The PageObjects and StepObjects pattern

The PageObjects and StepObjects pattern

Based on the principles of object-oriented programming, you can easily organize the autotest structure using the PageObjects and StepObjects patterns.



The principle of class interaction:



- PageObjects (file ExamplePage.cs) - this is a static class that describes locators and methods that return entire objects to the page.
- StepObjects (file ExampleSteps.cs) - this is a class that inherits all the basic methods from the Tester base class (HatFramework library) and contains its own methods that perform actions with web elements.
- Autotest (file ExampleTest2.cs) - this is a class that directly performs testing using the PageObjects and StepObjects patterns.
- (in this case, the global variable tester has the type ExampleSteps, which inherits the base type Tester)

Demo autotest:

file ExamplePage.cs

```

using System;
using HatFramework;

namespace Hat
{
    public static class ExamplePage
    {
        public static string URL =
@"https://somovstudio.github.io/test_eng.html";
        public static string InputLogin = "login";
        public static string InputPass = "pass";
        public static string ButtonLogin = "buttonLogin";
        public static string Result = "result";
        public static string Textarea = "textarea";
    }
}

```

file ExampleSteps.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;

```

```

using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleSteps : Tester
    {
        public ExampleSteps(Form browserWindow): base(browserWindow) {}

        public async Task FillForm()
        {
            await this.WaitVisibleElementByIdAsync(ExamplePage.InputLogin,
15);
            await this.SetValueInElementByIdAsync(ExamplePage.InputLogin,
"admin");
            await this.WaitAsync(2);
            await this.SetValueInElementByIdAsync(ExamplePage.InputPass,
"0000");
            await this.WaitAsync(2);
        }
    }
}

```

file ExampleTest2.cs

```

using System;
using System.IO;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading;
using System.Threading.Tasks;
using System.Windows.Forms;
using HatFramework;

namespace Hat
{
    public class ExampleTest2
    {
        ExampleSteps tester;

        public async void Main(Form browserWindow)
        {
            tester = new ExampleSteps(browserWindow);

            await setUp();
            await test();
            await tearDown();
        }
    }
}

```

```

public async Task setUp()
{
    await tester.BrowserFullScreenAsync();
}

public async Task test()
{
    await tester.TestBeginAsync();
    await tester.GoToUrlAsync(ExamplePage.URL, 5);
    await tester.FillForm();
    await tester.ClickElementByIdAsync(ExamplePage.ButtonLogin);
    await tester.WaitVisibleElementByIdAsync(ExamplePage.Result, 5);
    string actual = await
tester.GetValueFromElementByIdAsync(ExamplePage.Textarea);
    string expected = "Authorization was successful";
    await tester.AssertEqualsAsync(expected, actual);
    await tester.TestEndAsync();
}

public async Task tearDown()
{
    await tester.BrowserCloseAsync();
}
}

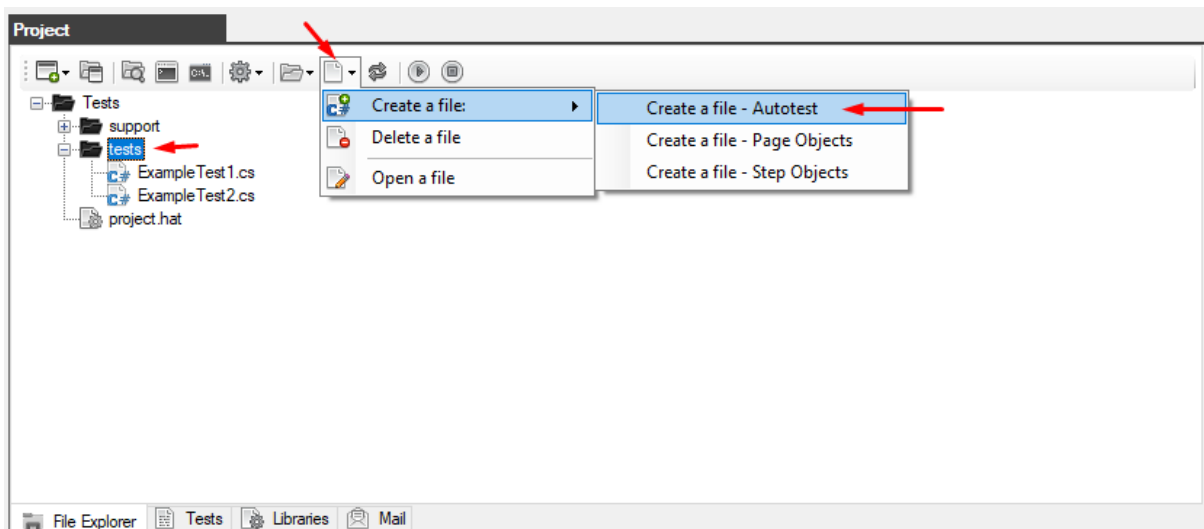
```

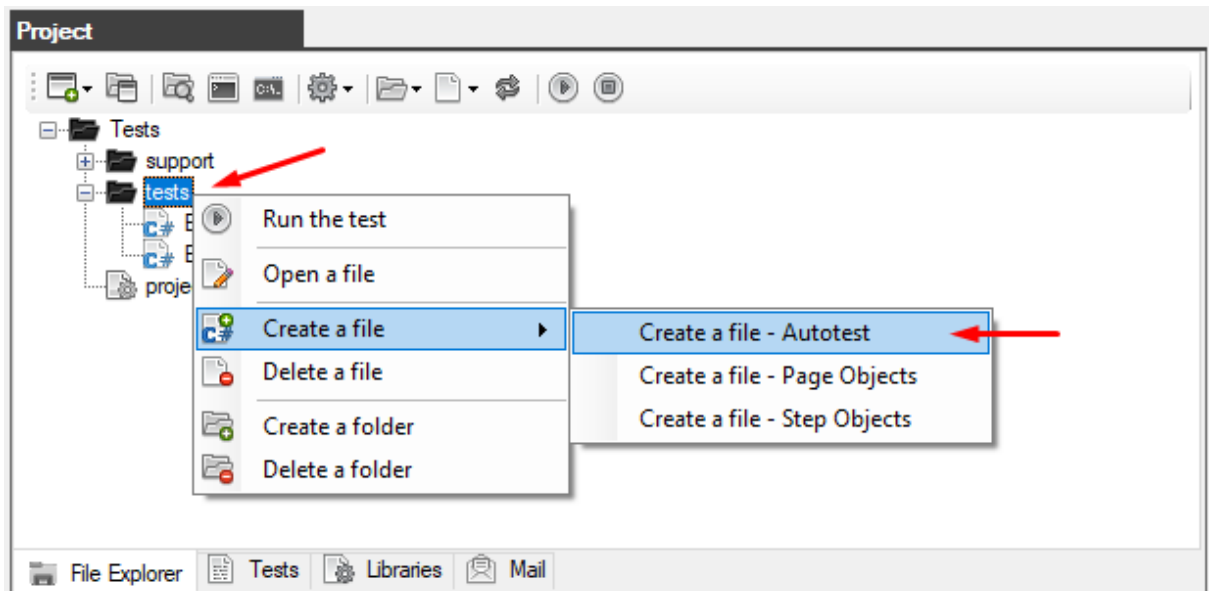
Created with the Personal Edition of HelpNDoc: [Generate EPub eBooks with ease](#)

Creating a new autotest and patterns

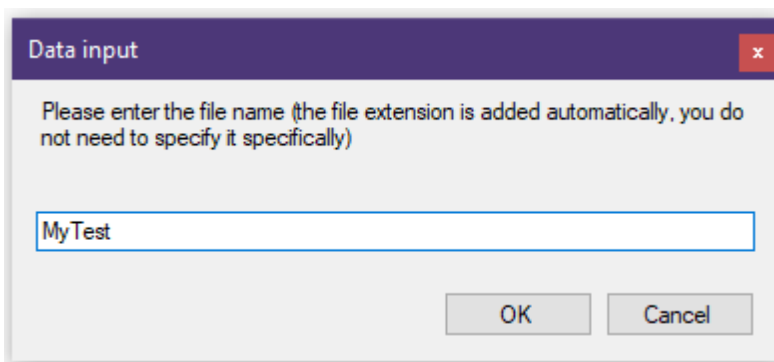
Creating a new autotest

To create a new autotest, you need to select the folder in which you want to create the file in the "Project" window (on the "Explorer" tab), then click on the "Files" button to open the menu and select "Create Autotest file"

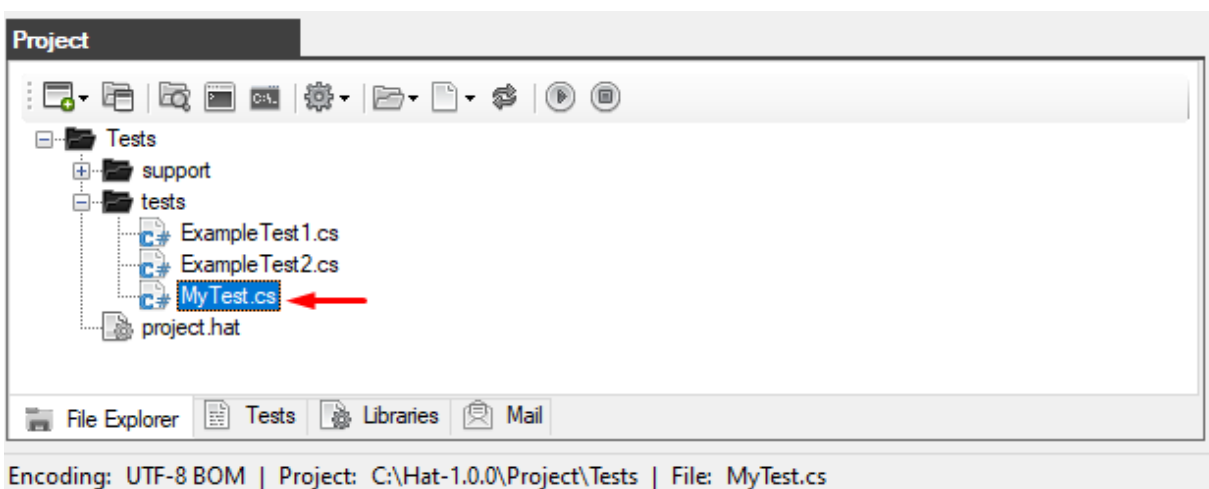




You will be prompted to enter a name for the new file (you do not need to specify the extension)



Click OK and a new autotest file will be created. It will immediately appear in the Project window (on the Explorer tab).



The file is not empty, it will immediately describe all the necessary structure.

```
using System;
```

```
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class MyTest
    {
        Tester tester;

        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow);
            await setUp();
            await test();
            await tearDown();
        }

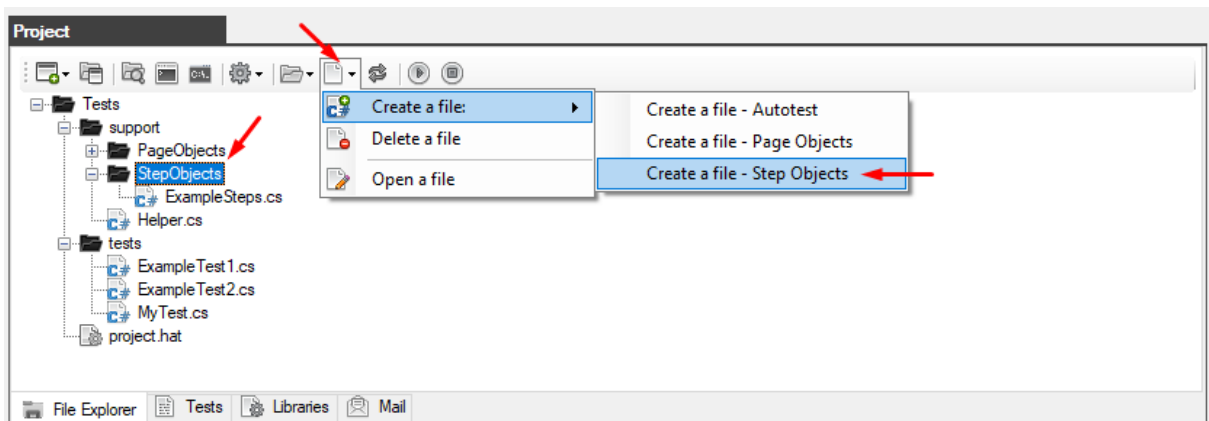
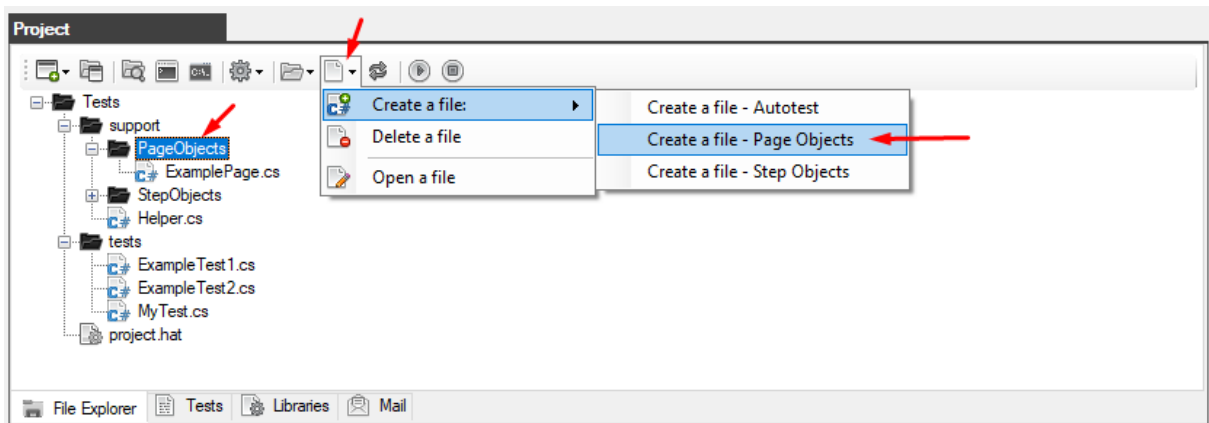
        public async Task setUp()
        {
        }

        public async Task test()
        {
        }

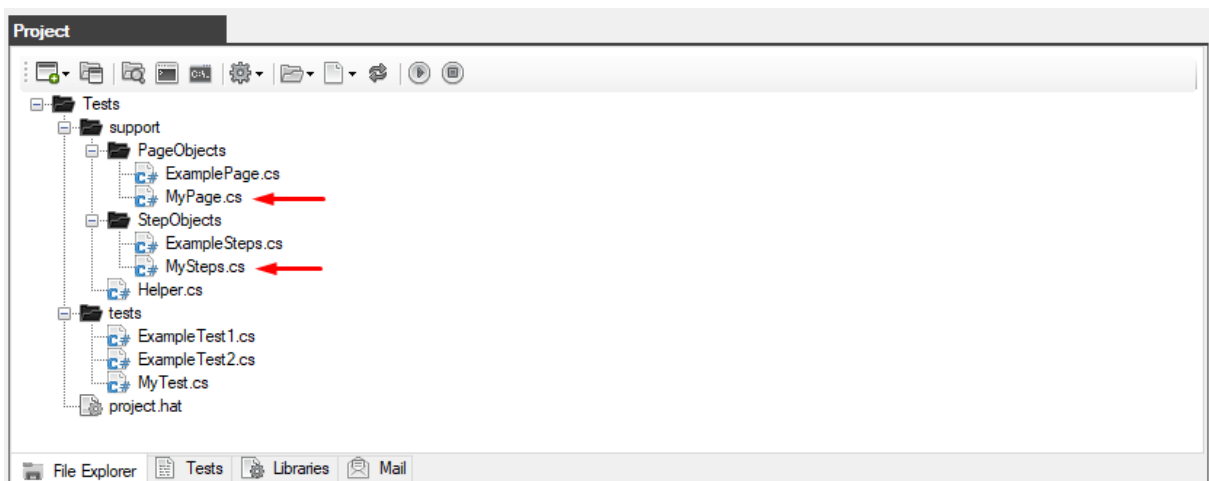
        public async Task tearDown()
        {
        }
    }
}
```

Creating patterns

Special templates are also provided for creating the PageObjects and StepObjects pattern files.



As a result, the following pattern files with a basic description will be created



file: MyPage.cs

```
using System;
using HatFramework;

namespace Hat
{
    public static class MyPage
    {
        public static string URL =
```

file: MySteps.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
```

<pre>"https://test.com/"; public static string ButtonLogin = "buttonLogin"; } }</pre>	<pre>using System.Drawing; using System.Linq; using System.Text; using System.Text.RegularExpressions; using System.Net; using System.Net.Http; using System.Net.Http.Headers; using System.Reflection; using Newtonsoft.Json; using HatFramework; namespace Hat { public class MySteps : Tester { public MySteps(Form browserWindow): base(browserWindow) {} public async Task Test() { await this.AssertTrueAsync(true); } } }</pre>
---	--

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

HatFramework

Created with the Personal Edition of HelpNDoc: [Easily create PDF Help documents](#)

Class: Tester

Created with the Personal Edition of HelpNDoc: [Easily create Web Help sites](#)

Constructor

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

Tester

Tester

Description: the main class of autotests

Syntax: Tester(Form browserForm)

Example:

```
Tester tester;
public async void Main(Form browserWindow)
{
    tester = new Tester(browserWindow);
}
```

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

Constants

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

BY_CSS

BY_CSS

Description: the constant indicates the type of locator being entered

Syntax: BY_CSS = "BY_CSS"

Example:

Tester.BY_CSS

Created with the Personal Edition of HelpNDoc: [Free iPhone documentation generator](#)

BY_XPATH

BY_XPATH

Description: the constant indicates the type of locator being entered

Syntax: BY_XPATH = "BY_XPATH"

Example:

Tester.BY_XPATH

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

IMAGE_STATUS_PROCESS

IMAGE_STATUS_PROCESS

Description: the index of the image that indicates the status in the process

Syntax: IMAGE_STATUS_PROCESS = 0

Example:

Tester.IMAGE_STATUS_PROCESS

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

IMAGE_STATUS_PASSED

IMAGE_STATUS_PASSED

Description: the index of the image that indicates the status of success

Syntax: IMAGE_STATUS_PASSED = 1

Example:

Tester.IMAGE_STATUS_PASSED

IMAGE_STATUS_FAILED

IMAGE_STATUS_FAILED

Description: the index of the image that indicates the failed status

Syntax: IMAGE_STATUS_FAILED = 2

Example:

Tester.IMAGE_STATUS_FAILED

IMAGE_STATUS_MESSAGE

IMAGE_STATUS_MESSAGE

Description: the index of the image that indicates the status of the message

Syntax: IMAGE_STATUS_MESSAGE = 3

Example:

Tester.IMAGE_STATUS_MESSAGE

IMAGE_STATUS_WARNING

IMAGE_STATUS_WARNING

Description: the index of the image that indicates the warning status

Syntax: IMAGE_STATUS_WARNING = 4

Example:

Tester.IMAGE_STATUS_WARNING

IMAGE_STATUS_DEBUG

IMAGE_STATUS_DEBUG

Description: the index of the image that indicates the debug status

Syntax: IMAGE_STATUS_DEBUG = 5

Example:

Tester.IMAGE_STATUS_DEBUG

COMPLETED

COMPLETED

Description: the constant indicates the completed status

Syntax: COMPLETED = "COMPLETED"

Example:

Tester.COMPLETED

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

FAILED

FAILED

Description: the constant indicates the failed status

Syntax: FAILED = "FAILED"

Example:

Tester.FAILED

Created with the Personal Edition of HelpNDoc: [Easily create EPub books](#)

PASSED

PASSED

Description: the constant indicates the success status

Syntax: PASSED = "PASSED"

Example:

Tester.PASSED

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

PROCESS

PROCESS

Description: the constant indicates the status in progress

Syntax: PROCESS = "PROCESS"

Example:

Tester.PROCESS

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

STOPPED

STOPPED

Description: the constant indicates the stopped status

Syntax: STOPPED = "STOPPED"

Example:

Tester.STOPPED

Created with the Personal Edition of HelpNDoc: [Free EPub producer](#)

WARNING

WARNING

Description: the constant indicates the warning status

Syntax: WARNING = "WARNING"

Example:

Tester.WARNING

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

DEBUG

DEBUG

Description: the constant indicates the debug status

Syntax: DEBUG = "DEBUG"

Example:

Tester.DEBUG

Created with the Personal Edition of HelpNDoc: [Free Web Help generator](#)

DEFAULT

DEFAULT

Description: the constant indicates the encoding type for the file

Syntax: DEFAULT = "DEFAULT"

Example:

```
string text = await tester.FileReadAsync(Tester.DEFAULT, "C:\\\\Hat\\file.txt");  
await tester.FileWriteAsync(text, Tester.DEFAULT, "C:\\\\Hat\\file_copy.txt");
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

UTF8

UTF8

Description: the constant indicates the encoding type for the file

Syntax: UTF8 = "UTF-8"

Example:

```
string text = await tester.FileReadAsync(Tester.UTF8, "C:\\Hat\\file.txt");
await tester.FileWriteAsync(text, Tester.UTF8, "C:\\Hat\\file_copy.txt");
```

Created with the Personal Edition of HelpNDoc: [Generate EPub eBooks with ease](#)

UTF8BOM**UTF8BOM**

Description: the constant indicates the encoding type for the file

Syntax: UTF8BOM = "UTF-8 BOM"

Example:

```
string text = await tester.FileReadAsync(Tester.UTF8BOM, "C:\\Hat\\file.txt");
await tester.FileWriteAsync(text, Tester.UTF8BOM, "C:\\Hat\\file_copy.txt");
```

Created with the Personal Edition of HelpNDoc: [Write eBooks for the Kindle](#)

WINDOWS1251**WINDOWS1251**

Description: the constant indicates the encoding type for the file

Syntax: WINDOWS1251 = "WINDOWS-1251"

Example:

```
string text = await tester.FileReadAsync(Tester.WINDOWS1251, "C:\\Hat\\file.txt");
await tester.FileWriteAsync(text, Tester.WINDOWS1251, "C:\\Hat\\file_copy.txt");
```

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

Variables

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

BrowserView**BrowserView**

Description: the variable refers to an object representing the browser area

Syntax: WebView2 BrowserView

Example:

```
tester.BrowserView.Refresh();
tester.BrowserView.Reload();
tester.BrowserView.Source = new Uri(url);
tester.BrowserView.Update();
```

Created with the Personal Edition of HelpNDoc: [Write eBooks for the Kindle](#)

BrowserWindow

BrowserWindow

Description: the variable refers to the application window

Syntax: Form BrowserWindow

Example:

```
Tester tester;
public async void Main(Form browserWindow)
{
    tester = new Tester(browserWindow);
    tester.BrowserWindow.Text = "Browser Hat";
    tester.BrowserWindow.Width = 800;
    tester.BrowserWindow.Height = 600;
    tester.BrowserWindow.Close();
}
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

Locators

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

Locator

Locator

Description: an additional class for creating locators

Syntax:

```
public class Locator
{
    public string name { get; set; }
    public string description { get; set; }
    public string type { get; set; }
    public string value { get; set; }
}
```

Example:

```
Locator locator = new Locator();
locator.name = "inputLogin";
locator.description = "This is the login field";
locator.type = Tester.BY_XPATH;
locator.value = "//input[@id='login']";
```

Created with the Personal Edition of HelpNDoc: [Full-featured Documentation generator](#)

AddLocator

AddLocator

Description: the method creates a locator and adds it to the shared locator storage.

Syntax: AddLocator(string name, string type, string value, string description)

Return value: no return value

Example:

```
tester.AddLocator("inputLogin", Tester.BY_XPATH, "//input[@id='login']", "This is the login field");
```

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

GetLocators

GetLocators

Description: the method returns the repositories of locators

Syntax: GetLocators()

Return value: Dictionary<string, Locator>

Example:

```
Dictionary<string, Locator> locators = tester.GetLocators();
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

GetLocator

GetLocator

Description: the method returns the locator from the locator storage as an object whose class is Locator

Syntax: GetLocator(string name)

Return value: object (Locator)

Example:

```
await tester.WaitVisibleElementAsync(tester.GetLocator("inputLogin"), 25);
```

Created with the Personal Edition of HelpNDoc: [Easily create EBooks](#)

GetLocatorValue

GetLocatorValue

Description: the method returns the locator value from the locator storage

Syntax: GetLocatorValue(string name)

Return value: string

Example:

```
string value = tester.GetLocatorValue("inputLogin");
```

GetCountLocators

GetCountLocators

Description: the method returns the number of locators in the locator storage

Syntax: GetCountLocators()

Return value: int

Example:

```
int count = tester.GetCountLocators();
```

ClearLocators

ClearLocators

Description: the method clears the locator storage

Syntax: ClearLocators()

Return value: no return value

Example:

```
tester.ClearLocators();
```

RemoveLocator

RemoveLocator

Description: the method removes the locator from the locator storage and returns a logical result true or false.

Syntax: RemoveLocator(string name)

Return value: bool (true or false)

Example:

```
bool result = tester.RemoveLocator("inputLogin");
```

Methods for working with the browser

BrowserBasicAuthenticationAsync

BrowserBasicAuthenticationAsync

Description: the browser's navigation method performs the forward action with the specified wait in seconds

Syntax: BrowserBasicAuthenticationAsync(string user, string pass)

Return value: no return value

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.BrowserBasicAuthenticationAsync("user", "pass");
await tester.GoToUrlAsync("http://test.ru/basic_auth.html", 5);
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Easily create EBooks](#)

BrowserClearNetworkAsync

[BrowserClearNetworkAsync](#)

Description: the method cleans network events in the browser

Syntax: BrowserClearNetworkAsync()

Return value: no return value

Example:

```
string events = await tester.BrowserClearNetworkAsync();
tester.ConsoleMsg(events);
```

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

BrowserCloseAsync

[BrowserCloseAsync](#)

Description: the method closes the browser window

Syntax: BrowserCloseAsync()

Return value: no return value

Example:

```
await tester.BrowserCloseAsync();
```

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

BrowserEnableSendMailAsync

[BrowserEnableSendMailAsync](#)

Description: the method includes the option to send a report to the mail in case of failure of the autotest

Syntax: BrowserEnableSendMailAsync(bool byFailure = true, bool bySuccess = true)

Return value: no return value

Example:

```
Tester tester = new Tester(browserForm);
await tester.BrowserEnableSendMailAsync(true, false); // only in case of failure
await tester.BrowserEnableSendMailAsync(false, true); // only if successful
```

```
await tester.BrowserEnableSendMailAsync(); // in both cases
await tester.TestBeginAsync();
...
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

BrowserFullScreenAsync

BrowserFullScreenAsync

Description: the method sets the full-screen size of the browser

Syntax: BrowserFullScreenAsync()

Return value: no return value

Example:

```
await tester.BrowserFullScreenAsync();
```

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

BrowserScreenshotAsync

BrowserScreenshotAsync

Description: the method takes a screenshot of the browser screen

Syntax: BrowserScreenshotAsync(string filename)

Return value: string

Example:

```
string screenshot = await tester.BrowserScreenshotAsync(null);
screenshot = await tester.BrowserScreenshotAsync("");

screenshot = await tester.BrowserScreenshotAsync("screenshot.jpg");
screenshot = await tester.BrowserScreenshotAsync("C:\\Users\\User\\Desktop\\MyTests\\
\\reports\\screenshots\\test_screenshot.jpg");

tester.ConsoleMsg(screenshot);
```

Created with the Personal Edition of HelpNDoc: [Easily create HTML Help documents](#)

BrowserSizeAsync

BrowserSizeAsync

Description: the method sets the browser size

Syntax: BrowserSizeAsync(int width, int height)

Return value: no return value

Example:

```
await tester.BrowserSizeAsync(800, 600);
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

BrowserGetUserAgentAsync

BrowserGetUserAgentAsync

Description: the method returns the lowercase value of the browser's User-Agent

Syntax: BrowserGetUserAgentAsync()

Return value: string

Example:

```
string ua = await tester.BrowserGetUserAgentAsync();
```

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

BrowserSetUserAgentAsync

BrowserSetUserAgentAsync

Description: the method sets the User-Agent value for the browser

Syntax: BrowserSetUserAgent(string value)

Return value: no return value

Example:

```
await tester.TestBeginAsync();
await tester.BrowserSetUserAgent("my user-agent");
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

BrowserGetErrorsAsync

BrowserGetErrorsAsync

Description: the method returns a list of browser errors and warnings

Syntax: BrowserGetErrorsAsync()

Return value: List

Example:

```
List<string> errors = await tester.BrowserGetErrorsAsync();
foreach (string error in errors)
{
    tester.ConsoleMsg(error);
}
```

Created with the Personal Edition of HelpNDoc: [Easy CHM and documentation editor](#)

BrowserGetNetworkAsync

BrowserGetNetworkAsync

Description: the method returns all current messages from the network in json format

Syntax: BrowserGetNetworkAsync()

Return value: string

Example:

```
string events = await tester.BrowserGetNetworkAsync();
tester.ConsoleMsg(events);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

BrowserGoBackAsync**BrowserGoBackAsync**

Description: the browser's navigation method performs the action backwards with the specified wait in seconds

The abortLoadAfterTime flag automatically completes page loading after the timer is completed

Syntax: BrowserGoBackAsync(int sec, bool abortLoadAfterTime = false)

Return value: no return value

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://www.yahoo.com/", 5);
string currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://www.yahoo.com/", currentUrl);
```

```
await tester.GoToUrlAsync("https://yandex.ru/", 5);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://yandex.ru/", currentUrl);
```

```
await tester.BrowserGoBackAsync(10);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://www.yahoo.com/", currentUrl);
```

```
await tester.BrowserGoForwardAsync(10);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://yandex.ru/", currentUrl);
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

BrowserGoForwardAsync**BrowserGoForwardAsync**

Description: the browser's navigation method performs the forward action with the specified wait in seconds

The abortLoadAfterTime flag automatically completes page loading after the timer is completed

Syntax: BrowserGoForwardAsync(int sec, bool abortLoadAfterTime = false)

Return value: no return value

Example:

```

Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://www.yahoo.com/", 5);
string currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://www.yahoo.com/", currentUrl);

await tester.GoToUrlAsync("https://yandex.ru/", 5);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://yandex.ru/", currentUrl);

await tester.BrowserGoBackAsync(10);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://www.yahoo.com/", currentUrl);

await tester.BrowserGoForwardAsync(10);
currentUrl = await tester.GetUrlAsync();
await tester.AssertEqualsAsync("https://yandex.ru/", currentUrl);
await tester.TestEndAsync();

```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

BrowserPageReloadAsync**BrowserPageReloadAsync**

Description: the method reloads the open page in the browser

The abortLoadAfterTime flag automatically completes page loading after the timer is completed

Syntax: BrowserPageReloadAsync(int sec, bool abortLoadAfterTime = false)

Return value: no return value

Example:

```
await tester.BrowserPageReloadAsync(25);
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

Methods for displaying messages

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

ConsoleMsg**ConsoleMsg**

Description: the method displays a message in the browser console

Syntax:

ConsoleMsg(string message)
ConsoleMsg(string messageRus, string messageEng)
Return value: no return value

Example:

```
tester.ConsoleMsg("Текст на Русском языке", "The text is in English");
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

ConsoleMsgError

ConsoleMsgError

Description: the method outputs an error message to the system console and the browser console

Syntax: ConsoleMsgError(string message)

Return value: no return value

Example:

```
try
{
}
catch (Exception ex)
{
    tester.ConsoleMsgError(ex.ToString());
}
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

ClearMessage

ClearMessage

Description: the method clears the messages in the output table of the test execution process

Syntax: ClearMessage()

Return value: no return value

Example:

```
tester.ClearMessage();
```

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

Description

Description

Description: the method sets the description of the test, which is then displayed in the report and letter

Syntax: Description(string text)

Return value: no return value

Example:

```
public async Task setUp()
{
    tester.Description("The test checks the authorization on the site");
}
```

Created with the Personal Edition of HelpNDoc: [Write eBooks for the Kindle](#)

DisableDebugInReport**DisableDebugInReport**

Description: this method disables the output of SendMessageDebug debug messages to the report

Syntax: DisableDebugInReport()

Return value: no return value

Example:

```
tester.DisableDebugInReport();
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

SendMessage**SendMessage**

Description: the method displays messages in the test execution process table

When executed, action is output to the system console if status is FAILED, WARNING, "", null, otherwise only status and comment are output (without action).

Syntax: SendMessage(string action, string status, string comment)

Example:

```
tester.SendMessage("the text of the actions", Tester.PROCESS, "the text of the comment");
```

```
/* Examples of console output */
```

```
// action comment
```

```
tester.SendMessage("actions", null, "comment");
```

```
tester.SendMessage("actions", "", "comment");
```

```
// Step[progress]: comment
```

```
tester.SendMessage("actions", Tester.PROCESS, "comment");
```

```
// Step[passed]: comment
```

```
tester.SendMessage("actions", Tester.PASSED, "comment");
```

```
// Step[completed]: comment
```

```
tester.SendMessage("actions", Tester.COMPLETED, "comment");
```

```
// Step[stopped]: comment
```

```
tester.SendMessage("actions", Tester.STOPPED, "comment");
```

```
// Step[failed]: действие комментарий
tester.SendMessage("actions", Tester.FAILED, "comment");
// Step[warning]: действие комментарий
tester.SendMessage("actions", Tester.WARNING, "comment");
```

Created with the Personal Edition of HelpNDoc: [Full-featured Documentation generator](#)

SendMessageDebug

SendMessageDebug

Description: the method outputs a debugging message that can be disabled for output to a report and an email

When executed, action is output to the system console if status is FAILED, WARNING, "", null, otherwise only status and comment are output (without action).

Syntax: SendMessageDebug(string actionRus, string actionEng, string status, string commentRus, string commentEng, int image)

Example:

```
tester.SendMessageDebug("действия", "action", Tester.PROCESS, "комментарий", "comment",
Tester.IMAGE_STATUS_PROCESS);
```

Created with the Personal Edition of HelpNDoc: [Free EPub producer](#)

Methods for preparing and completing testing

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

TestBeginAsync

TestBeginAsync

Description: the method of preparing for the test (each test must necessarily begin with this method)

Syntax: TestBeginAsync()

Return value: no return value

Example:

```
await tester.TestBeginAsync();
```

Created with the Personal Edition of HelpNDoc: [Full-featured Kindle eBooks generator](#)

TestEndAsync

TestEndAsync

Description: the test completion method (each test must necessarily end with this method), it is this method that saves the report and sends it by mail if such an option is configured and

enabled.

Syntax: TestEndAsync()

Return value: no return value

Example:

```
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

TestStopAsync

TestStopAsync

Description: the method forcibly stops the testing process

Syntax: TestStopAsync()

Return value: no return value

Example:

```
await tester.TestStopAsync();
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

GetTestResult

GetTestResult

Description: the method returns the status of the test being performed

Syntax: GetTestResult()

Return value: string (the value of which is checked by constants: Tester.PASSED, Tester.FAILED, Tester.PROCESS)

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
...
if(tester.GetTestResult() == Tester.PROCESS) { }
...
await tester.TestEndAsync();
if(tester.GetTestResult() == Tester.FAILED) { }
if(tester.GetTestResult() == Tester.PASSED) { }
```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

DefineTestStop

DefineTestStop

Description: the method checks the status of the process (stopped or not)

Syntax: DefineTestStop()

Return value: bool (true or false)

Example:

```
if (tester.DefineTestStop() == true) return; // so the test is stopped
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

Methods for performing actions

Created with the Personal Edition of HelpNDoc: [Free iPhone documentation generator](#)

Attributes

Created with the Personal Edition of HelpNDoc: [Easy CHM and documentation editor](#)

GetAttributeFromElementAsync

GetAttributeFromElementAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax:

GetAttributeFromElementAsync(string by, string locator, string attribute)

GetAttributeFromElementAsync(Locator locator, string attribute)

Return value: string

Example:

```
string value = await tester.GetAttributeFromElementAsync(Tester.BY_CSS, "input", "name");
```

```
string value = await tester.GetAttributeFromElementAsync(Tester.BY_XPATH, "//input", "name");
```

```
string value = await tester.GetAttributeFromElementAsync(tester.GetLocator("locatorName"),  
"name");
```

GetAttributeFromElementByClassAsync

GetAttributeFromElementByClassAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax: GetAttributeFromElementByClassAsync(string _class, int index, string attribute)

Return value: string

Example:

```
string value = await tester.GetAttributeFromElementByClassAsync("my-element", 0, "href");
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

GetAttributeFromElementByIdAsync

GetAttributeFromElementByIdAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax: GetAttributeFromElementByIdAsync(string id, string attribute)

Return value: string

Example:

```
string value = await tester.GetAttributeFromElementByIdAsync("MyElement", "href");
```

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

GetAttributeFromElementByNameAsync

GetAttributeFromElementByNameAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax: GetAttributeFromElementByNameAsync(string name, int index, string attribute)

Return value: string

Example:

```
string value = await tester.GetAttributeFromElementByNameAsync("MyElement", 0, "href");
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

GetAttributeFromElementByTagAsync

GetAttributeFromElementByTagAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax: GetAttributeFromElementByTagAsync(string tag, int index, string attribute)

Return value: string

Example:

```
string value = await tester.GetAttributeFromElementByTagAsync("a", 0, "href");
```

Created with the Personal Edition of HelpNDoc: [Produce Kindle eBooks easily](#)

GetAttributeFromElementsAsync

GetAttributeFromElementsAsync

Description: the method returns a list of values of the specified attribute from a set of elements

Syntax:

GetAttributeFromElementsAsync(string by, string locator, string attribute)

GetAttributeFromElementsAsync(Locator locator, string attribute)

Return value: List

Example:

```
List<string> values = await tester.GetAttributeFromElementsAsync(Tester.BY_CSS, "input",
"name");
if(values != null)
{
    foreach (string attr in values)
        tester.ConsoleMsg(attr);
}
```

```
List<string> values = await tester.GetAttributeFromElementsAsync(Tester.BY_XPATH, "//input",
"name");
if(values != null)
{
    foreach (string attr in values)
        tester.ConsoleMsg(attr);
}
```

```
List<string> values = await
tester.GetAttributeFromElementsAsync(tester.GetLocator("locatorName"), "name");
if(values != null)
{
    foreach (string attr in values)
        tester.ConsoleMsg(attr);
}
```

GetAttributeFromElementsByClassAsync

GetAttributeFromElementsByClassAsync

Description: the method returns a list of values of the specified attribute from a set of elements

Syntax: GetAttributeFromElementsByClassAsync(string _class, string attribute)

Return value: List

Example:

```
List<string> values = await tester.GetAttributeFromElementsByClassAsync("text-field", "name");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

GetAttributeFromElementsByNameAsync

GetAttributeFromElementsByNameAsync

Description: the method returns a list of values of the specified attribute from a set of elements

Syntax: GetAttributeFromElementsByNameAsync(string name, string attribute)

Return value: List

Example:

```
List<string> values = await tester.GetAttributeFromElementsByNameAsync("link", "href");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

GetAttributeFromElementsByTagAsync

GetAttributeFromElementsByTagAsync

Description: the method returns a list of values of the specified attribute from a set of elements

Syntax: GetAttributeFromElementsByTagAsync(string tag, string attribute)

Return value: List

Example:

```
List<string> values = await tester.GetAttributeFromElementsByTagAsync("a", "href");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

SetAttributeInElementAsync

SetAttributeInElementAsync

Description: the method inserts an attribute with a value into the specified element

Syntax:

SetAttributeInElementAsync(string by, string locator, string attribute, string value)

SetAttributeInElementAsync(Locator locator, string attribute, string value)

Return value: no return value

Example:

```
await tester.SetAttributeInElementAsync(Tester.BY_CSS, "#auth > h2", "name", "test");  
await tester.SetAttributeInElementAsync(Tester.BY_XPATH, "///div[@id='auth']/h2", "name",  
"test");
```

```
await tester.SetAttributeInElementAsync(tester.GetLocator("locatorName"), "name", "test");
```


SetAttributeInElementByClassAsync

SetAttributeInElementByClassAsync

Description: the method inserts an attribute with a value into the specified element

Syntax: SetAttributeInElementByClassAsync(string _class, int index, string attribute, string value)

Return value: no return value

Example:

```
await tester.SetAttributeInElementByClassAsync("my-element", 0, "value", "значение");
```

Created with the Personal Edition of HelpNDoc: [Full-featured EPub generator](#)

SetAttributeInElementByIdAsync

SetAttributeInElementByIdAsync

Description: the method inserts an attribute with a value into the specified element

Syntax: SetAttributeInElementByIdAsync(string id, string attribute, string value)

Return value: no return value

Example:

```
await tester.SetAttributeInElementByIdAsync("MyElement", "value", "value");
```

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

SetAttributeInElementByNameAsync

SetAttributeInElementByNameAsync

Description: the method inserts an attribute with a value into the specified element

Syntax: SetAttributeInElementByNameAsync(string name, int index, string attribute, string value)

Return value: no return value

Example:

```
await tester.SetAttributeInElementByNameAsync("MyElement", 0, "value", "value");
```

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

SetAttributeInElementByTagAsync

SetAttributeInElementByTagAsync

Description: the method inserts an attribute with a value into the specified element

Syntax: SetAttributeInElementByTagAsync(string tag, int index, string attribute, string value)

Return value: no return value

Example:

```
await tester.SetAttributeInElementByTagAsync("input", 0, "value", "value");
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

SetAttributeInElementsAsync

SetAttributeInElementsAsync

Description: the method inserts an attribute with the specified value into a set of elements and returns a list as a result

Syntax:

SetAttributeInElementsAsync(string by, string locator, string attribute, string value)

SetAttributeInElementsAsync(Locator locator, string attribute, string value)

Return value: List

Example:

```
List<string> values = await tester.SetAttributeInElementsAsync(Tester.BY_CSS, "input", "class",
"test-class");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

```
List<string> values = await tester.SetAttributeInElementsAsync(Tester.BY_XPATH, "//input",
"class", "test-class");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

```
List<string> values = await
tester.SetAttributeInElementsAsync(tester.GetLocator("locatorName"), "class", "test-class");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

SetAttributeInElementsByClassAsync

SetAttributeInElementsByClassAsync

Description: the method inserts an attribute with the specified value into a set of elements and returns a list as a result

Syntax: SetAttributeInElementsByClassAsync(string _class, string attribute, string value)

Return value: List

Example:

```
List<string> values = await tester.SetAttributeInElementsByClassAsync("text-field", "value",  
"test");  
foreach (string value in values)  
{  
    tester.ConsoleMsg(value);  
}
```

SetAttributeInElementsByNameAsync

SetAttributeInElementsByNameAsync

Description: the method inserts an attribute with the specified value into a set of elements and returns a list as a result

Syntax: SetAttributeInElementsByNameAsync(string name, string attribute, string value)

Return value: List

Example:

```
List<string> values = await tester.SetAttributeInElementsByNameAsync("link", "href",  
"www.test.ru");  
foreach (string value in values)  
{  
    tester.ConsoleMsg(value);  
}
```

SetAttributeInElementsByTagAsync

SetAttributeInElementsByTagAsync

Description: the method inserts an attribute with the specified value into a set of elements and returns a list as a result

Syntax: SetAttributeInElementsByTagAsync(string tag, string attribute, string value)

Return value: List

Example:

```
List<string> values = await tester.SetAttributeInElementsByTagAsync("a", "href",  
"www.test.ru");  
foreach (string value in values)  
{  
    tester.ConsoleMsg(value);  
}
```

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

Values

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

GetValueFromElementAsync

GetValueFromElementAsync

Description: the method returns the value from the specified element

Syntax:

GetValueFromElementAsync(string by, string locator)

GetValueFromElementAsync(Locator locator)

Return value: string

Example:

```
string value = await tester.GetValueFromElementAsync(Tester.BY_CSS, "input[id='login']");
```

```
string value = await tester.GetValueFromElementAsync(Tester.BY_XPATH,  
"//input[@id='login']");
```

```
string value = await tester.GetValueFromElementAsync(tester.GetLocator("locatorName");
```

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

GetValueFromElementByClassAsync

GetValueFromElementByClassAsync

Description: the method returns the value from the specified element

Syntax: GetValueFromElementByClassAsync(string _class, int index)

Return value: string

Example:

```
string value = await tester.GetValueFromElementByClassAsync("my-element", 0);
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

GetValueFromElementByIdAsync

GetValueFromElementByIdAsync

Description: the method returns the value from the specified element

Syntax: GetValueFromElementByIdAsync(string id)

Return value: string

Example:

```
string value = await tester.GetValueFromElementByIdAsync("MyElement");
```

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

GetValueFromElementByNameAsync

GetValueFromElementByNameAsync

Description: the method returns the value from the specified element

Syntax: GetValueFromElementByNameAsync(string name, int index)

Return value: строка (string)

Example:

```
string value = await tester.GetValueFromElementByNameAsync("MyElement", 0);
```

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

GetValueFromElementByTagAsync

GetValueFromElementByTagAsync

Description: the method returns the value from the specified element

Syntax: GetValueFromElementByTagAsync(string tag, int index)

Return value: string

Example:

```
string value = await tester.GetValueFromElementByTagAsync("input", 0);
```

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

SetValueInElementAsync

SetValueInElementAsync

Description: the method inserts value into the specified element

Syntax:

SetValueInElementAsync(string by, string locator, string value)

SetValueInElementAsync(Locator locator, string value)

Return value: no return value

Example:

```
await tester.SetValueInElementAsync(Tester.BY_CSS, "input[id='login']", "admin");
```

```
await tester.SetValueInElementAsync(Tester.BY_CSS, "input[id='pass']", "0000");
```

```
await tester.SetValueInElementAsync(Tester.BY_XPATH, "//input[@id='login']", "admin");
```

```
await tester.SetValueInElementAsync(Tester.BY_XPATH, "//input[@id='pass']", "0000");
```

```
await tester.SetValueInElementAsync(tester.GetLocator("locatorName"), "admin");
```

```
await tester.SetValueInElementAsync(tester.GetLocator("locatorName"), "0000");
```

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

SetValueInElementByClassAsync

SetValueInElementByClassAsync

Description: the method inserts value into the specified element

Syntax: SetValueInElementByClassAsync(string _class, int index, string value)

Return value: no return value

Example:

```
await tester.SetValueInElementByClassAsync("my-element", 0, "value");
```

Created with the Personal Edition of HelpNDoc: [Create help files for the Qt Help Framework](#)

SetValueInElementByIdAsync

SetValueInElementByIdAsync

Description: the method inserts value into the specified element

Syntax: SetValueInElementByIdAsync(string id, string value)

Return value: no return value

Example:

```
await tester.SetValueInElementByIdAsync("MyElement", "value");
```

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

SetValueInElementByNameAsync

SetValueInElementByNameAsync

Description: the method inserts value into the specified element

Syntax: SetValueInElementByNameAsync(string name, int index, string value)

Return value: no return value

Example:

```
await tester.SetValueInElementByNameAsync("MyElement", 0, "value");
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

SetValueInElementByTagAsync

SetValueInElementByTagAsync

Description: the method inserts value into the specified element

Syntax: SetValueInElementByTagAsync(string tag, int index, string value)

Return value: no return value

Example:

```
await tester.SetValueInElementByTagAsync("input", 0, "value");
```

Created with the Personal Edition of HelpNDoc: [Write eBooks for the Kindle](#)

Clicking

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

ClickElementAsync

ClickElementAsync

Description: the method performs a click on the element

Syntax:

ClickElementAsync(string by, string locator)

ClickElementAsync(Locator locator)

Return value: no return value

Example:

```
await tester.ClickElementAsync(Tester.BY_CSS, "#auth #buttonLogin");
```

```
await tester.ClickElementAsync(Tester.BY_XPATH,  
"//div[@id='auth']//input[@id='buttonLogin']");
```

```
await tester.ClickElementAsync(tester.GetLocator("locatorName"));
```

ClickElementByClassAsync

ClickElementByClassAsync

Description: the method performs a click on the element

Syntax: ClickElementByClassAsync(string _class, int index);

Return value: no return value

Example:

```
await tester.ClickElementByClassAsync("my-element", 0)
```

ClickElementByIdAsync

ClickElementByIdAsync

Description: the method performs a click on the element

Syntax: ClickElementByIdAsync(string id)

Return value: no return value

Example:

```
await tester.ClickElementByIdAsync("MyElement");
```

ClickElementByNameAsync

ClickElementByNameAsync

Description: the method performs a click on the element

Syntax: ClickElementByNameAsync(string name, int index)

Return value: no return value

Example:

```
await tester.ClickElementByNameAsync("MyElement", 0);
```

ClickElementByTagAsync

ClickElementByTagAsync

Description: the method performs a click on the element

Syntax: ClickElementByTagAsync(string tag, int index)

Return value: no return value

Example:

```
await tester.ClickElementByTagAsync("a", 0);
```

FocusElementAsync

FocusElementAsync

Description: the method switches focus to the specified element

Syntax:

FocusElementAsync(string by, string locator)

FocusElementAsync(Locator locator)

Return value: no return value

Example:

```
await tester.FocusElementAsync(Tester.BY_CSS, "input[id='login']");
```

```
await tester.SetValueInElementAsync(Tester.BY_CSS, "input[id='login']", "admin");
```

```
await tester.FocusElementAsync(Tester.BY_XPATH, "//input[@id='login']");
```

```
await tester.SetValueInElementAsync(Tester.BY_XPATH, "//input[@id='login']", "admin");
```

```
await tester.FocusElementAsync(tester.GetLocatorValue("locatorName"));
```

```
await tester.SetValueInElementAsync(tester.GetLocator("locatorName"), "admin");
```

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

IsClickableElementAsync

IsClickableElementAsync

Description: the method determines the clickability of the element and returns true or false

Syntax:

IsClickableElementAsync(string by, string locator)

IsClickableElementAsync(Locator locator)

Return value: bool (true or false)

Пример:

```
Tester tester = new Tester(browserForm);
```

```
await tester.TestBeginAsync();
```

```
await tester.GoToUrlAsync("https://somovstudio.github.io/test_eng.html", 5);
```

```
bool clickable = await tester.IsClickableElementAsync(Tester.BY_XPATH,  
"//*[@id='buttonLogin']");
```

```
await tester.AssertTrueAsync(clickable);
```

```
await tester.TestEndAsync();
```

```
bool clickable = await tester.IsClickableElementAsync(tester.GetLocator("locatorName"));
```

```
await tester.AssertTrueAsync(clickable);
```

ScrollToElementAsync

ScrollToElementAsync

Description: the method scrolls to the specified element (the behaviorSmooth parameter determines the smoothness of scrolling)

Syntax:

ScrollToElementAsync(string by, string locator, bool behaviorSmooth = false)

ScrollToElementAsync(Locator locator, bool behaviorSmooth = false)

Return value: no return value

Example:

```
await tester.ScrollToElementAsync(Tester.BY_CSS, "body > footer", true);
```

```
await tester.ScrollToElementAsync(Tester.BY_XPATH, "/html/body/footer", true);
```

```
await tester.ScrollToElementAsync(tester.GetLocator("locatorName"), true);
```

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

Objects

Created with the Personal Edition of HelpNDoc: [Produce Kindle eBooks easily](#)

GetElementAsync

GetElementAsync

Description: the method returns an element in the form of an object whose class [HTMLElement](#)

Syntax:

GetElementAsync(string by, string locator)

GetElementAsync(Locator locator)

Return value: object HTMLElement

Example:

```
HTMLElement element = await tester.GetElementAsync(tester.GetLocator("locatorName"));
await element.ClickAsync();
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_CSS, "#auth #buttonLogin");
await element.ClickAsync();
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//div[@id='auth']//input[@id='buttonLogin']");
await element.ClickAsync();
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//*[@id='MyFile']");
tester.ConsoleMsg("ID: " + element.Id);
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//h1");
string text = await element.GetTextAsync();
tester.ConsoleMsg(text);
await element.SetTextAsync("text");
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MyInput']");
await element.SetValueAsync("text");
string value = await element.GetValueAsync();
tester.ConsoleMsg(value);
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//h1");
await element.SetAttributeAsync("class", "my-class");
string attrClass = await element.GetAttributeAsync("class");
tester.ConsoleMsg(attrClass);
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//h1");
string html = await element.GetHtmlAsync();
tester.ConsoleMsg(html);
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//h1");
await element.SetHtmlAsync("<div>text</div>");
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "/html/body/footer");
await element.ScrollToAsync();
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//h1");  
await element.WaitVisibleAsync(2);  
await element.WaitNotVisibleAsync(2);
```

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

GetFrameAsync

GetFrameAsync

Description: the method returns an element in the form of an object whose class

[FRAMEElement](#)

Syntax: GetFrameAsync(int index)

Return value: object FRAMEElement

Example:

```
Tester tester = new Tester(browserForm);
```

```
await tester.TestBeginAsync();
```

```
await tester.GoToUrlAsync("https://somovstudio.github.io/test2.html", 5);
```

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
tester.ConsoleMsg("Index: " + frame.Index);
```

```
tester.ConsoleMsg("Name: " + frame.Name);
```

```
string name = await frame.GetAttributeFromElementAsync(Tester.BY_XPATH,
"//input[@id='login']", "name");
```

```
List<string> values = await frame.GetAttributeFromElementsAsync(Tester.BY_XPATH, "//input",
"name");
```

```
if (values != null)
```

```
{
```

```
    foreach (string attr in values)
```

```
        tester.ConsoleMsg(attr);
```

```
}
```

```
await frame.SetAttributeInElementAsync(Tester.BY_XPATH, "//input[@id='buttonLogin']",
"name", "NameButtonLogin");
```

```
await frame.SetAttributeInElementsAsync(Tester.BY_XPATH, "//input", "name", "test");
```

```
await frame.SetValueInElementAsync(Tester.BY_XPATH, "//input[@id='login']", "Тестировщик");
```

```
string value = await frame.GetValueFromElementAsync(Tester.BY_XPATH,
"//input[@id='login']");
```

```
await frame.ClickElementAsync(Tester.BY_XPATH, "//*[@id='buttonLogin']");
```

```
bool result = await frame.IsClickableElementAsync(Tester.BY_XPATH, "//*[@id='buttonLogin']");
```

```
await frame.ScrollToElementAsync(Tester.BY_XPATH, "//*[@id='buttonLogin']", true);
```

```
int result = await frame.GetCountElementsAsync(Tester.BY_XPATH, "//input");

string html = await frame.GetHtmlFromElementAsync(Tester.BY_XPATH,
"//*[@id='buttonLogin']");

await frame.SetHtmlInElementAsync(Tester.BY_XPATH, "//*[@id='auth']/h2", "<h2>Тестовый
заголовок</h2>");

await frame.WaitNotVisibleElementAsync(Tester.BY_XPATH, "//*[@id='result']", 5);
await frame.ClickElementAsync(Tester.BY_XPATH, "//*[@id='buttonLogin']");
await frame.WaitVisibleElementAsync(Tester.BY_XPATH, "//*[@id='result']", 5);

bool result = await frame.FindElementAsync(Tester.BY_XPATH, "//*[@id='result']", 5);

bool result = await frame.FindVisibleElementAsync(Tester.BY_XPATH, "//*[@id='result']", 5);

string title = await frame.GetTitleAsync();

string url = await frame.GetUrlAsync();

await frame.SetTextInElementAsync(Tester.BY_XPATH, "//*[@id='auth']/h2", "Это тест");
string text = await frame.GetTextFromElementAsync(Tester.BY_XPATH, "//*[@id='auth']/h2");

await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_INDEX, "2");
await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_VALUE, "Mobile");
await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_TEXT, "Other");

string index = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_INDEX);
string value = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_VALUE);
string text = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_TEXT);
```

GetCountElementsAsync

GetCountElementsAsync

Description: the method returns the number of found items

Syntax:

GetCountElementsAsync(string by, string locator)

GetCountElementsAsync(Locator locator)

Return value: int

Example:

```
int count = await tester.GetCountElementsAsync(Tester.BY_CSS, "input");
```

```
int count = await tester.GetCountElementsAsync(Tester.BY_XPATH, "//input");
```

```
int count = await tester.GetCountElementsAsync(tester.GetLocator("locatorName"));
```

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

GetCountElementsByClassAsync

GetCountElementsByClassAsync

Description: the method returns the number of found items

Syntax: GetCountElementsByClassAsync(string _class)

Return value: int

Example:

```
int count = await tester.GetCountElementsByClassAsync("my-element");
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetCountElementsByNameAsync

GetCountElementsByNameAsync

Description: the method returns the number of found items

Syntax: GetCountElementsByNameAsync(string name)

Return value: int

Example:

```
int count = await tester.GetCountElementsByNameAsync("MyElement");
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

GetCountElementsByTagAsync

GetCountElementsByTagAsync

Description: the method returns the number of found items

Syntax: GetCountElementsByTagAsync(string tag)

Return value: int

Example:

```
int count = await tester.GetCountElementsByTagAsync("h2");
```

GetHtmlFromElementAsync

GetHtmlFromElementAsync

Description: the method returns the html representation of the object in a string expression

Syntax:

GetHtmlFromElementAsync(string by, string locator)

GetHtmlFromElementAsync(Locator locator)

Return value: string

Example:

```
string html = await tester.GetHtmlFromElementAsync(Tester.BY_CSS, "#auth > h2");
```

```
string html = await tester.GetHtmlFromElementAsync(Tester.BY_XPATH,  
"//div[@id='auth']//h2");
```

```
string html = await tester.GetHtmlFromElementAsync(tester.GetLocator("locatorName"));
```

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

GetHtmlFromElementByClassAsync

GetHtmlFromElementByClassAsync

Description: the method returns the html representation of the object in a string expression

Syntax: GetHtmlFromElementByClassAsync(string _class, int index)

Return value: string

Example:

```
string html = await tester.GetHtmlFromElementByClassAsync("text-field", 0);
```

Created with the Personal Edition of HelpNDoc: [Full-featured EPub generator](#)

GetHtmlFromElementByIdAsync

GetHtmlFromElementByIdAsync

Description: the method returns the html representation of the object in a string expression

Syntax: GetHtmlFromElementByIdAsync(string id)

Return value: string

Example:

```
string html = await tester.GetHtmlFromElementByIdAsync("login");
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetHtmlFromElementByNameAsync

GetHtmlFromElementByNameAsync

Description: the method returns the html representation of the object in a string expression

Syntax: GetHtmlFromElementByNameAsync(string name, int index)

Return value: string

Example:

```
string html = await tester.GetHtmlFromElementByNameAsync("field", 0);
```

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

GetHtmlFromElementByTagAsync

GetHtmlFromElementByTagAsync

Description: the method returns the html representation of the object in a string expression

Syntax: GetHtmlFromElementByTagAsync(string tag, int index)

Return value: string

Example:

```
string html = await tester.GetHtmlFromElementByTagAsync("h1", 0);
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

IsVisibleElementAsync

IsVisibleElementAsync

Description: the method determines the visibility of the element and returns the value true or false

Syntax:

IsVisibleElementAsync(string by, string locator)

IsVisibleElementAsync(Locator locator)

Return value: bool (true or false)

Example:

```
bool result = await tester.IsVisibleElementAsync(Tester.BY_XPATH, "//*[@id='login']");  
bool result = await tester.IsVisibleElementAsync(Tester.BY_CSS, "#login");  
await tester.AssertTrueAsync(result);
```

```
bool result = await tester.IsVisibleElementAsync(tester.GetLocator("locatorName"));  
await tester.AssertTrueAsync(result);
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

MakeElementVisibleAsync

MakeElementVisibleAsync

Description: the method makes the element visible by adding styles

Syntax:

```
MakeElementVisibleAsync(string by, string locator, string visibility = "visible", int opacity = 1, int index = 1000)
```

```
MakeElementVisibleAsync(Locator locator, string visibility = "visible", int opacity = 1, int index = 1000)
```

Example:

```
await tester.MakeElementVisibleAsync(Tester.BY_XPATH, "//*[@id='button']");
```

```
await tester.MakeElementVisibleAsync(Tester.BY_BY_CSS, "//*[@id='button']");
```

```
await tester.MakeElementVisibleAsync(tester.GetLocator("locatorName"));
```

Created with the Personal Edition of HelpNDoc: [Create help files for the Qt Help Framework](#)

SetHtmlInElementAsync

SetHtmlInElementAsync

Description: the method inserts the html representation of the object into the specified element

Syntax:

SetHtmlInElementAsync(string by, string locator, string html)

SetHtmlInElementAsync(Locator locator, string html)

Return value: no return value

Example:

```
await tester.SetHtmlInElementAsync(Tester.BY_CSS, "#auth > h2", "<div>text</div>");
```

```
await tester.SetHtmlInElementAsync(Tester.BY_XPATH, "//div[@id='auth']/h2",  
"<div>text</div>");
```

```
await tester.SetHtmlInElementAsync(tester.GetLocator("locatorName"), "<div>text</div>");
```

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

SetHtmlInElementByClassAsync

SetHtmlInElementByClassAsync

Description: the method inserts the html representation of the object into the specified element

Syntax: SetHtmlInElementByClassAsync(string _class, int index, string html)

Return value: no return value

Example:

```
await tester.SetHtmlInElementByClassAsync("text-field", 0, "<h1>text</h1>");
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

SetHtmlInElementByIdAsync

SetHtmlInElementByIdAsync

Description: the method inserts the html representation of the object into the specified element

Syntax: SetHtmlInElementByIdAsync(string id, string html)

Return value: no return value

Example:

```
await tester.SetHtmlInElementByIdAsync("auth", "<h1>text</h1>");
```

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

SetHtmlInElementByNameAsync

SetHtmlInElementByNameAsync

Description: the method inserts the html representation of the object into the specified element

Syntax: SetHtmlInElementByNameAsync(string name, int index, string html)

Return value: no return value

Example:

```
await tester.SetHtmlInElementByNameAsync("block", 0, "<h1>text</h1>");
```

SetHtmlInElementByTagAsync

SetHtmlInElementByTagAsync

Description: the method inserts the html representation of the object into the specified element

Syntax: SetHtmlInElementByTagAsync(string tag, int index, string html)

Return value: no return value

Example:

```
await tester.SetHtmlInElementByTagAsync("div", 0, "<h1>text</h1>");
```

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

Waiting

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

WaitAsync

WaitAsync

Description: the method temporarily stops the test execution for the specified number of seconds

Syntax: WaitAsync(int sec)

Return value: no return value

Example:

```
await tester.WaitAsync(5);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

WaitElementInDomAsync

WaitElementInDomAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to appear in the DOM

Syntax:

WaitElementInDomAsync(string by, string locator, int sec)

WaitElementInDomAsync(Locator locator, int sec)

Return value: no return value

Example:

```
await tester.WaitElementInDomAsync(Tester.BY_XPATH, "//div[@id='result']", 5);
```

```
await tester.WaitElementInDomAsync(Tester.BY_CSS, "#result", 5);
```

```
await tester.WaitElementInDomAsync(tester.GetLocator("locatorName"), 5);
```


WaitElementNotDomAsync

WaitElementNotDomAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to cease to be present in the DOM

Syntax:

WaitElementNotDomAsync(string by, string locator, int sec)

WaitElementNotDomAsync(Locator locator, int sec)

Return value: no return value

Example:

```
await tester.WaitElementNotDomAsync(Tester.BY_XPATH, "//div[@id='element']", 5);
```

```
await tester.WaitElementNotDomAsync(Tester.BY_CSS, "#element", 5);
```

```
await tester.WaitElementNotDomAsync(tester.GetLocator("locatorName"), 5);
```

Created with the Personal Edition of HelpNDoc: [Easy CHM and documentation editor](#)

WaitNotVisibleElementAsync

WaitNotVisibleElementAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax:

WaitNotVisibleElementAsync(string by, string locator, int sec)

WaitNotVisibleElementAsync(Locator locator, int sec)

Return value: no return value

Example:

```
await tester.WaitNotVisibleElementAsync(Tester.BY_CSS, "div[id='result']", 2);
```

```
await tester.WaitNotVisibleElementAsync(Tester.BY_XPATH, "//div[@id='result']", 2);
```

```
await tester.WaitNotVisibleElementAsync(tester.GetLocator("locatorName"), 2);
```

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

WaitNotVisibleElementByClassAsync

WaitNotVisibleElementByClassAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax: WaitNotVisibleElementByClassAsync(string _class, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitNotVisibleElementByClassAsync("my-element", 0, 25);
```

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

WaitNotVisibleElementByIdAsync

WaitNotVisibleElementByIdAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax: WaitNotVisibleElementByIdAsync(string id, int sec)

Return value: no return value

Example:

```
await tester.WaitNotVisibleElementByIdAsync("MyElement", 25);
```

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

WaitNotVisibleElementByNameAsync

WaitNotVisibleElementByNameAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax: WaitNotVisibleElementByNameAsync(string name, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitNotVisibleElementByNameAsync("MyElement", 0, 25);
```

Created with the Personal Edition of HelpNDoc: [Easy EBook and documentation generator](#)

WaitNotVisibleElementByTagAsync

WaitNotVisibleElementByTagAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax: WaitNotVisibleElementByTagAsync(string tag, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitNotVisibleElementByTagAsync("h1", 0, 25);
```

Created with the Personal Edition of HelpNDoc: [Easily create EBooks](#)

WaitVisibleElementAsync

WaitVisibleElementAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax:

WaitVisibleElementAsync(string by, string locator, int sec)

WaitVisibleElementAsync(Locator locator, int sec)

Return value: no return value

Example:

```
await tester.WaitVisibleElementAsync(Tester.BY_CSS, "div[id='result']", 2);
```

```
await tester.WaitVisibleElementAsync(Tester.BY_XPATH, "//div[@id='result']", 2);
```

```
await tester.WaitVisibleElementAsync(tester.GetLocator("locatorName"), 2);
```

WaitVisibleElementByClassAsync

WaitVisibleElementByClassAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax: WaitVisibleElementByClassAsync(string _class, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitVisibleElementByClassAsync("my-element", 0, 25);
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

WaitVisibleElementByIdAsync

WaitVisibleElementByIdAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax: WaitVisibleElementByIdAsync(string id, int sec)

Return value: no return value

Example:

```
await tester.WaitVisibleElementByIdAsync("MyElement", 25);
```

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

WaitVisibleElementByNameAsync

WaitVisibleElementByNameAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax: WaitVisibleElementByNameAsync(string name, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitVisibleElementByNameAsync("MyElement", 25);
```

Created with the Personal Edition of HelpNDoc: [Easy EBook and documentation generator](#)

WaitVisibleElementByTagAsync

WaitVisibleElementByTagAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax: WaitVisibleElementByTagAsync(string tag, int index, int sec)

Return value: no return value

Example:

```
await tester.WaitVisibleElementByTagAsync("h1", 25);
```

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

Search

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

FindElementAsync

FindElementAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax:

FindElementAsync(string by, string locator, int sec)

FindElementAsync(Locator locator, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindElementAsync(Tester.BY_CSS, "div[id='result']", 2);
```

```
bool result = await tester.FindElementAsync(Tester.BY_XPATH, "//div[@id='result']", 2);
```

```
bool result = await tester.FindElementAsync(tester.GetLocator("locatorName"), 2);
```

FindElementByClassAsync

FindElementByClassAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax: FindElementByClassAsync(string _class, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindElementByClassAsync("my-element", 0, 5);
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

FindElementByIdAsync

FindElementByIdAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax: FindElementByIdAsync(string id, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindElementByIdAsync("MyElement", 5);
```

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

FindElementByNameAsync

FindElementByNameAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax: FindElementByNameAsync(string name, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindElementByNameAsync("MyElement", 0, 5);
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

FindElementByTagAsync

FindElementByTagAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax: FindElementByTagAsync(string tag, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindElementByTagAsync("h1", 0, 5);
```


FindVisibleElementAsync

FindVisibleElementAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax:

FindVisibleElementAsync(string by, string locator, int sec)

FindVisibleElementAsync(Locator locator, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindVisibleElementAsync(Tester.BY_CSS, "#auth #buttonLogin", 2);  
bool result = await tester.FindVisibleElementAsync(Tester.BY_XPATH,  
"//div[@id='auth']//input[@id='buttonLogin']", 2);
```

```
bool result = await tester.FindVisibleElementAsync(tester.GetLocator("locatorName"), 2);
```

FindVisibleElementByClassAsync

FindVisibleElementByClassAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax: FindVisibleElementByClassAsync(string _class, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindVisibleElementByClassAsync("my-element", 0, 5);
```

Created with the Personal Edition of HelpNDoc: [Full-featured multi-format Help generator](#)

FindVisibleElementByIdAsync

FindVisibleElementByIdAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax: FindVisibleElementByIdAsync(string id, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindVisibleElementByIdAsync("MyElement", 5);
```

Created with the Personal Edition of HelpNDoc: [Easily create EPub books](#)

FindVisibleElementByNameAsync

FindVisibleElementByNameAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax: FindVisibleElementByNameAsync(string name, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindVisibleElementByNameAsync("MyElement", 0, 5);
```

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

FindVisibleElementByTagAsync

FindVisibleElementByTagAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax: FindVisibleElementByTagAsync(string tag, int index, int sec)

Return value: bool (true or false)

Example:

```
bool result = await tester.FindVisibleElementByTagAsync("h1", 0, 5);
```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

Styles

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

GetStyleFromElementAsync

GetStyleFromElementAsync

Description: the method returns the style of the element from the specified property

Syntax:

GetStyleFromElementAsync(string by, string locator, string property)

GetStyleFromElementAsync(Locator locator, string property)

Return value: string

Example:

```
string style = await tester.GetStyleFromElementAsync(Tester.BY_CSS, "#auth", "padding");  
string style = await tester.GetStyleFromElementAsync(Tester.BY_XPATH, "//div[@id='auth']",  
"position");
```

```
string style = await tester.GetStyleFromElementAsync(tester.GetLocator("locatorName"),  
"padding");
```

Created with the Personal Edition of HelpNDoc: [Free Web Help generator](#)

GetStyleFromElementByClassAsync

GetStyleFromElementByClassAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleFromElementByClassAsync(string _class, int index, string property)

Return value: string

Example:

```
string style = await tester.GetStyleFromElementByClassAsync("text-field", 0, "border");
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

GetStyleFromElementByIdAsync

GetStyleFromElementByIdAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleFromElementByIdAsync(string id, string property)

Return value: string

Example:

```
string style = await tester.GetStyleFromElementByIdAsync("buttonLogin", "background-color");
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetStyleFromElementByNameAsync

GetStyleFromElementByNameAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleFromElementByNameAsync(string name, int index, string property)

Return value: string

Example:

```
string style = await tester.GetStyleFromElementByNameAsync("pass", 0, "height");
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetStyleFromElementByTagAsync

GetStyleFromElementByTagAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleFromElementByTagAsync(string tag, int index, string property)

Return value: string

Example:

```
string style = await tester.GetStyleFromElementByTagAsync("h2", 0, "width");
```

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

SetStyleInElementAsync

SetStyleInElementAsync

Description: the method sets the style of the element

Syntax:

SetStyleInElementAsync(string by, string locator, string cssText)

SetStyleInElementAsync(Locator locator, string cssText)

Return value: no return value

Example:

```
await tester.SetStyleInElementAsync(Tester.BY_XPATH, "//div[@id='auth']", "width: 250px; color: white; background-color: #000000;");  
await tester.SetStyleInElementAsync(Tester.BY_CSS, "#auth", "width: 250px; color: white; background-color: #000000;");
```

```
await tester.SetStyleInElementAsync(tester.GetLocator("locatorName"), "width: 250px; color: white; background-color: #000000;");
```

SetStyleInElementByClassAsync

SetStyleInElementByClassAsync

Description: the method sets the style of the element

Syntax: SetStyleInElementByClassAsync(string _class, int index, string cssText)

Return value: no return value

Example:

```
await tester.SetStyleInElementByClassAsync("text-field", 0, "background-color: #123456;");
```

Created with the Personal Edition of HelpNDoc: [Free Web Help generator](#)

SetStyleInElementByIdAsync

SetStyleInElementByIdAsync

Description: the method sets the style of the element

Syntax: SetStyleInElementByIdAsync(string id, string cssText)

Return value: no return value

Example:

```
await tester.SetStyleInElementByIdAsync("buttonLogin", "background-color: #123456;");
```

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

SetStyleInElementByNameAsync

SetStyleInElementByNameAsync

Description: the method sets the style of the element

Syntax: SetStyleInElementByNameAsync(string name, int index, string cssText)

Return value: no return value

Example:

```
await tester.SetStyleInElementByNameAsync("pass", 0, "background-color: #123456;");
```

SetStyleInElementByTagAsync

SetStyleInElementByTagAsync

Description: the method sets the style of the element

Syntax: SetStyleInElementByTagAsync(string tag, int index, string cssText)

Return value: no return value

Example:

```
await tester.SetStyleInElementByTagAsync("h2", 0, "background-color: #123456;");
```

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

Page

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

GetCookiesAsync

GetCookiesAsync

Description: the method returns a list of cookies (document.cookie)..

Syntax: GetCookiesAsync()

Example:

```
List<string> cookies = await tester.GetCookiesAsync();
if (cookies.Count() > 0)
{
    foreach (string cookie in cookies)
    {
        tester.ConsoleMsg("COOKIE: " + cookie);
    }
}
```


GetListRedirectUrlAsync

GetListRedirectUrlAsync

Description: the method returns a list of redirects that occurred when the page was loaded

Syntax: GetListRedirectUrlAsync()

Return value: List

Example:

```
Tester tester = new Tester(browserForm);  
await tester.TestBeginAsync();  
await tester.GoToUrlAsync("https://yandex.ru/", 5);
```

```
List<string> redirects = await tester.GetListRedirectUrlAsync();  
foreach (string url in redirects)  
{  
    tester.ConsoleMsg(url);  
}
```

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

GetTitleAsync

GetTitleAsync

Description: the method returns the page title

Syntax: GetTitleAsync()

Return value: string

Example:

```
string text = await tester.GetTitleAsync();
```

GetUrlAsync

GetUrlAsync

Description: the method returns the current URL

Syntax: GetUrlAsync()

Return value: string

Example:

```
string text = await tester.GetUrlAsync();
```

GetUrlResponseAsync

GetUrlResponseAsync

Description: the method returns the HTTP response of the specified URL

Syntax: GetUrlResponseAsync(string url)

Return value: int

Example:

```
int response = await tester.GetUrlResponseAsync("https://somovstudio.github.io/test.html");  
await tester.AssertEqualsAsync(200, response);
```

Created with the Personal Edition of HelpNDoc: [Free CHM Help documentation generator](#)

GoToUrlAsync

GoToUrlAsync

Description: the method loads the website at the specified URL with the specified wait in seconds

The abortLoadAfterTime flag automatically completes page loading after the timer is completed

Syntax: GoToUrlAsync(string url, int sec, bool abortLoadAfterTime = false)

Return value: no return value

Example:

```
await tester.GoToUrlAsync(@"https://www.google.com/", 25);
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

GoToUrlBaseAuthAsync

GoToUrlBaseAuthAsync

Description: the method loads the website at the specified URL during basic authorization with the specified wait in seconds

The abortLoadAfterTime flag automatically completes page loading after the timer is completed

Syntax: GoToUrlBaseAuthAsync(string url, string login, string pass, int sec, bool abortLoadAfterTime = false)

Return value: no return value

Example:

```
await tester.GoToUrlBaseAuthAsync("https://dev.site.com", "login", "pass", 25);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

LoadPageAsync

LoadPageAsync

Description: the method loads a web page at the specified URL with the specified wait time in seconds. The abort Load After Time flag forcibly terminates page loading after the timer ends. Errors when loading the page are ignored and do not cause the autotest to crash.

Syntax: LoadPageAsync(string url, int sec, bool abortLoadAfterTime = false)

Example:

```
await tester.LoadPageAsync (@"https://www.google.com/", 25, true);
```

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

Text

Created with the Personal Edition of HelpNDoc: [Easily create Web Help sites](#)

GetTextFromElementAsync

GetTextFromElementAsync

Description: the method returns the text from the specified element

Syntax:

GetTextFromElementAsync(string by, string locator)

GetTextFromElementAsync(Locator locator)

Return value: string

Example:

```
string text = await tester.GetTextFromElementAsync(Tester.BY_CSS, "#auth > h2");
```

```
string text = await tester.GetTextFromElementAsync(Tester.BY_XPATH, "//div[@id='auth']/h2");
```

```
string text = await tester.GetTextFromElementAsync(tester.GetLocator("locatorName"));
```

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

GetTextFromElementByClassAsync

GetTextFromElementByClassAsync

Description: the method returns the text from the specified element

Syntax: GetTextFromElementByClassAsync(string _class, int index)

Return value: string

Example:

```
string text = await tester.GetTextFromElementByClassAsync("my-element", 0);
```

GetTextFromElementByIdAsync

GetTextFromElementByIdAsync

Description: the method returns the text from the specified element

Syntax: GetTextFromElementByIdAsync(string id)

Return value: string

Example:

```
string text = await tester.GetTextFromElementByIdAsync("MyElement");
```

Created with the Personal Edition of HelpNDoc: [Free Web Help generator](#)

GetTextFromElementByNameAsync

GetTextFromElementByNameAsync

Description: the method returns the text from the specified element

Syntax: GetTextFromElementByNameAsync(string name, int index)

Return value: string

Example:

```
string text = await tester.GetTextFromElementByNameAsync("MyElement", 0);
```

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

GetTextFromElementByTagAsync

GetTextFromElementByTagAsync

Description: the method returns the text from the specified element

Syntax: GetTextFromElementByTagAsync(string tag, int index)

Return value: string

Example:

```
string text = await tester.GetTextFromElementByTagAsync("h1", 0);
```

Created with the Personal Edition of HelpNDoc: [Easy CHM and documentation editor](#)

SetTextInElementAsync

SetTextInElementAsync

Description: the method inserts text into the specified element

Syntax:

SetTextInElementAsync(string by, string locator, string text)

SetTextInElementAsync(Locator locator, string text)

Return value: no return value

Example:

```
await tester.SetTextInElementAsync(Tester.BY_CSS, "#auth > h2", "text");
```

```
await tester.SetTextInElementAsync(Tester.BY_XPATH, "//div[@id='auth']/h2", "text");
```

```
await tester.SetTextInElementAsync(tester.GetLocator("locatorName"), "text");
```

Created with the Personal Edition of HelpNDoc: [Free help authoring tool](#)

SetTextInElementByClassAsync

SetTextInElementByClassAsync

Description: the method inserts text into the specified element

Syntax: SetTextInElementByClassAsync(string _class, int index, string text)

Return value: no return value

Example:

```
await tester.SetTextInElementByClassAsync("my-element", 0, "текст");
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

SetTextInElementByIdAsync

SetTextInElementByIdAsync

Description: the method inserts text into the specified element

Syntax: SetTextInElementByIdAsync(string id, string text)

Return value: no return value

Example:

```
await tester.SetTextInElementByIdAsync("MyElement", "текст");
```

SetTextInElementByNameAsync

SetTextInElementByNameAsync

Description: the method inserts text into the specified element

Syntax: SetTextInElementByNameAsync(string name, int index, string text)

Return value: no return value

Example:

```
await tester.SetTextInElementByNameAsync("MyElement", 0, "текст");
```

Created with the Personal Edition of HelpNDoc: [Full-featured Kindle eBooks generator](#)

SetTextInElementByTagAsync

SetTextInElementByTagAsync

Description: the method inserts text into the specified element

Syntax: SetTextInElementByTagAsync(string tag, int index, string text)

Return value: no return value

Example:

```
await tester.SetTextInElementByTagAsync("h1", 0, "текст");
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

Methods for executing JavaScript

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

ExecuteJavaScriptAsync

ExecuteJavaScriptAsync

Description: the method executes JavaScript code and returns a string with the result of execution

Syntax: ExecuteJavaScriptAsync(string script)

Return value: string

Example:

```
string script = "(function(){ var element = document.getElementById('MyElement'); return element.innerText; }());";
string result = await tester.ExecuteJavaScriptAsync(script);
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

Methods for executing Rest requests

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

RestGetAsync

RestGetAsync

Description: the method executes a Get Rest request and gets the result in json format

Syntax: RestGetAsync(string url, TimeSpan timeout, string charset = "UTF-8")

Return value: string

Example:

```
string result = await tester.RestGetAsync("https://jsonplaceholder.typicode.com/posts/1/",
TimeSpan.FromDays(1), "UTF-8");
tester.ConsoleMsg(result);
```

This method uses a standard approach:

```
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;

Uri uri = new Uri(url);
HttpClient client = new HttpClient();
client.Timeout = TimeSpan.FromDays(1);
client.BaseAddress = uri;
client.DefaultRequestHeaders.Clear();
client.DefaultRequestHeaders.Accept.Clear();
client.DefaultRequestHeaders.Accept.Add(new
    MediaTypeWithQualityHeaderValue("application/json"));
client.DefaultRequestHeaders.Add("charset", "UTF-8");
client.DefaultRequestHeaders.Add("User-Agent", userAgent);
HttpResponseMessage response = await client.GetAsync(url);
if (response.IsSuccessStatusCode)
{
    return await response.Content.ReadAsStringAsync();
}
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

RestGetBasicAuthAsync

RestGetBasicAuthAsync

Description: the method executes a Get Rest request and gets the result in json format

Syntax: RestGetBasicAuthAsync(string login, string pass, string url, string charset = "UTF-8")

Return value: string

Example:

```
string result = await tester.RestGetBasicAuthAsync("admin", "0000",
    "https://jsonplaceholder.typicode.com/posts/1/", TimeSpan timeout, "UTF-8");
tester.ConsoleMsg(result);
```

This method uses a standard approach:

```
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;

byte[] authToken = Encoding.ASCII.GetBytes($"{login}:{pass}");
Uri uri = new Uri(url);
HttpClient client = new HttpClient();
client.Timeout = TimeSpan.FromDays(1);
client.BaseAddress = uri;
```

```

client.DefaultRequestHeaders.Clear();
client.DefaultRequestHeaders.Accept.Clear();
client.DefaultRequestHeaders.Accept.Add(new
    MediaTypeWithQualityHeaderValue("application/json"));
client.DefaultRequestHeaders.Add("charset", "UTF-8");
client.DefaultRequestHeaders.Add("User-Agent", userAgent);
client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Basic",
    Convert.ToBase64String(authToken));
HttpResponseMessage response = await client.GetAsync(url);
if (response.IsSuccessStatusCode)
{
    return await response.Content.ReadAsStringAsync();
}

```

Created with the Personal Edition of HelpNDoc: [Create iPhone web-based documentation](#)

RestGetStatusCodeAsync

RestGetStatusCodeAsync

Description: the method executes a Get Rest request and as a result receives a status code

Syntax:

RestGetStatusCodeAsync(string url)

RestGetStatusCodeAsync(string login, string pass, string url)

Return value: int

Example:

```

int statusCode = await tester.RestGetAsync("https://jsonplaceholder.typicode.com");
tester.ConsoleMsg(statusCode.ToString());

```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

RestPostAsync

RestPostAsync

Description: the method executes a Post Rest request with sending data in json format and receives the result in the same way in json format

Syntax: RestPostAsync(string url, string json, TimeSpan timeout, string charset = "UTF-8")

Return value: string

Example:

```

string result = await tester.RestPostAsync("https://jsonplaceholder.typicode.com/posts/1/",
    "{}", TimeSpan.FromDays(1), "UTF-8");
tester.ConsoleMsg(result);

```

This method uses a standard approach:

```
using System.Net;
```

```
using System.Net.Http;
```

```
using System.Net.Http.Headers;
```

```
Uri uri = new Uri(url);
HttpClient client = new HttpClient();
client.Timeout = TimeSpan.FromDays(1);
client.DefaultRequestHeaders.Add("charset", "UTF-8");
client.DefaultRequestHeaders.Add("User-Agent", userAgent);
HttpContent content = new StringContent("{} ", Encoding.UTF8, "application/json");
HttpResponseMessage response = await client.PostAsync(uri, content);
if (response.IsSuccessStatusCode)
{
    return await response.Content.ReadAsStringAsync();
}
```

Created with the Personal Edition of HelpNDoc: [Full-featured EPub generator](#)

Methods for measuring the time spent

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

TimerStart

TimerStart

Description: the method starts the countdown and returns the value DateTime

Syntax: TimerStart()

Return value: DateTime

Example:

```
Tester tester = new Tester(browserForm);
DateTime start = await tester.TimerStart();
await tester.TestBeginAsync();
...
await tester.TestEndAsync();
TimeSpan result = await tester.TimerStop(start);
tester.ConsoleMsg("Time " + result.TotalSeconds);
```

Created with the Personal Edition of HelpNDoc: [Create cross-platform Qt Help files](#)

TimerStop

TimerStop

Description: the method completes the countdown and returns the TimeSpan value with the result (for example, 7,132157, which means 7 seconds and 132157 milliseconds)

Syntax: TimerStop(DateTime start)

Return value: TimeSpan

Example:

```

Tester tester = new Tester(browserForm);
DateTime start = await tester.TimerStart();
await tester.TestBeginAsync();
...
await tester.TestEndAsync();
TimeSpan result = await tester.TimerStop(start);
tester.ConsoleMsg("Time " + result.TotalSeconds);

```

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

Methods for sending email and message

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

SendMsgToMailAsync

[SendMsgToMailAsync](#)

Description: the method sends an email

Syntax: SendMsgToMailAsync(string subject, string body, string filename = "")

Return value: no return value

Example:

```

Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
...
await tester.TestEndAsync();
if(tester.GetTestResult() == Tester.PASSED)
{
    await tester.SendMsgToMailAsync("Text message", "The test was completed
successfully");
}

```

Created with the Personal Edition of HelpNDoc: [Easily create HTML Help documents](#)

SendMsgToTelegramAsync

[SendMsgToTelegramAsync](#)

Description: the method sends a message to Telegram. To do this, you need to create a bot, add it to the chat and transfer the bot token (botToken), chat ID (chatId), message text (text) and, if necessary, specify the encoding (charset) to the function

Syntax: SendMsgToTelegramAsync(string botToken, string chatId, string text, string charset = "UTF-8", int timeHourFrom = 0, int timeHourBefore = 0)

Return value: no return value

Example:

```

Tester tester = new Tester(browserForm);

```

```

await tester.TestBeginAsync();
...
await tester.TestEndAsync();
if(tester.GetTestResult() == Tester.FAILED)
{
    // The message will be sent only if the current sending time falls within the range from
    // 9:00 to 21:00
    // if you specify 0, 0, in this case the message will be sent without taking into account
    // the current time
    await
    tester.SendMsgToTelegramAsync("0000000001:ABCDabcd123ABCDabcd123ABCDabcd123ZX",
    "-123456789", "Text message", "UTF-8", 9, 21);
}

```

Created with the Personal Edition of HelpNDoc: [Free iPhone documentation generator](#)

SendMsgToBitrixChatAsync

SendMsgToBitrixChatAsync

Description: the method sends a message to the CRM Bitrix 24 chat

Syntax: SendMsgToBitrixChatAsync(string webhookUrl, string chatId, string messageText, string system = "N")

Return value: string

Example:

```

if(tester.GetTestResult() == Tester.FAILED)
{
    if (DateTime.Now.Hour >= 9 && DateTime.Now.Hour < 21)
    {
        string result = await
        tester.SendMsgToBitrixChatAsync("https://bitrix/rest/123/abcd/im.message.add", "chat10",
        "text");
        tester.SendMessage("BITRIX result: ", Tester.DEBUG, result);
    }
}

```

Created with the Personal Edition of HelpNDoc: [Free help authoring tool](#)

Methods for checking the result

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

AssertEqualsAsync

AssertEqualsAsync

Description: the method performs a check between the actual and expected values, in case of a mismatch, the check will be considered a failure

Syntax: AssertEqualsAsync(dynamic expected, dynamic actual)

Return value: bool (true or false)

Example:

```
string expected = "xyz";
```

```
string actual = "xyz";
```

```
bool result = await tester.AssertEqualsAsync(expected, actual);
```

Created with the Personal Edition of HelpNDoc: [Free EBook and documentation generator](#)

AssertNotEqualsAsync

AssertNotEqualsAsync

Description: the method performs a check between the actual and expected values, in case of a match, the check will be considered a failure

Syntax: AssertNotEqualsAsync(dynamic expected, dynamic actual)

Return value: bool (true or false)

Example:

```
string expected = "abc";
```

```
string actual = "xyz";
```

```
bool result = await tester.AssertNotEqualsAsync(expected, actual);
```

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

AssertTrueAsync

AssertTrueAsync

Description: the method checks the value of true or false, and if the value is false, the check will be considered a failure

Syntax: AssertTrueAsync(bool condition)

Return value: bool (true or false)

Example:

```
bool flag = true;
```

```
bool result = await tester.AssertTrueAsync(flag);
```

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

AssertFalseAsync

AssertFalseAsync

Description: the method checks the value of true or false, and if the value is true, the check will be considered a failure

Syntax: AssertFalseAsync(bool condition)

Return value: bool (true or false)

Example:

```
bool flag = false;
bool result = await tester.AssertFalseAsync(flag);
```

Created with the Personal Edition of HelpNDoc: [Single source CHM, PDF, DOC and HTML Help creation](#)

AssertNotNullAsync

AssertNotNullAsync

Description: the method checks a value that should not be null, and if the value is null, the check will be considered a failure

Syntax: AssertNotNullAsync(dynamic obj)

Return value: bool (true or false)

Example:

```
string text = "текст";
await tester.AssertNotNullAsync(text);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

AssertNullAsync

AssertNullAsync

Description: the method checks the value that should be null, and if the value is not null, the check will be considered a failure

Syntax: AssertNullAsync(dynamic obj)

Return value: bool (true or false)

Example:

```
string text = null;
await tester.AssertNullAsync(text);
```

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

AssertNoErrorsAsync

AssertNoErrorsAsync

Description: the method checks for the absence of errors on the page and if errors are present, the check will be considered a failure

Syntax: AssertNoErrorsAsync(bool showListErrors = false, string[] listIgnored = null)

Return value: bool (true or false)

Example:

```
Tester tester = new Tester(browserForm);
...
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://somovstudio.github.io/test_error.html", 25);
await tester.AssertNoErrorsAsync(true, new string[1] { "stats.g.doubleclick.net" });
await tester.TestEndAsync();
```


AssertNetworkEventsAsync

AssertNetworkEventsAsync

Description: the method checks the presence (presence = true) or absence (presence = false) specified events (events), this method is good to use when checking the events Google Analytics and Yandex Metrika

Syntax: AssertNetworkEventsAsync(bool presence, string[] events)

Return value: bool (true or false)

Example:

```
await tester.AssertNetworkEventsAsync(true, new string[] {  
    "ec=zayavka", "ea=b2c_new_main", "el=some_shpd_nm", "zayavka_shpd"  
});
```

Methods for working with files

FileDownloadAsync

FileDownloadAsync

Description: the method downloads the file at the specified URL

Syntax: FileDownloadAsync(string fileURL, string filename, int waitingSec = 60)

Return value: no return value

Example:

```
await tester.FileDownloadAsync("https://somovstudio.github.io/img/logo.png", "C:\\download\\  
\\logo.png", 60);
```

FileGetHashMD5Async

FileGetHashMD5Async

Description: the method determines the MD5 Hash code of the specified file

Syntax: FileGetHashMD5Async(string filename)

Return value: string

Example:

```
string hash = await tester.FileGetHashMD5Async("C:\\download\\logo.png");
```

FileReadAsync

FileReadAsync

Description: the method reads the specified file

Syntax: FileReadAsync(string encoding, string filename)

Return value: string

Example:

```
string text = await tester.FileReadAsync(Tester.UTF8, "C:\\\\Hat\\file.txt");
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

FileWriteAsync

FileWriteAsync

Description: the method writes text to the specified file

Syntax: FileWriteAsync(string content, string encoding, string filename)

Return value: no return value

Example:

```
string text = "my text";
await tester.FileWriteAsync(text, Tester.UTF8, "C:\\\\Hat\\file.txt");
```

Created with the Personal Edition of HelpNDoc: [Easy to use tool to create HTML Help files and Help web sites](#)

Methods for different tasks

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

CreateHashMD5FromTextAsync

CreateHashMD5FromTextAsync

Description: the method creates a HashDM5 code from the specified text

Syntax: CreateHashMD5FromTextAsync(string text)

Return value: string

Example:

```
string text = "Hello World";
string hash = await tester.CreateHashMD5FromTextAsync(text);
tester.ConsoleMsg(hash);
```

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

GetProjectPath

GetProjectPath

Description: the method returns the path to the project folder

Syntax: GetProjectPath()

Return value: string

Example:

```
string path = tester.GetProjectPath();
tester.ConsoleMsg(path);
```

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

Class: HTMLElement

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

Constructor

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

HTMLElement

[HTMLElement](#)

Description: auxiliary class, an html element object

Syntax: HTMLElement(Tester tester, string by, string locator)

An example of getting an object:

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_CSS, "#auth #buttonLogin");
await element.ClickAsync();
```

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//div[@id='auth']//input[@id='buttonLogin']");
await element.ClickAsync();
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

Constants

Created with the Personal Edition of HelpNDoc: [Free CHM Help documentation generator](#)

BY_INDEX

[BY_INDEX](#)

Description: the constant indicates the type of value being processed

Syntax: BY_INDEX = "BY_INDEX"

Example:

```
HTMLElement.BY_INDEX
```

Created with the Personal Edition of HelpNDoc: [Easily create EBooks](#)

BY_TEXT

[BY_TEXT](#)

Description: the constant indicates the type of value being processed

Syntax: BY_TEXT = "BY_TEXT"

Example:

HTMLElement.BY_TEXT

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

BY_VALUE

BY_VALUE

Description: the constant indicates the type of value being processed

Syntax: BY_VALUE = "BY_VALUE"

Example:

HTMLElement.BY_VALUE

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

Variables

Created with the Personal Edition of HelpNDoc: [Easily create Web Help sites](#)

Id

Id

Description: the variable returns or receives the ID data

Syntax: string Id { get; set; }

Example:

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MyInput']");
tester.ConsoleMsg("ID: " + element.Id);
```

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

Name

Name

Description: the variable returns or receives the Name data

Syntax: string Name { get; set; }

Example:

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MyInput']");
tester.ConsoleMsg("NAME: " + element.Name);
```

Created with the Personal Edition of HelpNDoc: [Full-featured multi-format Help generator](#)

Class

Class

Description: the variable returns or receives Class data

Syntax: string Class { get; set; }

Example:

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MyInput']");
tester.ConsoleMsg("CLASS: " + element.Class);
```

Created with the Personal Edition of HelpNDoc: [Free help authoring tool](#)

Type

Type

Description: the variable returns or receives Type data

Syntax: string Type { get; set; }

Example:

```
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MyInput']");
tester.ConsoleMsg("TYPE: " + element.Type);
```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

Methods

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

ClickAsync

ClickAsync

Description: the method performs a click on the element

Syntax: ClickAsync();

Return value: no return value

Example:

```
await element.ClickAsync();
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

ClickMouseAsync

ClickMouseAsync

Description: the method performs clicking on an element by emulating a mouse

Syntax: ClickMouseAsync()

Return value: no return value

Example:

```
await element.ClickMouseAsync();
```

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

FocusAsync**FocusAsync**

Description: the method switches the focus to the element

Syntax: FocusAsync();

Return value: no return value

Example:

```
await element.FocusAsync();
```

Created with the Personal Edition of HelpNDoc: [Single source CHM, PDF, DOC and HTML Help creation](#)

GetAttributeAsync**GetAttributeAsync**

Description: the method returns the value of the specified attribute from the element

Syntax: GetAttributeAsync(string name);

Return value: string

Example:

```
string attrClass = await element.GetAttributeAsync("class");
```

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

GetHtmlAsync**GetHtmlAsync**

Description: the method returns the html representation of the element in a string expression

Syntax: GetHtmlAsync();

Return value: string

Example:

```
string html = await element.GetHtmlAsync();
```

Created with the Personal Edition of HelpNDoc: [Produce Kindle eBooks easily](#)

GetLocatorAsync**GetLocatorAsync**

Description: the method returns the element's locator

Syntax: GetLocatorAsync()

Return value: string

Example:

```
string locator = await element.GetLocatorAsync();
tester.ConsoleMsg(locator);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

GetOptionAsync

GetOptionAsync

Description: the method returns the index, text, or value of the selected option of the selected option from the select element

Syntax: GetOptionAsync(string by)

Return value: string

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://somovstudio.github.io/test2.html", 5);
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MySelect']");
await element.SelectOptionAsync(HTMLElement.BY_INDEX, "2");
await element.SelectOptionAsync(HTMLElement.BY_VALUE, "Mobile");
await element.SelectOptionAsync(HTMLElement.BY_TEXT, "Other");
string index = await element.GetOptionAsync(HTMLElement.BY_INDEX);
string text = await element.GetOptionAsync(HTMLElement.BY_TEXT);
string value = await element.GetOptionAsync(HTMLElement.BY_VALUE);
...
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Easily create HTML Help documents](#)

GetStyleAsync

GetStyleAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleAsync(string property)

Return value: string

Example:

```
string style = await element.GetStyleAsync("width");
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

GetTextAsync

GetTextAsync

Description: the method returns the text from the element

Syntax: GetTextAsync();

Return value: string

Example:

```
string text = await element.GetTextAsync();
```

Created with the Personal Edition of HelpNDoc: [Free iPhone documentation generator](#)

GetValueAsync**GetValueAsync**

Description: the method returns the value from the element

Syntax: GetValueAsync();

Return value: string

Example:

```
string value = await element.GetValueAsync();
```

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

IsClickableAsync**IsClickableAsync**

Description: the method determines the clickability of the element and returns true or false

Syntax: IsClickableAsync()

Return value: bool (true or false)

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://somovstudio.github.io/test2.html", 5);
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH, "//button");
bool clickable = await element.IsClickableAsync();
await tester.AssertTrueAsync(clickable);
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Free EPub producer](#)

MakeVisibleAsync**MakeVisibleAsync**

Description: the method makes the element visible by adding styles

Syntax: MakeVisibleAsync(string by, string locator, string visibility = "visible", int opacity = 1, int index = 1000)

Example:

```
await tester.MakeVisibleAsync(Tester.BY_XPATH, "//*[@id='button']");
```

```
await tester.MakeVisibleAsync(Tester.BY_BY_CSS, "//*[@id='button']");
```

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

ScrollToAsync

ScrollToAsync

Description: the method scrolls to the element (the behavior Smooth parameter determines the smoothness of scrolling)

Syntax: ScrollToAsync(bool behaviorSmooth = false);

Return value: no return value

Example:

```
await element.ScrollToAsync();
await element.ScrollToAsync(true);
await element.ScrollToAsync(false);
```

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

SelectOptionAsync

SelectOptionAsync

Description: the method selects an option from the select element by the specified index, text, or value.

Syntax: SelectOptionAsync(string by, string value)

Return value: no return value

Example:

```
Tester tester = new Tester(browserForm);
await tester.TestBeginAsync();
await tester.GoToUrlAsync("https://somovstudio.github.io/test2.html", 5);
HTMLElement element = await tester.GetElementAsync(Tester.BY_XPATH,
"//*[@id='MySelect']");
await element.SelectOptionAsync(HTMLElement.BY_INDEX, "2");
await element.SelectOptionAsync(HTMLElement.BY_VALUE, "Mobile");
await element.SelectOptionAsync(HTMLElement.BY_TEXT, "Other");
string index = await element.GetOptionAsync(HTMLElement.BY_INDEX);
string text = await element.GetOptionAsync(HTMLElement.BY_TEXT);
string value = await element.GetOptionAsync(HTMLElement.BY_VALUE);
...
await tester.TestEndAsync();
```

Created with the Personal Edition of HelpNDoc: [Free EPub and documentation generator](#)

SetAttributeAsync

SetAttributeAsync

Description: the method inserts the value of the specified attribute into the element

Syntax: SetAttributeAsync(string name, string value);

Return value: no return value

Example:

```
await element.SetAttributeAsync("class", "my-class");
```

Created with the Personal Edition of HelpNDoc: [Easily create PDF Help documents](#)

SetHtmlAsync**SetHtmlAsync**

Description: the method inserts html into the element

Syntax: SetHtmlAsync(string html);

Return value: no return value

Example:

```
await element.SetHtmlAsync("<div>Это тест</div>");
```

Created with the Personal Edition of HelpNDoc: [Easily create EBooks](#)

SetStyleAsync**SetStyleAsync**

Description: the method sets the style of the element

Syntax: SetStyleAsync(string cssText)

Return value: no return value

Example:

```
await element.SetStyleAsync("width: 250px; background-color: #000000;");
```

Created with the Personal Edition of HelpNDoc: [Free EPub producer](#)

SetTextAsync**SetTextAsync**

Description: the method inserts text into the element

Syntax: SetTextAsync(string text);

Return value: no return value

Example:

```
await element.SetTextAsync("this is a test text");
```

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

SetValueAsync**SetValueAsync**

Description: the method inserts a value into an element

Syntax: SetValueAsync(string value);

Return value: no return value

Example:

```
await element.SetValueAsync("this is a text value");
```

WaitNotVisibleAsync

WaitNotVisible

Description: the method waits for the specified number of seconds until the element is no longer displayed

Syntax: WaitNotVisibleAsync(int sec)

Return value: no return value

Example:

```
await element.WaitNotVisibleAsync(2);
```

WaitVisibleAsync

WaitVisibleAsync

Description: the method waits for the element to be displayed for the specified number of seconds

Syntax: WaitVisibleAsync(int sec)

Return value: no return value

Example:

```
await element.WaitVisibleAsync(2);
```

Class: FRAMEElement

Constructor

FRAMEElement

FRAMEElement

Description: an auxiliary class for working with the frame of the specified index

Syntax: FRAMEElement(Tester tester, int index)

An example of getting an object:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
tester.ConsoleMsg("Index: " + frame.Index);  
tester.ConsoleMsg("Name: " + frame.Name);
```

Constants

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

BY_INDEX

BY_INDEX

Description: the constant indicates the type of value being processed

Syntax: BY_INDEX = "BY_INDEX"

Example:

FRAMEElement.BY_INDEX

Created with the Personal Edition of HelpNDoc: [Easy EBook and documentation generator](#)

BY_TEXT

BY_TEXT

Description: the constant indicates the type of value being processed

Syntax: BY_TEXT = "BY_TEXT"

Example:

FRAMEElement.BY_TEXT

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

BY_VALUE

BY_VALUE

Description: the constant indicates the type of value being processed

Syntax: BY_VALUE = "BY_VALUE"

Example:

FRAMEElement.BY_VALUE

Created with the Personal Edition of HelpNDoc: [Free help authoring tool](#)

Variables

Created with the Personal Edition of HelpNDoc: [Free PDF documentation generator](#)

Name

Name

Description: the variable returns or gets the name of the frame

Syntax: string Name { get; set; }

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
tester.ConsoleMsg("Name: " + frame.Name);
```

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

Index

Index

Description: the variable returns or receives the index of the frame

Syntax: int Index { get; set; }

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
tester.ConsoleMsg("Index: " + frame.Index);
```

Created with the Personal Edition of HelpNDoc: [Benefits of a Help Authoring Tool](#)

Index

Index

Description: the variable returns or receives the index of the frame

Syntax: int Index { get; set; }

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
tester.ConsoleMsg("Index: " + frame.Index);
```

Created with the Personal Edition of HelpNDoc: [Easily create HTML Help documents](#)

Methods

Created with the Personal Edition of HelpNDoc: [Free HTML Help documentation generator](#)

ClickElementAsync

ClickElementAsync

Description: the method performs a click on the element

Syntax: ClickElementAsync(string by, string locator)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
await frame.ClickElementAsync(Tester.BY_CSS, "#auth #buttonLogin");
```

```
await frame.ClickElementAsync(Tester.BY_XPATH,  
"//div[@id='auth']//input[@id='buttonLogin']");
```

Created with the Personal Edition of HelpNDoc: [Qt Help documentation made easy](#)

FindElementAsync

FindElementAsync

Description: the method searches for an element in the DOM with a wait in seconds and returns a logical result true or false

Syntax: FindElementAsync(string by, string locator, int sec)

Return value: bool (true or false)

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
bool result = await frame.FindElementAsync(Tester.BY_CSS, "div[id='result']", 2);
```

```
bool result = await frame.FindElementAsync(Tester.BY_XPATH, "//div[@id='result']", 2);
```

Created with the Personal Edition of HelpNDoc: [Generate Kindle eBooks with ease](#)

FindVisibleElementAsync

FindVisibleElementAsync

Description: the method searches for a visually displayed element with a wait in seconds and returns a logical result true or false

Syntax: FindVisibleElementAsync(string by, string locator, int sec)

Return value: bool (true or false)

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
bool result = await frame.FindVisibleElementAsync(Tester.BY_CSS, "#auth #buttonLogin", 2);
```

```
bool result = await frame.FindVisibleElementAsync(Tester.BY_XPATH,
"//div[@id='auth']//input[@id='buttonLogin']", 2);
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetAttributeFromElementAsync

GetAttributeFromElementAsync

Description: the method returns a lowercase value from the specified attribute in the selected element

Syntax: GetAttributeFromElementAsync(string by, string locator, string attribute)

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
string value = await frame.GetAttributeFromElementAsync(Tester.BY_CSS, "input", "name");
```

```
string value = await frame.GetAttributeFromElementAsync(Tester.BY_XPATH, "//input", "name");
```

Created with the Personal Edition of HelpNDoc: [Free EBook and documentation generator](#)

GetAttributeFromElementsAsync

GetAttributeFromElementsAsync

Description: the method returns a list of values of the specified attribute from a set of elements

Syntax: GetAttributeFromElementsAsync(string by, string locator, string attribute)

Return value: List

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
List<string> values = await frame.GetAttributeFromElementsAsync(Tester.BY_CSS, "input",
"name");
if(values != null)
{
    foreach (string attr in values)
        tester.ConsoleMsg(attr);
}
```

```
List<string> values = await frame.GetAttributeFromElementsAsync(Tester.BY_XPATH, "//input",
"name");
if(values != null)
{
    foreach (string attr in values)
        tester.ConsoleMsg(attr);
}
```

Created with the Personal Edition of HelpNDoc: [Single source CHM, PDF, DOC and HTML Help creation](#)

GetCountElementsAsync

GetCountElementsAsync

Description: the method returns the number of found items

Syntax: GetCountElementsAsync(string by, string locator)

Return value: int

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
int count = await frame.GetCountElementsAsync(Tester.BY_CSS, "input");
```

```
int count = await frame.GetCountElementsAsync(Tester.BY_XPATH, "//input");
```

Created with the Personal Edition of HelpNDoc: [Full-featured multi-format Help generator](#)

GetHtmlFromElementAsync

GetHtmlFromElementAsync

Description: the method returns the html representation of the object in a string expression

Syntax: GetHtmlFromElementAsync(string by, string locator)

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
string html = await frame.GetHtmlFromElementAsync(Tester.BY_CSS, "#auth > h2");
```

```
string html = await frame.GetHtmlFromElementAsync(Tester.BY_XPATH,
"//div[@id='auth']//h2");
```

Created with the Personal Edition of HelpNDoc: [Easily create EPub books](#)

GetOptionAsync

GetOptionAsync

Description: the method returns the index, text, or value of the selected option of the selected option from the select element

Syntax: GetOptionAsync(string by, string locator, string type)

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
string index = await frame.GetOptionAsync(Tester.BY_CSS, "#MySelect",
FRAMEElement.BY_INDEX);
string value = await frame.GetOptionAsync(Tester.BY_CSS, "#MySelect",
FRAMEElement.BY_VALUE);
string text = await frame.GetOptionAsync(Tester.BY_CSS, "#MySelect",
FRAMEElement.BY_TEXT);
```

```
string index = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_INDEX);
string value = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_VALUE);
string text = await frame.GetOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_TEXT);
```

Created with the Personal Edition of HelpNDoc: [Easily create Qt Help files](#)

GetStyleFromElementAsync

GetStyleFromElementAsync

Description: the method returns the style of the element from the specified property

Syntax: GetStyleFromElementAsync(string by, string locator, string property)

Return value: string

Example:


```
FRAMEElement frame = await tester.GetFrameAsync(0);
string style = await frame.GetStyleFromElementAsync(Tester.BY_CSS, "#auth > h2", "width");

string style = await frame.GetStyleFromElementAsync(Tester.BY_XPATH, "//div[@id='auth']",
"width");
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

GetTextFromElementAsync

GetTextFromElementAsync

Description: the method returns the text from the specified element

Syntax: GetTextFromElementAsync(string by, string locator)

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
string text = await frame.GetTextFromElementAsync(Tester.BY_CSS, "#auth > h2");

string text = await frame.GetTextFromElementAsync(Tester.BY_XPATH, "//div[@id='auth']//h2");
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

GetTitleAsync

GetTitleAsync

Description: the method returns the page title

Syntax: GetTitleAsync()

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
string title = await frame.GetTitleAsync();
```

Created with the Personal Edition of HelpNDoc: [Produce electronic books easily](#)

GetUrlAsync

GetUrlAsync

Description: the method returns the current URL

Syntax: GetUrlAsync()

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
string url = await frame.GetUrlAsync();
```

GetValueFromElementAsync

GetValueFromElementAsync

Description: the method returns the value from the specified element

Syntax: GetValueFromElementAsync(string by, string locator)

Return value: string

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
string value = await frame.GetValueFromElementAsync(Tester.BY_CSS, "input[id='login']");
```

```
string value = await frame.GetValueFromElementAsync(Tester.BY_XPATH,  
"//input[@id='login']");
```

IsClickableElementAsync

IsClickableElementAsync

Description: the method determines the clickability of the element and returns true or false

Syntax: IsClickableElementAsync(string by, string locator)

Return value: bool (true or false)

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);  
bool clickable = await frame.IsClickableElementAsync(Tester.BY_CSS, "#buttonLogin");  
await tester.AssertTrueAsync(clickable);
```

```
bool clickable = await frame.IsClickableElementAsync(Tester.BY_XPATH,  
"//*[@id='buttonLogin']");  
await frame.AssertTrueAsync(clickable);
```

IsVisibleElementAsync

IsVisibleElementAsync

Description: the method determines the visibility of the element and returns the value true or false

Syntax: IsVisibleElementAsync(string by, string locator)

Return value: bool (true or false)

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
bool result = await frame.IsVisibleElementAsync(Tester.BY_XPATH, "//*[@id='login']");
```

```
bool result = await frame.IsVisibleElementAsync(Tester.BY_CSS, "#login");
```

```
await frame.AssertTrueAsync(result);
```

Created with the Personal Edition of HelpNDoc: [Easy EBook and documentation generator](#)

MakeVisibleAsync

MakeVisibleAsync

Description: the method makes the element visible by adding styles

Syntax: MakeVisibleAsync(string by, string locator, string visibility = "visible", int opacity = 1, int index = 1000)

Example:

```
await tester.MakeVisibleAsync(Tester.BY_XPATH, "//*[@id='button']");
```

```
await tester.MakeVisibleAsync(Tester.BY_BY_CSS, "//*[@id='button']");
```

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

ScrollToElementAsync

ScrollToElementAsync

Description: the method scrolls to the specified element (the behavior Smooth parameter determines the smoothness of scrolling)

Syntax: ScrollToElementAsync(string by, string locator, bool behaviorSmooth = false)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
await frame.ScrollToElementAsync(Tester.BY_CSS, "body > footer", true);
```

```
await frame.ScrollToElementAsync(Tester.BY_XPATH, "/html/body/footer", true);
```

Created with the Personal Edition of HelpNDoc: [What is a Help Authoring tool?](#)

SelectOptionAsync

SelectOptionAsync

Description: the method selects an option from the select element by the specified index, text, or value

Syntax: SelectOptionAsync(string by, string locator, string type, string value)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
await frame.SelectOptionAsync(Tester.BY_CSS, "#MySelect", FRAMEElement.BY_INDEX, "2");
await frame.SelectOptionAsync(Tester.BY_CSS, "#MySelect", FRAMEElement.BY_VALUE,
"Mobile");
await frame.SelectOptionAsync(Tester.BY_CSS, "#MySelect", FRAMEElement.BY_TEXT, "Other");
```

```
await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_INDEX, "2");
await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_VALUE, "Mobile");
await frame.SelectOptionAsync(Tester.BY_XPATH, "//*[@id='MySelect']",
FRAMEElement.BY_TEXT, "Other");
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

SetAttributeInElementAsync

SetAttributeInElementAsync

Description: the method inserts an attribute with a value into the specified element

Syntax: SetAttributeInElementAsync(string by, string locator, string attribute, string value)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.SetAttributeInElementAsync(Tester.BY_CSS, "#auth > h2", "name", "test");
```

```
await frame.SetAttributeInElementAsync(Tester.BY_XPATH, "///div[@id='auth']/h2", "name",
"test");
```

Created with the Personal Edition of HelpNDoc: [Produce Kindle eBooks easily](#)

SetAttributeInElementsAsync

SetAttributeInElementsAsync

Description: the method inserts an attribute with the specified value into a set of elements and returns a list as a result

Syntax: SetAttributeInElementsAsync(string by, string locator, string attribute, string value)

Return value: List

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
```

```
List<string> values = await frame.SetAttributeInElementsAsync(Tester.BY_CSS, "input", "class",
"test-class");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

```
}
```

```
List<string> values = await frame.SetAttributeInElementsAsync(Tester.BY_XPATH, "//input",
"class", "test-class");
foreach (string value in values)
{
    tester.ConsoleMsg(value);
}
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

SetHtmlInElementAsync

SetHtmlInElementAsync

Description: the method inserts the html representation of the object into the specified element

Syntax: SetHtmlInElementAsync(string by, string locator, string html)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.SetHtmlInElementAsync(Tester.BY_CSS, "#auth > h2", "<div>Тестовый
блок</div>");
```

```
await frame.SetHtmlInElementAsync(Tester.BY_XPATH, "//div[@id='auth']/h2", "<div>Тестовый
блок</div>");
```

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

SetStyleInElementAsync

SetStyleInElementAsync

Description:the method sets the style of the element

Syntax: SetStyleInElementAsync(string by, string locator, string cssText)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.SetStyleInElementAsync(Tester.BY_XPATH, "//div[@id='auth']", "background-color:
#000000;");
```

```
await frame.SetStyleInElementAsync(Tester.BY_CSS, "#auth", "background-color: #000000;");
```

Created with the Personal Edition of HelpNDoc: [Full-featured multi-format Help generator](#)

SetTextInElementAsync

SetTextInElementAsync

Description: the method inserts text into the specified element

Syntax: SetTextInElementAsync(string by, string locator, string text)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.SetTextInElementAsync(Tester.BY_CSS, "#auth > h2", "Тестовый заголовок");
```

```
await frame.SetTextInElementAsync(Tester.BY_XPATH, "///div[@id='auth']/h2", "Тестовый заголовок");
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

SetValueInElementAsync

SetValueInElementAsync

Description: the method inserts a value into the specified element

Syntax: SetValueInElementAsync(string by, string locator, string value)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.SetValueInElementAsync(Tester.BY_CSS, "input[id='login']", "admin");
await frame.SetValueInElementAsync(Tester.BY_CSS, "input[id='pass']", "0000");
```

```
await frame.SetValueInElementAsync(Tester.BY_XPATH, "///input[@id='login']", "admin");
await frame.SetValueInElementAsync(Tester.BY_XPATH, "///input[@id='pass']", "0000");
```

Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

WaitNotVisibleElementAsync

WaitNotVisibleElementAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to stop being displayed

Syntax: WaitNotVisibleElementAsync(string by, string locator, int sec)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.WaitNotVisibleElementAsync(Tester.BY_CSS, "div[id='result']", 2);
```

```
await frame.WaitNotVisibleElementAsync(Tester.BY_XPATH, "///div[@id='result']", 2);
```

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

WaitVisibleElementAsync

WaitVisibleElementAsync

Description: the method temporarily stops the test execution for the specified number of seconds and waits for the requested element to be displayed

Syntax: WaitVisibleElementAsync(string by, string locator, int sec)

Return value: no return value

Example:

```
FRAMEElement frame = await tester.GetFrameAsync(0);
await frame.WaitVisibleElementAsync(Tester.BY_CSS, "div[id='result']", 2);

await frame.WaitVisibleElementAsync(Tester.BY_XPATH, "//div[@id='result']", 2);
```

Created with the Personal Edition of HelpNDoc: [Easily create EPub books](#)

Plugins

Created with the Personal Edition of HelpNDoc: [Full-featured Help generator](#)

HatPluginMySql

Created with the Personal Edition of HelpNDoc: [Easily create HTML Help documents](#)

Installing the plugin

Installing the plugin HatPluginMySql

1. Download and install the browser **Hat**

link: <https://github.com/SomovStudio/Hat/releases>

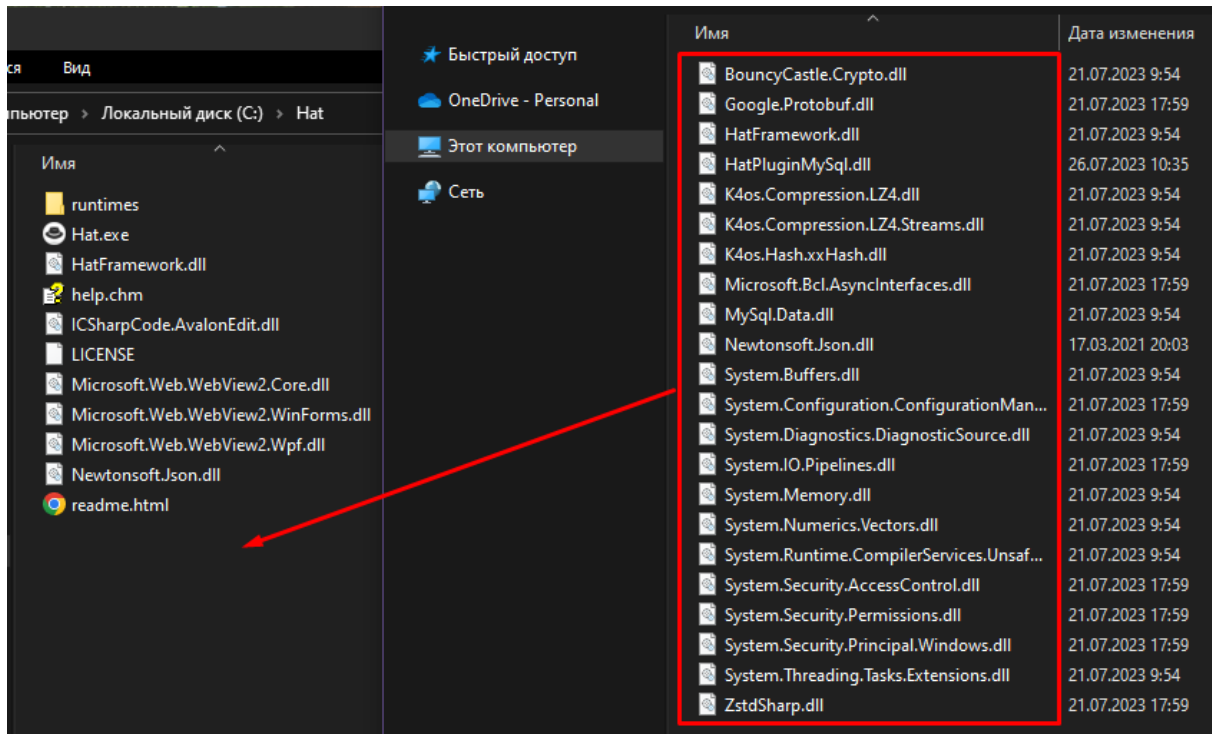
2. Download the plugin **HatPluginMySql**

link: <https://github.com/SomovStudio/HatPluginMySql/releases>

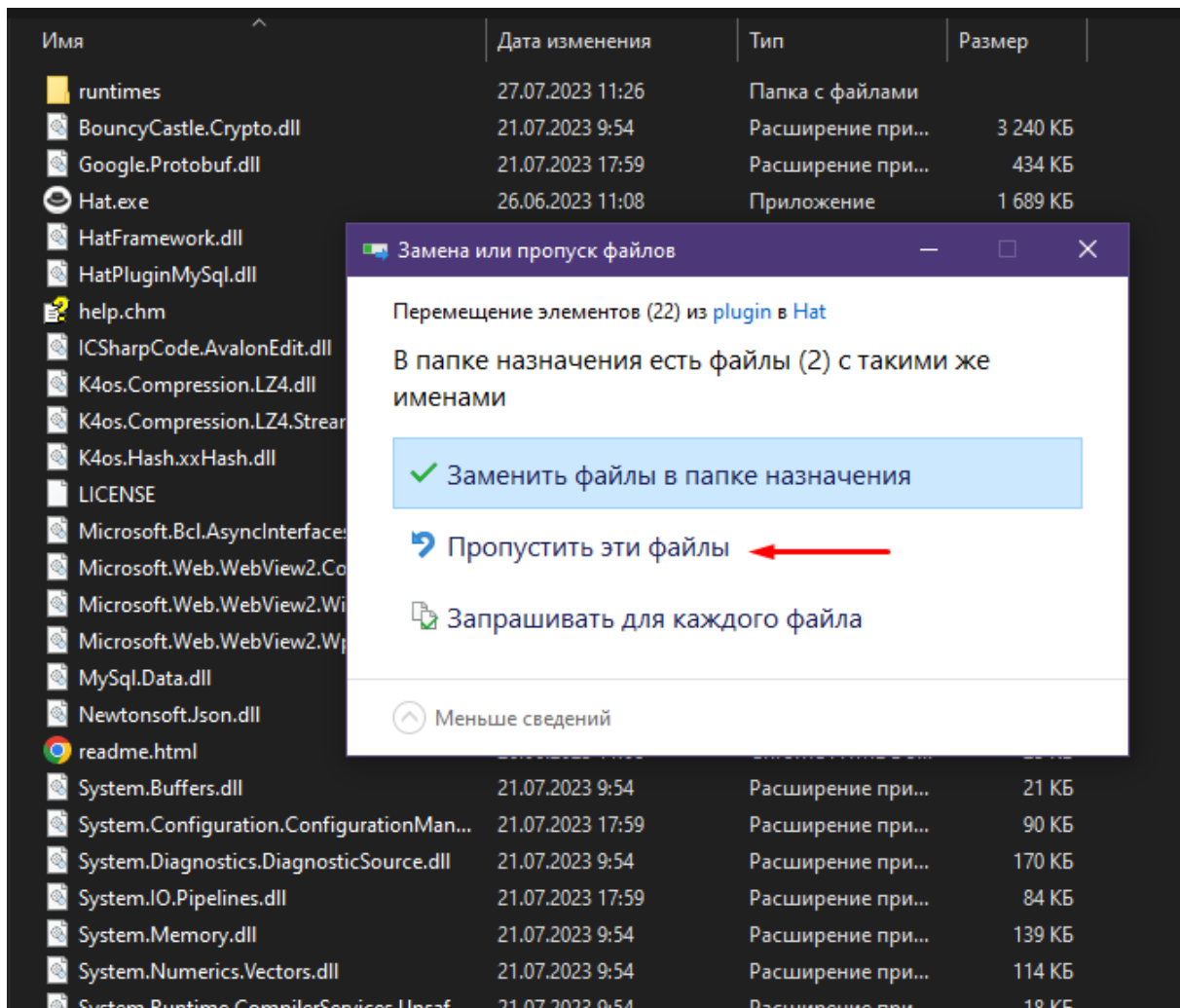
3. Unpack the archive **HatPluginMySql-1.0.0.zip**

Имя	Дата изменения	Тип	Размер
plugin	26.07.2023 10:45	Папка с файлами	
LICENSE	18.07.2023 11:34	Файл	18 КБ
Readme.txt	26.07.2023 10:46	Файл "TXT"	1 КБ

4. Copy the contents of the **plugin** folder to the root of the **Hat** browser folder



When copying, you may be asked about replacing existing files.
In this case, it is recommended to select "Skip these files".



As a result, the following files should be in the root of the folder

Имя	Дата изменения	Тип	Размер
runtimes	27.07.2023 11:26	Папка с файлами	
BouncyCastle.Crypto.dll	21.07.2023 9:54	Расширение при...	3 240 КБ
Google.Protobuf.dll	21.07.2023 17:59	Расширение при...	434 КБ
Hat.exe	26.06.2023 11:08	Приложение	1 689 КБ
HatFramework.dll	26.06.2023 11:06	Расширение при...	325 КБ
HatPluginMySql.dll	26.07.2023 10:35	Расширение при...	26 КБ
help.chm	26.06.2023 11:28	Скомпилирован...	3 308 КБ
ICSharpCode.AvalonEdit.dll	03.04.2023 9:54	Расширение при...	606 КБ
K4os.Compression.LZ4.dll	21.07.2023 9:54	Расширение при...	66 КБ
K4os.Compression.LZ4.Streams.dll	21.07.2023 9:54	Расширение при...	73 КБ
K4os.Hash.xxHash.dll	21.07.2023 9:54	Расширение при...	13 КБ
LICENSE	28.02.2023 14:56	Файл	2 КБ
Microsoft.Bcl.AsyncInterfaces.dll	21.07.2023 17:59	Расширение при...	27 КБ
Microsoft.Web.WebView2.Core.dll	28.05.2023 11:30	Расширение при...	462 КБ
Microsoft.Web.WebView2.WinForms.dll	28.05.2023 11:30	Расширение при...	38 КБ
Microsoft.Web.WebView2.Wpf.dll	28.05.2023 11:30	Расширение при...	44 КБ
MySql.Data.dll	21.07.2023 9:54	Расширение при...	1 144 КБ
Newtonsoft.Json.dll	16.03.2023 13:00	Расширение при...	696 КБ
readme.html	26.06.2023 11:08	Chrome HTML Do...	25 КБ
System Buffers.dll	21.07.2023 9:54	Расширение при...	21 КБ
System.Configuration.ConfigurationMan...	21.07.2023 17:59	Расширение при...	90 КБ
System.Diagnostics.DiagnosticSource.dll	21.07.2023 9:54	Расширение при...	170 КБ
System.IO.Pipelines.dll	21.07.2023 17:59	Расширение при...	84 КБ
System.Memory.dll	21.07.2023 9:54	Расширение при...	139 КБ
System.Numerics.Vectors.dll	21.07.2023 9:54	Расширение при...	114 КБ
System.Runtime.CompilerServices.Unsaf...	21.07.2023 9:54	Расширение при...	18 КБ
System.Security.AccessControl.dll	21.07.2023 17:59	Расширение при...	36 КБ
System.Security.Permissions.dll	21.07.2023 17:59	Расширение при...	30 КБ
System.Security.Principal.Windows.dll	21.07.2023 17:59	Расширение при...	18 КБ
System.Threading.Tasks.Extensions.dll	21.07.2023 9:54	Расширение при...	26 КБ
ZstdSharp.dll	21.07.2023 17:59	Расширение при...	427 КБ

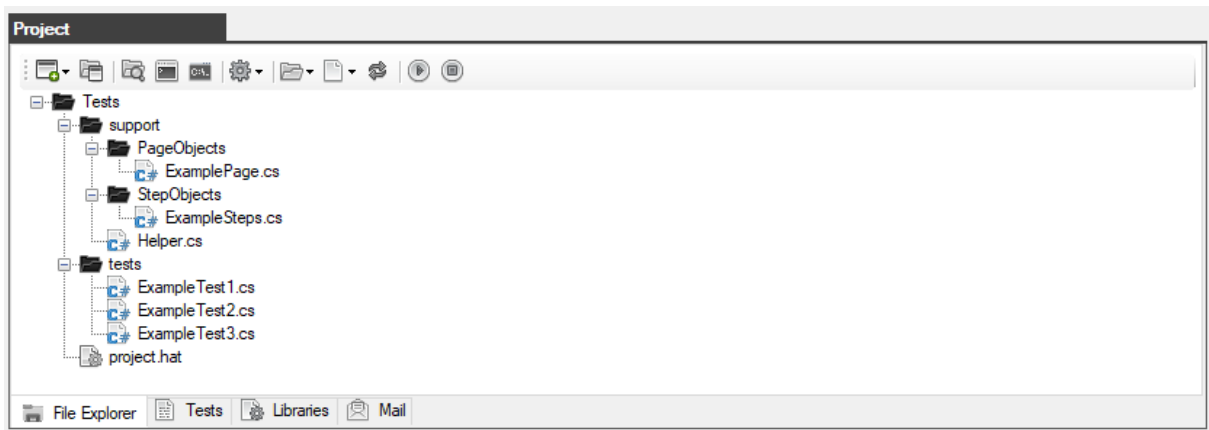
This completes the installation of the plugin!

Created with the Personal Edition of HelpNDoc: [Easily create CHM Help documents](#)

Connecting the plugin to the project

Connecting the HatPluginMySql plugin to the autotest project

1. Launch the browser **Hat**
2. Create a new project or open an existing project



3. Go to the Libraries tab and enter the name of one file at the end of the list
HatPluginMySql.dll



4. Click the "Save" button.

This completes the plug-in connection to the project

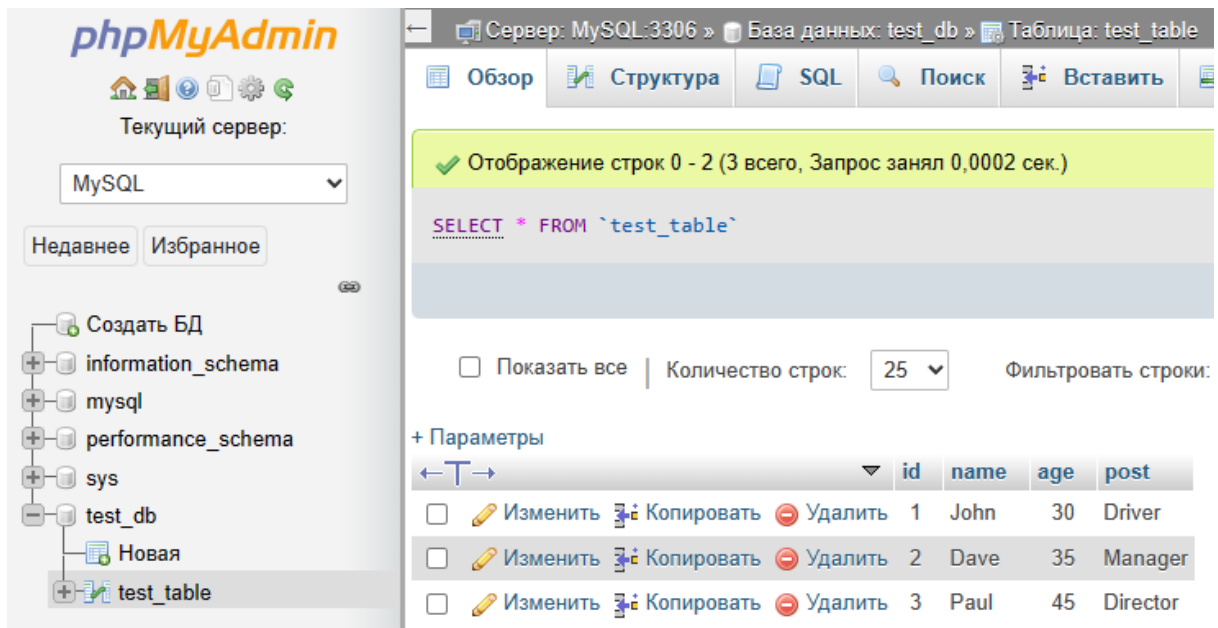
Created with the Personal Edition of HelpNDoc: [Free EBook and documentation generator](#)

An example of an autotest

An example of an autotest

After the plugin has been installed and connected to the project, let's consider a simple autotest.

We have created a simple database in MySQL named **test_db**
 There is one **test_table** table in this database with the fields: **id**, **name**, **age**, **post**



Now let's describe a simple autotest that will add, modify and delete data in the database while checking the correctness of the results.

Файл: ExampleTest3.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;
using HatPluginMySQL;

namespace Hat
{
    public class ExampleTest3
    {
        Tester tester;

        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow);
        }
    }
}
```

```

        await setUp();
        await test();
        await tearDown();
    }

    public async Task setUp()
    {
        tester.Description("Test #3 checking the
database");
        await tester.BrowserFullScreenAsync();
    }

    public async Task test()
    {
        DataTable dataTable;
        List<List<string>> entries = new
List<List<string>>();
        TesterMySQL testerMySQL =
TesterMySQL(tester);

        await tester.TestBeginAsync();

        await
testerMySQL.ConnectionOpenAsync
("server=127.0.0.1;uid=root;pwd=;database=test_db");

        int count = await
testerMySQL.GetCountEntriesAsync("SELECT * FROM test_table");
        if (count > 0)
        {
            entries =
testerMySQL.GetEntriesAsync("SELECT * FROM test_table");
            dataTable =
testerMySQL.GetDataTableAsync("SELECT * FROM test_table");
            foreach (DataRow row in dataTable.Rows)
                foreach (DataColumn col in
dataTable.Columns)
                    tester.ConsoleMsg(row[col].ToString()
);
        }

        await testerMySQL.SetEntryAsync("INSERT INTO
test_table VALUES(NULL, 'I am Tester', 100, 'My post QA')");
        bool result = await
testerMySQL.FindEntryAsync("test_table", "name", "'I am
Tester'");
        await
testerMySQL.AssertHaveInTableAsync("test_table", "name", "'I
am Tester'");
    }

```

```

        entries = await
testerMySQL.GetEntriesFromTableAsync("test_table");
        if (entries != null){
            foreach(List<string> entry in entries)
                foreach(string value in entry)
                    tester.ConsoleMsg(value);
        }

        await testerMySQL.EditEntryAsync("UPDATE
test_table SET age = 111 WHERE name = 'I am Tester'");
        result = await
testerMySQL.FindEntryAsync("test_table", "age", "111");
        await
testerMySQL.AssertHaveInTableAsync("test_table", "age",
"111");

        await testerMySQL.RemoveEntryAsync("DELETE FROM
test_table WHERE name = 'I am Tester'");
        result = await
testerMySQL.FindEntryAsync("test_table", "name", "'I am
Tester'");
        await
testerMySQL.AssertDontHaveInTableAsync("test_table", "name",
"'I am Tester'");

        await testerMySQL.ConnectionCloseAsync();

        await tester.TestEndAsync();
    }

    public async Task tearDown()
    {
        // await tester.BrowserCloseAsync();
    }
}
}

```

Please note that for the autotests to work correctly, you need to connect the library

```
using HatPluginMySQL;
```

First, there is a connection to the database

```

TesterMySQL testerMySQL = new TesterMySQL(tester);
await
testerMySQL.ConnectionOpenAsync
("server=127.0.0.1;uid=root;pwd=;database=test_db");

```

Data manipulation is performed using different methods

```
await testerMySQL.GetCountEntriesAsync("SELECT * FROM
```

```

test_table");
await testerMySql.GetEntriesAsync("SELECT * FROM
test_table");
await testerMySql.GetEntriesFromTableAsync("test_table");
await testerMySql.GetDataTableAsync("SELECT * FROM
test_table");
await testerMySql.SetEntryAsync("INSERT INTO test_table
VALUES(NULL, 'I am Tester', 100, 'My post QA')");
await testerMySql.FindEntryAsync("test_table", "name", "'I
am Tester'");
await testerMySql.EditEntryAsync("UPDATE test_table SET age
= 111 WHERE name = 'I am Tester'");
await testerMySql.RemoveEntryAsync("DELETE FROM test_table
WHERE name = 'I am Tester'");

```

Special methods for checking the presence or absence of data in the database

```

await testerMySql.AssertHaveInTableAsync("test_table",
"name", "'I am Tester'");
await testerMySql.AssertDontHaveInTableAsync("test_table",
"name", "'I am Tester'")

```

At the end, the work with the database is completed by closing the connection to it

```

await testerMySql.ConnectionCloseAsync();

```

Let's run the autotest and look at the progress of the check and the result

Project		
Действие	Статус	Комментарий
Сообщение	В процессе	Запуск автотеста
Сообщение	В процессе	Запущен автотест из файла: ExampleTest3.cs
BrowserFullScreenAsync()	Выполнено	Размер браузера изменён
Тестирование началось	Выполнено	Инициализация теста
Инициализация теста	Выполнено	Выполнена инициализация теста
ConnectionOpenAsync("server=127.0.0.1;uid=root;pwd=;database=test_db")	В процессе	Подключение к базе данных и открытие соединения
ConnectionOpenAsync("server=127.0.0.1;uid=root;pwd=;database=test_db")	Успешно	Подключение к базе данных открыто
GetCountEntriesAsync("SELECT * FROM test_table")	Успешно	В таблице 3 записей
GetEntriesAsync("SELECT * FROM test_table")	Успешно	Получены записи из таблицы
GetDataTableAsync("SELECT * FROM test_table")	Успешно	Получена таблица записей
SetEntryAsync("INSERT INTO test_table VALUES(NULL, 'I am Tester', 100, 'My post QA')")	Успешно	Данные успешно добавлены в базу данных
FindEntryAsync("test_table", "name", "'I am Tester'")	Выполнено	В таблице test_table присутствует запись со значением 'I am Tester' в колонке name
AssertHaveInTableAsync("test_table", "name", "'I am Tester'")	Успешно	В таблице test_table присутствует запись со значением 'I am Tester' в колонке name
GetEntriesFromTableAsync("test_table")	Успешно	Получены все записи из таблицы 'test_table'
EditEntryAsync("UPDATE test_table SET age = 111 WHERE name = 'I am Tester'")	Успешно	Данные успешно обновлены в базу данных
FindEntryAsync("test_table", "age", "'111'")	Выполнено	В таблице test_table присутствует запись со значением 111 в колонке age
AssertHaveInTableAsync("test_table", "age", "'111'")	Успешно	В таблице test_table присутствует запись со значением 111 в колонке age
RemoveEntryAsync("DELETE FROM test_table WHERE name = 'I am Tester'")	Успешно	Данные успешно удалены из базы данных
FindEntryAsync("test_table", "name", "'I am Tester'")	Выполнено	В таблице test_table в колонке name нет записи со значением 'I am Tester'
AssertDontHaveInTableAsync("test_table", "name", "'I am Tester'")	Успешно	В таблице test_table в колонке name нет записи со значением 'I am Tester'
ConnectionCloseAsync()	Успешно	Подключение к базе данных закрыто
Тестирование завершено	Успешно	Тест завершен - все шаги выполнены успешно

All actions were completed, all checks were successful.

The test was completed successfully.

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

Class: TesterMySql

Created with the Personal Edition of HelpNDoc: [Free EBook and documentation generator](#)

Constructor

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

TesterMySql

TesterMySql

Description: the main class of working with a MySQL database

Syntax: TesterMySql(Tester tester)

Example:

```
Tester tester = new Tester(browserWindow);
```

```
TesterMySql testerMySql = new TesterMySql(tester);
```

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

Methods

Created with the Personal Edition of HelpNDoc: [Free help authoring environment](#)

ConnectionOpenAsync

ConnectionOpenAsync

Description: the method opens a connection to the MySQL database

Syntax: ConnectionOpenAsync(string connectionString)

Return value: no return value

Example:

```
await testerMySQL.ConnectionOpenAsync("server=127.0.0.1;uid=root;pwd=;database=test_db");
```

Created with the Personal Edition of HelpNDoc: [Full-featured multi-format Help generator](#)

ConnectionCloseAsync

ConnectionCloseAsync

Description: the method closes the connection to the MySQL database

Syntax: ConnectionCloseAsync()

Return value: no return value

Example:

```
await testerMySQL.ConnectionCloseAsync();
```

Created with the Personal Edition of HelpNDoc: [iPhone web sites made easy](#)

GetCountEntriesAsync

GetCountEntriesAsync

Description: the method returns the number of records in the table after executing the query

Syntax: GetCountEntriesAsync(string sqlQuerySelect)

Return value: int

Example:

```
int count = await testerMySQL.GetCountEntriesAsync("SELECT * FROM test_table");
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

GetEntriesAsync

GetEntriesAsync

Description: the method returns a list of records from the table after executing the query

Syntax: GetEntriesAsync(string sqlQuertSelect)

Return value: List

Example:

```
List<List<string>> entries = new List<List<string>>>();
entries = await testerMySQL.GetEntriesAsync("SELECT * FROM test_table");

foreach(List<string> entry in entries)
    foreach(string value in entry)
        tester.ConsoleMsg(value);
```

GetEntriesFromTableAsync

GetEntriesFromTableAsync

Description: the method returns a list of records from the specified database table

Syntax: GetEntriesFromTableAsync(string tableName)

Return value: List

Example:

```
List<List<string>> entries = new List<List<string>>();  
entries = await testerMySQL.GetEntriesFromTableAsync("test_table");
```

```
foreach(List<string> entry in entries)  
    foreach(string value in entry)  
        tester.ConsoleMsg(value);
```

GetDataTableAsync

GetDataTableAsync

Description: the method returns a table of records from the database table after executing the query

Syntax: GetDataTableAsync(string sqlQuerySelect)

Return value: DataTable

Example:

```
DataTable dataTable = null;
dataTable = await testerMySql.GetDataTableAsync("SELECT * FROM test_table");

foreach (DataRow row in dataTable.Rows)
    foreach (DataColumn col in dataTable.Columns)
        tester.ConsoleMsg(row[col].ToString());
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)

SetEntryAsync

SetEntryAsync

Description: the method executes a query that adds data to the database table and returns the record number

Syntax: SetEntryAsync(string sqlQuertInsert)

Return value: int

Example:

```
int result = await testerMySQL.SetEntryAsync("INSERT INTO test_table VALUES(NULL, 'I am Tester', 100, 'My post QA')");
```

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

EditEntryAsync

EditEntryAsync

Description: the method executes a query that modifies the data in the database table and returns the record number

Syntax: EditEntryAsync(string sqlQuertUpdate)

Return value: int

Example:

```
int result = await testerMySQL.EditEntryAsync("UPDATE test_table SET age = 111 WHERE name = 'I am Tester'");
```

Created with the Personal Edition of HelpNDoc: [Full-featured EBook editor](#)

RemoveEntryAsync

RemoveEntryAsync

Description: the method executes a query that deletes the data in the database table and returns the record number

Syntax: RemoveEntryAsync(string sqlQuertDelete)

Return value: int

Example:

```
int result = await testerMySQL.RemoveEntryAsync("DELETE FROM test_table WHERE name = 'I am Tester'");
```

Created with the Personal Edition of HelpNDoc: [Write EPub books for the iPad](#)

FindEntryAsync

FindEntryAsync

Description: the method searches for data in the specified database table and returns the boolean value of the search result

Syntax: FindEntryAsync(string tableName, string columnName, string value)

Return value: bool (true or false)

Example:

```
bool result = await testerMySQL.FindEntryAsync("test_table", "name", "I am Tester");
```

Created with the Personal Edition of HelpNDoc: [Easily create Help documents](#)

AssertHaveInTableAsync

AssertHaveInTableAsync

Description: the method checks the data in the specified database table and returns the boolean value of the search result, in case of a negative result, the check will be considered a failure

Syntax: AssertHaveInTableAsync(string tableName, string columnName, string value)

Return value: bool (true or false)

Example:

```
bool result = await testerMySQL.AssertHaveInTableAsync("test_table", "name", "I am Tester");
```

Created with the Personal Edition of HelpNDoc: [Free Kindle producer](#)

AssertDontHaveInTableAsync

AssertDontHaveInTableAsync

Description: the method checks the data in the specified database table and returns the boolean value of the search result, in case of a positive result, the check will be considered a failure

Syntax: AssertDontHaveInTableAsync(string tableName, string columnName, string value)

Return value: bool (true or false)

Example:

```
bool result = await testerMySQL.AssertDontHaveInTableAsync("test_table", "name", "I am Tester");
```

Created with the Personal Edition of HelpNDoc: [Free Web Help generator](#)

Practical examples

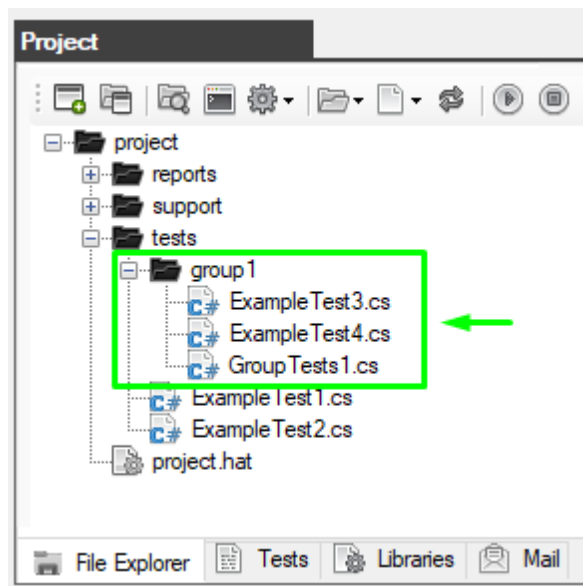
Created with the Personal Edition of HelpNDoc: [Produce online help for Qt applications](#)

Autotest Group

Autotest Group

You can organize a group of autotests and run them.

To do this, create a folder in which the autotests of one group will be stored (for example, the folder group 1)



Next, you need to describe several tests and put them in the **group1** folder.

Attention: for tests, the Main function must be declared as a Task

the first file: ExampleTest3.cs

File: ExampleTest3.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    // Auxiliary class for JSON data
    public class TestJson
    {
        public int userId { get; set; }
        public int id { get; set; }
        public string title { get; set; }
        public string body { get; set; }
    }

    public class ExampleTest3
    {
        // The main variable for the autotest operation
        Tester tester;

        // The main input function is declared as a Task. This is important!
        (the autotest starts working with this function)
        public async Task Main(Form browserWindow)
        {
            // Initialization of the main variable and additional functions
            tester = new Tester(browserWindow);

            await setUp();
            await test();
            await tearDown();
        }

        // The function of getting started with the autotest
        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
        }

        // The function of performing autotest actions
        public async Task test()
        {
            await tester.TestBeginAsync();

```

```

        await
tester.GoToUrlAsync("https://jsonplaceholder.typicode.com", 5);
        string result = await
tester.RestGetAsync(@"https://jsonplaceholder.typicode.com/posts/1/",
    TimeSpan.FromDays(1), "UTF-8");
        TestJson dataJson =
JsonConvert.DeserializeObject<TestJson>(result);
        tester.AssertEqualsAsync("1", dataJson.userId.ToString());
        await tester.TestEndAsync();
    }

    // Autotest shutdown function
    public async Task tearDown()
    {
        // await tester.BrowserCloseAsync(); in this case, we don't need
the browser to close
    }
}
}

```

the second file: ExampleTest4.cs

File: ExampleTest4.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest4
    {
        // The main variable for the autotest operation
        Tester tester;

        // The main input function is declared as a Task. This is important!
(the autotest starts working with this function)
        public async Task Main(Form browserWindow)
        {
            // Initialization of the main variable and additional functions
            tester = new Tester(browserWindow);

            await setUp();
            await test();
        }
    }
}

```

```

        await tearDown();
    }

    // The function of getting started with the autotest
    public async Task setUp()
    {
        await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
    }

    // The function of performing autotest actions
    public async Task test()
    {
        await tester.TestBeginAsync();
        await
tester.GoToUrlAsync("https://somovstudio.github.io/test_eng.html", 5);
        string script = @"(function(){ var element =
document.getElementsByTagName('h2')[0]; return element.outerText; }());";
        string actual = await tester.ExecuteJavaScriptAsync(script);
        string expected = "Authorization";
        await tester.AssertEqualsAsync(expected, actual);
        await tester.TestEndAsync();
    }

    // Autotest shutdown function
    public async Task tearDown()
    {
        // await tester.BrowserCloseAsync(); in this case, we don't need
the browser to close
    }
}

```

We have created two autotests "ExampleTest3.cs" and "ExampleTest4.cs" which are located in the "group1" folder.

Now you need to create a combining autotest GroupTests1.cs that will trigger other autotests.

File: GroupTests1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

```

```

namespace Hat
{
    public class GroupTests1
    {
        // The main variable for the autotest operation
        Tester tester;

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable
            await setUp(); // The function of getting started with the
autotest
            await test1(browserWindow); // The function that calls the
autotest from the script ExampleTest3.cs
            await test2(browserWindow); // The function that calls the
autotest from the script ExampleTest4.cs
            await tearDown(); // Autotest shutdown function
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync();
        }

        // This function runs the autotest from the script ExampleTest3.cs
        public async Task test1(Form browserWindow)
        {
            tester.SendMessage("Автотест ExampleTest3", "", "Файл:
ExampleTest3.cs", Tester.IMAGE_STATUS_MESSAGE);
            ExampleTest3 exampleTest3 = new ExampleTest3(); // An autotest
variable is being created
            await exampleTest3.Main(browserWindow); // The autotest is
started
        }

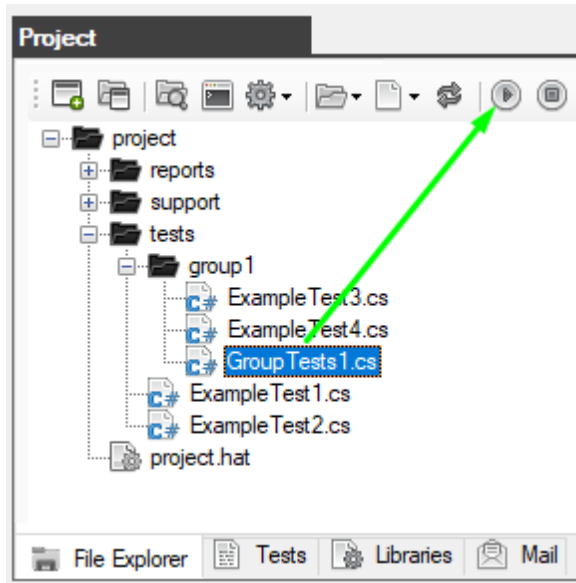
        // This function runs the autotest from the script ExampleTest4.cs
        public async Task test2(Form browserWindow)
        {
            tester.SendMessage("Автотест ExampleTest4", "", "Файл:
ExampleTest4.cs", Tester.IMAGE_STATUS_MESSAGE);
            ExampleTest4 exampleTest4 = new ExampleTest4(); // An autotest
variable is being created
            await exampleTest4.Main(browserWindow); // The autotest is
started
        }

        // Autotest shutdown function
        public async Task tearDown()
        {
            await tester.BrowserCloseAsync(); // Closes the browser
        }
    }
}

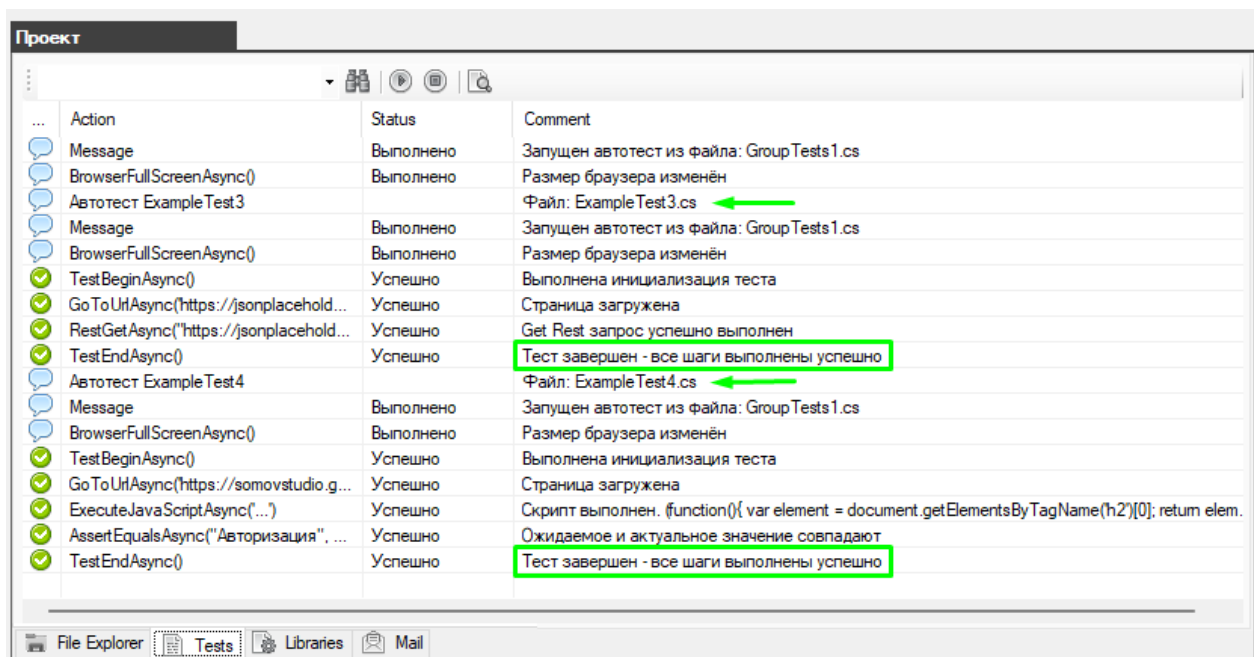
```

As you can see in the ExampleTest3.CS and ExampleTest4.cs autotest will be called sequentially in the test1 and test2 methods.

Run the GroupTests1.cs group autotest



The result on the "Test" tab will reflect the entire progress of the sequential execution of all autotests step by step.



If any of the autotests fails, the check will continue anyway until all the autotests are completed

Project			
Action	Status	Comment	
BrowserFullScreenAsync()	Выполнено	Размер браузера изменён	
Автотест ExampleTest3		Файл: ExampleTest3.cs	
Message	Выполнено	Запущен автотест из файла: GroupTests1.cs	
BrowserFullScreenAsync()	Выполнено	Размер браузера изменён	
TestBeginAsync()	Успешно	Выполнена инициализация теста	
GoToUrlAsync("https://jsonplaceholder...")	Успешно	Страница загружена	
RestGetAsync("https://jsonplaceholder...")	Успешно	Get Rest запрос успешно выполнен	
AssertEqualsAsync(2, 1)	Провально	Ожидаемое и фактическое значение не совпадают	
TestEndAsync()	Провально	Тест завершен - шаги теста выполнены неуспешно	
Автотест ExampleTest4		Файл: ExampleTest4.cs	
Message	Выполнено	Запущен автотест из файла: GroupTests1.cs	
BrowserFullScreenAsync()	Выполнено	Размер браузера изменён	
TestBeginAsync()	Успешно	Выполнена инициализация теста	
GoToUrlAsync("https://somovstudio.g...")	Успешно	Страница загружена	
ExecuteJavaScriptAsync("...")	Успешно	Скрипт выполнен. (function(){ var element = document.getElementsByTagName(h2)[0]; return el	
AssertEqualsAsync("Авторизация", ...)	Успешно	Ожидаемое и фактическое значение совпадают	
TestEndAsync()	Успешно	Тест завершен - все шаги выполнены успешно	

at the same time, the overall result of the GroupTests1.cs group autotest will be considered a failure.

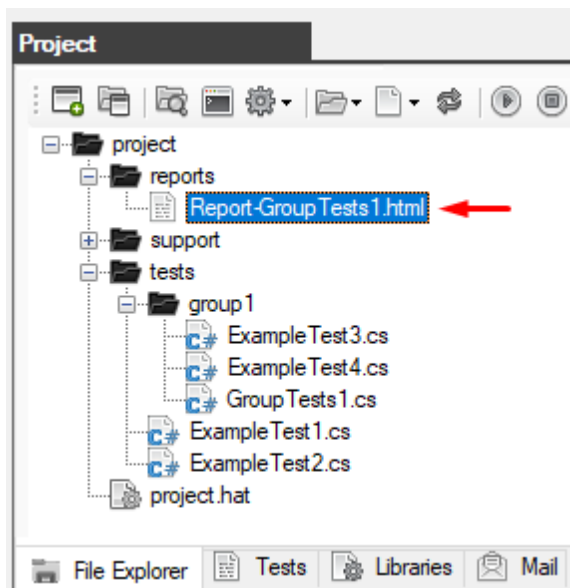
```

Действие: BrowserCloseAsync()
Статус: Выполняется
Комментарий: Браузер закрывается

=====
Tests ended. Finished: FAILURE

```

a report on the complete completion of all tests will also be created



Отчет о работе автотеста

Файл: GroupTests1.cs

Результат: **Провально**

Статус	Действие	Комментарий
Выполняется	AssertEqualsAsync(2, 1)	Проверка совпадения ожидаемого и фактического значения
Провально	AssertEqualsAsync(2, 1)	Ожидаемое и фактическое значение не совпадают
Выполняется	TestEndAsync()	Завершение теста
Провально	TestEndAsync()	Тест завершен - шаги теста выполнены неуспешно
Автотест ExampleTest4		Файл: ExampleTest4.cs
Выполняется	Message	Запуск автотеста
Выполнено	Message	Запущен автотест из файла: GroupTests1.cs
Выполняется	BrowserFullScreenAsync()	Изменяется размер браузера
Выполнено	BrowserFullScreenAsync()	Размер браузера изменен
Выполняется	TestBeginAsync()	Инициализация теста
Успешно	TestBeginAsync()	Выполнена инициализация теста
Выполняется	GoToUriAsync("https://somovstudio.github.io/test.html", 5)	Загрузка страницы
Успешно	GoToUriAsync("https://somovstudio.github.io/test.html", 5)	Страница загружена
Выполняется	ExecuteJavaScriptAsync("...")	Выполнение скрипта. (function(){ var element = document.getElementsByTagName("h2")[0]; return element.outerText; });
Успешно	ExecuteJavaScriptAsync("...")	Скрипт выполнен. (function(){ var element = document.getElementsByTagName("h2")[0]; return element.outerText; });
Выполняется	AssertEqualsAsync("Авторизация", "Авторизация")	Проверка совпадения ожидаемого и фактического значения
Успешно	AssertEqualsAsync("Авторизация", "Авторизация")	Ожидаемое и фактическое значение совпадают
Выполняется	TestEndAsync()	Завершение теста
Успешно	TestEndAsync()	Тест завершен - все шаги выполнены успешно
Ошибок:	2	

Created with the Personal Edition of HelpNDoc: [Qt Help documentation made easy](#)

Executing JavaScript code

Executing JavaScript code

A special ExecuteJavaScriptAsync method has been added to HatFramework to execute JavaScript code.

First, you need to declare a text variable and describe in it the script that you want to execute.

```
string script = @"(function(){
var element = document.getElementsByTagName('h2')[0];
return element.outerText;
})();";
```

Now, using the ExecuteJavaScriptAsync method, we will execute the above script and get the result.

```
string actual = await tester.ExecuteJavaScriptAsync(script);
```

In this example, you will get the text from the H2 element from the page where this script was called.

Full example:

File: ExampleTest4.cs

```
using System;
```

```

using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest4
    {
        Tester tester; // The main variable for the autotest operation

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable

            await setUp(); // The function of getting started with the
autotest

            await test(); // The function of performing test actions
            await tearDown(); // Autotest shutdown function
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
        }

        public async Task test()
        {
            await tester.TestBeginAsync(); // The beginning of the execution
of actions

            await
tester.GoToUrlAsync("https://somovstudio.github.io/test_eng.html", 5); //
Loading the page at the specified address

            // Declaring a variable with a JavaScript script description
            string script = @"(function(){
                var element = document.getElementsByTagName('h2')[0];
                return element.outerText;
            })";

            // Script Execution
            string actual = await tester.ExecuteJavaScriptAsync(script);

```

```

        // Checking the result
        string expected = "Authorization";
        await tester.AssertEqualsAsync(expected, actual); // The
expected and actual data must match

        await tester.TestEndAsync(); // Completing actions
    }

    public async Task tearDown()
    {
        await tester.BrowserCloseAsync(); // Closes the browser
    }
}

```

Examples of JavaScript and jQuery scripts for the method ExecuteJavaScriptAsync :

- focus on the input element and data entry

```

var element = document.evaluate("//input[@id='phone']", document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
element.focus();
element.value = '9999999999';
element.setAttribute("value", "9999999999");

```

- focus on the input element and insert data

```

document.getElementById("phone").focus();
document.execCommand("insertHTML", false, "9999999999");

```

- entering data into input and calling events

```

var element = document.evaluate("//input[@id='phone']", document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
element.value = '9999999999';
element.dispatchEvent(new KeyboardEvent('keydown', { bubbles: true }));
element.dispatchEvent(new KeyboardEvent('keypress', { bubbles: true }));
element.dispatchEvent(new KeyboardEvent('keyup', { bubbles: true }));
element.dispatchEvent(new Event('input', { bubbles: true }));
element.dispatchEvent(new Event('change', { bubbles: true }));

```

- highlight the value in the input

```

var element = document.evaluate("//input[@id='phone']", document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
element.select();

```

- mouse click

```

var clickEvent = new MouseEvent("click", { "view": window, "bubbles": true,
"cancelable": false });
var element = document.evaluate("//input[@id='phone']", document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
element.dispatchEvent(clickEvent);

var theEvent = document.createEvent("MouseEvent");
theEvent.initMouseEvent("click", true, true, window, 0, 0, 0, 0, false,

```

```
false, false, false, 0, null);
element.dispatchEvent(theEvent);
```

- using jQuery to click a button (if the library is connected on the page under test)

```
string script = @"$(function() {
console.log('autotest JQUERY');
$('#btn-send > div.btn > a').click();
});";
```

Created with the Personal Edition of HelpNDoc: [Full-featured Kindle eBooks generator](#)

Processing Json data using Newtonsoft

Processing Json data using Newtonsoft

The Newtonsoft.Json library has been added to work with Json data

The official page: <https://www.newtonsoft.com/json>

As an example, we use a Rest request to the address

<https://jsonplaceholder.typicode.com/posts/1/>

to get test data in Json format:

```
{ "userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati
excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae
consequuntur expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto" }
```

Next, with the help of Newtonsoft.Json processing of the received result.

First, you need to describe a separate class that lists the fields that are present in the Json

```
public class TestJson
{
    public int userId { get; set; }
    public int id { get; set; }
    public string title { get; set; }
    public string body { get; set; }
}
```

We execute a Rest request and get the Json data

```
string result = await
tester.RestGetAsync(@"https://jsonplaceholder.typicode.com/posts/1/",
    TimeSpan.FromDays(1), "UTF-8");
```

We process the received data and output them

```
TestJson dataJson = JsonConvert.DeserializeObject<TestJson>(result);
tester.ConsoleMsg("UserID: " + dataJson.userId.ToString());
```

Full example:

File: ExampleTest3.cs

<code>using System;</code>

```

using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    // Auxiliary class for JSON data
    public class TestJson
    {
        public int userId { get; set; }
        public int id { get; set; }
        public string title { get; set; }
        public string body { get; set; }
    }

    public class ExampleTest3
    {
        Tester tester; // The main variable for the autotest operation

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable

            await setUp(); // Функция начала работы с автотестом
            await test(); // Функция выполнения действий теста
            await tearDown(); // Функция завершения работы с автотестом
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
        }

        public async Task test()
        {
            await tester.TestBeginAsync(); // The beginning of the execution
of actions
            await
tester.GoToUrlAsync("https://jsonplaceholder.typicode.com", 5); // Loading
the page at the specified address

            // Executes a Rest request to the API at the specified URL and

```

```

gets the result in JSON format
    string result = await
tester.RestGetAsync(@"https://jsonplaceholder.typicode.com/posts/1/",
    TimeSpan.FromDays(1), "UTF-8");
    tester.ConsoleMsg(result); // Outputs the result to the console

    // Fetches data from the received JSON
    TestJson dataJson =
JsonConvert.DeserializeObject<TestJson>(result);

    // Outputs data to the console
    tester.ConsoleMsg("UserID: " + dataJson.userId.ToString());
    tester.ConsoleMsg("ID: " + dataJson.id.ToString());
    tester.ConsoleMsg("Title: " + dataJson.title.ToString());
    tester.ConsoleMsg("Body: " + dataJson.body.ToString());

    await tester.TestEndAsync(); // Completing actions
}

public async Task tearDown()
{
    await tester.BrowserCloseAsync(); // Closes browser
}
}
}

```

Created with the Personal Edition of HelpNDoc: [Easily create Web Help sites](#)

Processing XML data for checking the Sitemap

Processing XML data for checking the Sitemap

The following libraries are used to work with XML

```

using System.Collections;
using System.Xml;

```

Let's describe the ReadSitemapXML auxiliary method for reading data from the site map sitemap.xml

This method can be described in the Steps class, but in order for it to be available in any class, it is better to describe it in the Helper.

The blank of the ReadSitemapXML method is closed when reading a link from a file sitemap.xml and as a result, return the list of read links.

File: Helper.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;

```



```

using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;
using System.Collections;
using System.Xml;

namespace Hat
{
    public static class Helper
    {
        /* Method: reading the XML file of the site map */
        public static async Task<List<string>> ReadSitemapXML(Tester tester,
string filename, bool useragent)
        {
            // Declaring a variable whose type is a list (a list for storing
links)
            List<string> list = new List<string>();

            // Declaring a variable that is an error flag (the flag is
disabled by default)
            bool errors = false;

            try
            {
                // Setting up the system connection parameters
                ServicePointManager.Expect100Continue = true;
                ServicePointManager.SecurityProtocol =
SecurityProtocolType.Tls12;
                ServicePointManager.ServerCertificateValidationCallback =
delegate { return true; };

                // We are reading the xml file
                XmlDocument xDoc;
                if (useragent == false)
                {
                    xDoc = new XmlDocument();
                    xDoc.Load(filename);
                }
                else
                {
                    WebClient client = new WebClient();
                    client.Headers["User-Agent"] = await
tester.BrowserGetUserAgentAsync();
                    client.Headers["Accept"] =
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8";
                    string data = client.DownloadString(filename);

                    xDoc = new XmlDocument();
                    xDoc.LoadXml(data);
                }

                // Reading the elements from the resulting xml file
                XmlElement xRoot = xDoc.DocumentElement;
                foreach (XmlNode xnode in xRoot)

```

```

        {
            for (int j = 0; j <= xnode.ChildNodes.Count; j++)
            {
                if (xnode.ChildNodes[j].Name == "loc")
                {
                    // We save the received link to the list
                    list.Add(xnode.ChildNodes[j].InnerText);
                    break;
                }
            }
        }
    }
    catch (Exception ex)
    {
        errors = true; // enabling the error flag
    }

    // Checking the error flag (if the flag is set to true, then a
failure has occurred and the test will be aborted
    await tester.AssertFalseAsync(errors);

    return list; // returning the list of links
}
}
}

```

Now let's describe an autotest in which we will check the site map

File: Sitemap_Test.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class Sitemap_Test
    {
        // The main variable for the autotest operation
        Tester tester;

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {

```

```

        // Initialization of the main variable and additional functions
        tester = new Tester(browserWindow);
        await setUp();
        await test();
        await tearDown();
    }

    // The function of getting started with the autotest
    public async Task setUp()
    {
        await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
    }

    // The function of performing autotest actions
    public async Task test()
    {
        await tester.TestBeginAsync(); // The beginning of the execution
of actions

        // Loading the page at the specified address
        await
tester.GoToUrlAsync("https://somovstudio.github.io/sitemap.xml", 25);

        // Reading links from a file sitemap.xml
        List<string> list = await Helper.ReadSitemapXML(tester,
"https://somovstudio.github.io/sitemap.xml", false);

        // Processing the list of links
        int status = 0;
        foreach (string url in list)
        {
            // Getting the page status (200 - correct, 404 - no page,
502 - the server is not responding, etc.)
            status = await tester.RestGetStatusCodeAsync(url);
            // Output the result to the console
            tester.ConsoleMsg("Status: " + status.ToString() + " | Page:
" + url);

            // Checking the correctness of the result
            await tester.AssertEqualsAsync(200, status);
        }

        await tester.TestEndAsync(); // Completion of actions
    }

    // Функция завершения работы с автотестом
    public async Task tearDown()
    {
        await tester.BrowserCloseAsync(); // Closes the browser
    }
}

```

Alert, prompt and confirm dialog windows

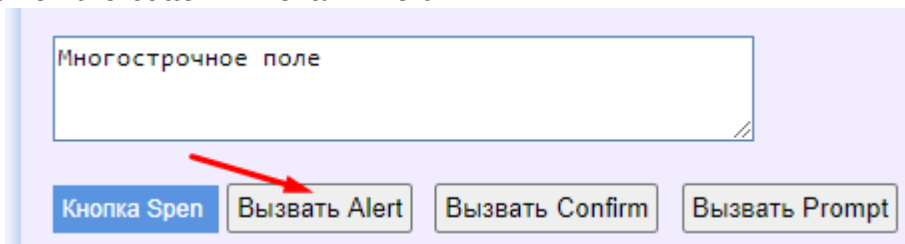
Alert, prompt and confirm dialog windows

The browser has disabled support for the alert, prompt and confirm dialog windows. So that the code is executed ignoring calls to these dialog windows.

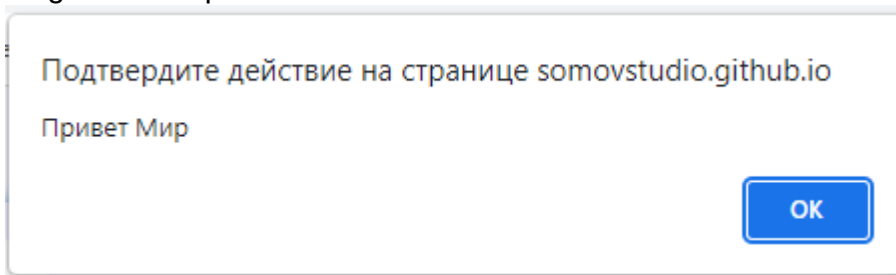
How does it work? Let's take a test page as an example

<https://somovstudio.github.io/test2.html>

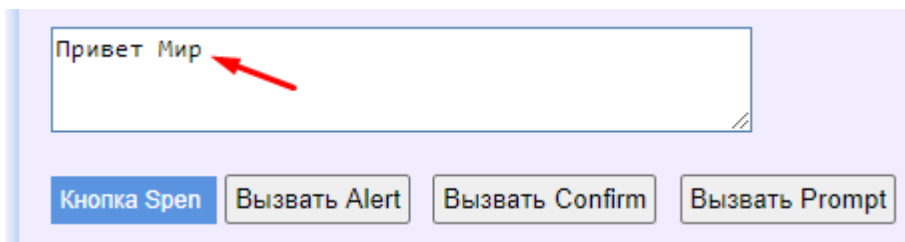
Click on the button "Вызвать Alert"



a dialog window opens



by clicking OK, we will see the message "Привет Мир" in the field



That is, the message "Привет Мир" will be displayed in the field only after clicking OK in the Alert dialog box.

If you press the button "Вызвать Confirm" a question will be asked and the answer will be displayed in the field after clicking on "OK" (or "Cancel")

Подтвердите действие на странице somovstudio.github.io

Вопрос: вы согласны?

OK Отмена

Да (спасибо за ответ)

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

If you press the button "Вызвать Confirm" a question will be asked and the answer will be displayed in the field after entering the number and clicking OK

Подтвердите действие на странице somovstudio.github.io

Какой сейчас год?

2022

OK Отмена

Сейчас 2022 год

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

Now let's do the same with autotest hands

File: ExampleTest5.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
```

```

using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest5
    {
        Tester tester; // The main variable for the autotest operation

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable

            await setUp(); // The function of getting started with the
autotest

            await test(); // The function of performing test actions
            await tearDown(); // Autotest shutdown function
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
        }

        public async Task test()
        {
            await tester.TestBeginAsync(); // The beginning of the execution
of actions

            await
tester.GoToUrlAsync("https://somovstudio.github.io/test2.html", 5); //
Loading the page at the specified address
            await tester.WaitAsync(2); // Waiting for 2 seconds
            await tester.ClickElementByIdAsync("btnAlert"); // Pressing the
button

            await tester.WaitAsync(2); // Waiting for 2 seconds
            await tester.ClickElementByIdAsync("btnConfirm"); // Pressing
the button

            await tester.WaitAsync(2); // Waiting for 2 seconds
            await tester.ClickElementByIdAsync("btnPrompt"); // Pressing the
button

            await tester.TestEndAsync(); // Completing actions
        }

        public async Task tearDown()
        {
            await tester.BrowserCloseAsync(); // Closes the browser
        }
    }
}

```

After running the autotest, we will see that the dialog boxes do not open, and the data changes in the field.

The page will load first in its default state

Многострочное поле

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

Then, autotest, click the "Call Alert" button, the dialog box will not open, and the message "Hello World" will immediately appear in the field

Привет Мир

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

Then, autotest, click the "Call Confirm" button, the dialog box will not open, and the message "No (thanks for the answer)" will immediately appear in the field

Нет (спасибо за ответ)

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

Then, autotest, click the "Call Prompt" button, the dialog box will not open, and the message "Now is a null year" will immediately appear in the field

Сейчас null год

Кнопка Spen Вызвать Alert Вызвать Confirm Вызвать Prompt

As you can see, the browser ignored the dialog boxes and displayed the default values. Thus, dialog boxes do not interfere with the execution of autotests.

Basic authorization

Basic authorization

There are cases when the page is closed with basic authorization

Войдите в систему, чтобы получить доступ к этому сайту

Требуется авторизация для http://test.site.com
Подключение к этому сайту не защищено.

Имя пользователя

Пароль

There are two methods for working with basic authorization:

- BrowserBasicAuthenticationAsync - performs authorization before completing the autotest steps
- GoToUrlBaseAuthAsync - navigates to the page with the username and password specified in the address URL

An example of using the method BrowserBasicAuthenticationAsync

File: ExampleTest6.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest5
    {
        Tester tester; // The main variable for the autotest operation

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable

            await setUp(); // The function of getting started with the
autotest
        }
    }
}
```



```

        await test(); // The function of performing test actions
        await tearDown(); // Autotest shutdown function
    }

    public async Task setUp()
    {
        await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
    }

    public async Task test()
    {
        await tester.TestBeginAsync(); // The beginning of the execution
of actions
        await tester.BrowserBasicAuthenticationAsync("login",
"pass"); // Performing basic authorization
        await tester.GoToUrlAsync("https://test.site.com", 5); //
Loading the page at the specified address
        await tester.TestEndAsync(); // Completing actions
    }

    public async Task tearDown()
    {
        await tester.BrowserCloseAsync(); // Closes the browser
    }
}

```

An example of using the method GoToUrlBaseAuthAsync

File: ExampleTest6.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest5
    {
        Tester tester; // The main variable for the autotest operation
        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {

```

```

        tester = new Tester(browserWindow); // Initializing the main
variable
        await setUp(); // The function of getting started with the
autotest
        await test(); // The function of performing test actions
        await tearDown(); // Autotest shutdown function
    }

    public async Task setUp()
    {
        await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen
    }

    public async Task test()
    {
        await tester.TestBeginAsync(); // The beginning of the execution
of actions

        // Performing basic authorization when loading a page
        await tester.GoToUrlBaseAuthAsync("https://test.site.com",
"login", "pass", 5);

        await tester.TestEndAsync(); // Completing actions
    }

    public async Task tearDown()
    {
        await tester.BrowserCloseAsync(); // Closes the browser
    }
}

```

Created with the Personal Edition of HelpNDoc: [Easily create PDF Help documents](#)

Google analytics and yandex metrika events

Google analytics and yandex metrika events

To get GA and YM events, use special methods.

```

await tester.AssertNetworkEventsAsync(true, new string[] {
    "ec=ga_category", "ea=ga_action", "el=ga_label", "yandex_event"
});
await Helper.ClearBrowserNetworkLogs(tester); // clearing events

```

Another way to intercept the YM event is to use it **?_ym_debug=1** in the address and browser event handling.

File: ExampleTest7.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Windows.Forms;

```

```

using System.Threading;
using System.Threading.Tasks;
using System.IO;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Reflection;
using Newtonsoft.Json;
using HatFramework;

namespace Hat
{
    public class ExampleTest5
    {
        Tester tester; // The main variable for the autotest operation
        List<string> events; // The variable into which all browser events
are collected

        // The main input function (the autotest starts with this function)
        public async void Main(Form browserWindow)
        {
            tester = new Tester(browserWindow); // Initializing the main
variable

            await setUp(); // The function of getting started with the
autotest

            await test(); // The function of performing test actions
            await tearDown(); // Autotest shutdown function
        }

        // A function that listens to browser events
        private void browserConsoleEvents(object sender,
Microsoft.Web.WebView2.Core.CoreWebView2DevToolsProtocolEventReceivedEventArgs e)
        {
            events.Add(e.ParameterObjectAsJson);
            tester.ConsoleMsg(e.ParameterObjectAsJson);
        }

        public async Task setUp()
        {
            await tester.BrowserFullScreenAsync(); // Sets the browser
resolution to full screen

            /*
             * Let's turn on listening for browser events
             * A source: https://chromedevtools.github.io/devtools-
protocol/tot/Network/
            */
            tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Log.entryAdded").DevToolsProtocolEventReceived += browserConsoleEvents;
            await
tester.BrowserView.CoreWebView2.CallDevToolsProtocolMethodAsync
("Log.enable", "{}");

```

```

        tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Console.messageAdded").DevToolsProtocolEventReceived +=
browserConsoleEvents;
        await
tester.BrowserView.CoreWebView2.CallDevToolsProtocolMethodAsync
("Console.enable", "{}");
        tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Runtime.consoleAPICalled").DevToolsProtocolEventReceived +=
browserConsoleEvents;
        tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Runtime.exceptionThrown").DevToolsProtocolEventReceived +=
browserConsoleEvents;
        await
tester.BrowserView.CoreWebView2.CallDevToolsProtocolMethodAsync
("Runtime.enable", "{}");
tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Network.dataReceived").DevToolsProtocolEventReceived +=
browserConsoleEvents;
        await
tester.BrowserView.CoreWebView2.CallDevToolsProtocolMethodAsync
("Network.enable", "{}");
        tester.BrowserView.CoreWebView2.GetDevToolsProtocolEventReceiver
("Debugger.scriptParsed").DevToolsProtocolEventReceived +=
browserConsoleEvents;
        await
tester.BrowserView.CoreWebView2.CallDevToolsProtocolMethodAsync
("Debugger.enable", "{}");
    }

    public async Task test()
    {
        await tester.TestBeginAsync(); // The beginning of the execution
of actions
        await tester.GoToUrlAsync("https://test.site.com/?ym_debug=1",
5); // Loading a page with YM event debugging enabled

        await tester.SetValueInElementAsync(Tester.BY_XPATH,
"//input[@id='SECOND_NAME']", Helper.TestName);
        await tester.WaitAsync(1);
        await tester.SetValueInElementAsync(Tester.BY_XPATH,
"//input[@id='NAME']", Helper.TestName);
        await tester.WaitAsync(1);
        await tester.SetValueInElementAsync(Tester.BY_XPATH,
"//input[@id='PHONE']", Helper.TestPhone);
        await tester.WaitAsync(1);
        await tester.ClickElementAsync(Tester.BY_XPATH,
"//input[@id='fb_close']");
        await tester.WaitAsync(1);
        await tester.WaitVisibleElementAsync(Tester.BY_XPATH,
"//p[text() = 'Спасибо! Ваша заявка принята.']", 25);
        await tester.WaitAsync(5);

        // Checking the YM event in the browser event list
        tester.SendMessage("Message", Tester.COMPLETED, "Проверка:
Событие YM (Goal id: yandex_event)", Tester.IMAGE_STATUS_MESSAGE);
        bool checkYM = false;
        foreach(string value in events)
        {
            checkYM = value.Contains("Goal id: yandex_event");

```

```

        if (checkYM == true)
        {
            tester.SendMessage("Message", Tester.COMPLETED,
"Сработало событие YM: " + value, Tester.IMAGE_STATUS_MESSAGE);
            break;
        }
    }
    await tester.AssertTrueAsync(checkYM); // The result of the
check

    await tester.TestEndAsync(); // Completing actions
}

public async Task tearDown()
{
    await tester.BrowserCloseAsync(); // Closes the browser
}

}
}

```

Thus, the autotest listens to all browser events and, after performing the actions, looks through all events in search of new ones
If the YM event is found, the test is considered successful.

Created with the Personal Edition of HelpNDoc: [Easily create iPhone documentation](#)

Version 1.3 (not current)

Created with the Personal Edition of HelpNDoc: [Full-featured Documentation generator](#)

Description

Starting with browser version 1.3, the following methods have been changed:

- o SendMessage - the method of change
- o SendMessageDebug - the method of change
- o EditMessage - the method has been deleted
- o EditMessageDebug - the method has been deleted
- o DefineTestStop -the method of change

Created with the Personal Edition of HelpNDoc: [Easy EPub and documentation editor](#)

SendMessage [changed since version 1.3.0]

SendMessage

Description: the method displays messages in the test execution process table

When executed, action is output to the system console if status is FAILED, WARNING, "", null, otherwise only status and comment are output (without action).

Syntax: SendMessage(string action, string status, string comment)

Example:

```
int step = tester.SendMessage("the text of the actions", Tester.PROCESS, "the text of the comment");
```

```
/* Examples of console output */
```

```
// action comment
```

```
int step = tester.SendMessage("actions", null, "comment");
```

```
int step = tester.SendMessage("actions", "", "comment");
```

```
// Step[progress]: comment
```

```
int step = tester.SendMessage("actions", Tester.PROCESS, "comment");
```

```
// Step[passed]: comment
```

```
int step = tester.SendMessage("actions", Tester.PASSED, "comment");
```

```
// Step[completed]: comment
```

```
int step = tester.SendMessage("actions", Tester.COMPLETED, "comment");
```

```
// Step[stopped]: comment
```

```
int step = tester.SendMessage("actions", Tester.STOPPED, "comment");
```

```
// Step[failed]: действие комментарий
```

```
int step = tester.SendMessage("actions", Tester.FAILED, "comment");
```

```
// Step[warning]: действие комментарий
```

```
int step = tester.SendMessage("actions", Tester.WARNING, "comment");
```

Created with the Personal Edition of HelpNDoc: [News and information about help authoring tools and software](#)

SendMessageDebug [changed since version 1.3.0]

SendMessageDebug

Description: the method outputs a debugging message that can be disabled for output to a report and an email

When executed, action is output to the system console if status is FAILED, WARNING, "", null, otherwise only status and comment are output (without action).

Syntax: SendMessageDebug(string actionRus, string actionEng, string status, string commentRus, string commentEng, int image)

Example:

```
int step = tester.SendMessageDebug("действия", "action", Tester.PROCESS, "комментарий", "comment", Tester.IMAGE_STATUS_PROCESS);
```

Created with the Personal Edition of HelpNDoc: [Free Qt Help documentation generator](#)

DefineTestStop [changed since version 1.3.0]**DefineTestStop**

Description: the method checks the status of the process (stopped or not)

Syntax: DefineTestStop()

Return value: bool (true or false)

Example:

```
int step = tester.SendMessage("the text of the actions", Tester.FAILED, "the text of the  
comment");
```

```
if (tester.DefineTestStop(step) == true) return; // so the test is stopped
```

Created with the Personal Edition of HelpNDoc: [Create HTML Help, DOC, PDF and print manuals from 1 single source](#)
